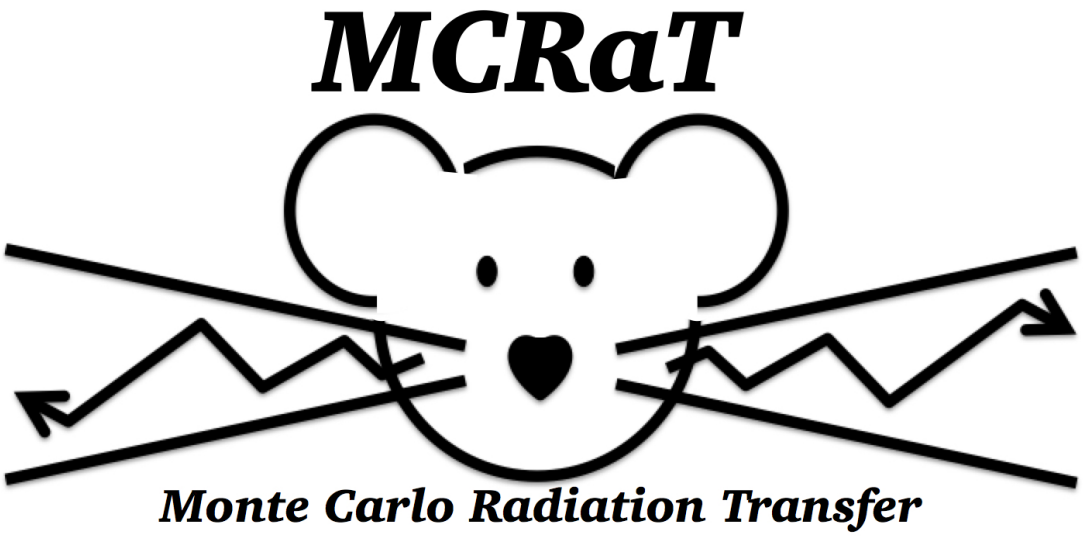


Monte Carlo Radiation Transfer (MCRaT) User's Guide

Tyler Parsotan

Contents

1 Introduction **3**

2 Getting Started **3**
2.1 Required Libraries . . . 3
2.2 Downloading and Compiling MCRaT . . . 4
2.3 The mcrat_input.h file . . . 4
2.4 The MCRaT Parameter File . . . 7
2.5 Understanding MCRaT's Parallelism . . . 9
2.6 Running MCRaT . . . 9
2.7 MCRaT Output Files . . . 10
2.8 Restarting MCRaT . . . 11
2.8.1 Using The mcrat_msp.sh Shell Script . . . 11
2.9 Using Hybrid-Parellelization . . . 12
2.10 Post-Processing MCRaT Data . . . 12

3 The MCRaT Code **13**
3.1 Assumptions about the Hydro Simulation . . . 13
3.2 Algorithm . . . 13
3.3 Saved Quantities . . . 17

3.4 Post-processing Algorithm 20

4 Physics Included in MCRaT 20
4.1 Background: Monte Carlo Radiation Transport 20
4.1.1 What is Monte Carlo radiation transport? 20
4.1.2 Photon packets and Monte Carlo estimators 20
4.1.3 Continuous absorption and weight attenuation 21
4.2 Injecting Photons 21
4.3 Scattering Photons 23
4.4 Polarization 24
4.5 Cyclo-synchrotron Radiation 27
4.5.1 Emitting Cyclo-synchrotron (CS) Photons 27
4.5.2 Calculating B 30
4.5.3 Rebinning The Scattered CS Photons 31
4.5.4 Absorption of CS Photons 32
4.6 Non-Thermal Synchrotron Emission, Self-Absorption, and Scattering Bias 32
4.6.1 Roadmap: the life cycle of a synchrotron packet 33
4.6.2 Enabling and Initializing the Module 33
4.6.3 Non-Thermal Electron Distribution 36
4.6.4 Universal Spectral Tables 37
4.6.5 Photon Emission 41
4.6.6 Synchrotron Self-Absorption 50
4.6.7 Hot Scattering Cross Sections 56
4.6.8 Scattering Bias and Importance Sampling 58
4.6.9 Russian Roulette for Low-Weight Packets 65
4.6.10 Summary of the Call Chain 69

5 User-Defined Injection and Outflows 70
5.1 Custom Photon Injection (`custom_spectrum.c`) 70
5.1.1 How MCRaT Uses These Functions 70
5.1.2 The `custom_photon_sampler` Function 70
5.1.3 What the User May Set, and the Contract with MCRaT 71
5.1.4 The `custom_spectrum` Helper Function 72
5.1.5 A Minimal Example 72
5.1.6 Enabling Custom Photon Injection 73
5.2 Custom Analytic Outflows (`custom_outflow.c`) 73
5.2.1 The `customOutflowPrep` Function 73
5.2.2 Consistency Requirements 74
5.2.3 Interaction with the Magnetic Field, Non-Thermal Electrons, and Scattering Bias 74
5.2.4 The Example Implementation 76
5.2.5 Enabling the Custom Outflow 77

1 Introduction

The Monte Carlo Radiation Transfer (MCRaT; pronounced *Em-Cee-Rat*) code is a next generation radiation transfer code that can be used to analyze the radiation signature expected from astrophysical outflows. The code is written in C and uses the Message Passing Interface (MPI) library for inter-process communication, the Open Multi-Processor (OpenMP) library for intra-node communication, the GNU Scientific Library, and the HDF5 library for parallel I/O.

MCRaT injects photons into a hydro simulation, using the FLASH or the PLUTO codes, and individually propagates and compton scatters the photons through the fluid until the end of the simulation. This process of injection and propagating occurs for a user specified number of times until there are no more photons to be injected. Users can then construct light curves and spectra with the MCRaT calculated results. The hydro simulation used in this version of MCRaT must be in 2D, however, the photon propagation and scattering is done in 3D by assuming cylindrical symmetry.

The code was initially written in python by Dr. Davide Lazzati as a proof of concept. The code was then translated into C by Tyler Parsotan and made to use the OpenMP, MPI, HDF5, and GNU Scientific libraries. MCRaT is highly parallelized and is easy to set up and use.

There are also python files with documented functions to process the MCRaT output located at: `https://github.com/parsotat/ProcessMCRaT`.

In using MCRaT and the ProcessMCRaT codes, we ask that you cite the following papers: Lazzati (2016); Parsotan & Lazzati (2018); Parsotan et al. (2018, 2020).

2 Getting Started

This section describes how to get the MCRaT code as well as the required libraries necessary to compile MCRaT. This section will also cover the necessary steps to run the compiled MCRaT code.

2.1 Required Libraries

MCRaT requires a number of libraries in order to compile properly. These are:

- MPI
- OpenMP
- GNU Scientific Library (GSL)
- HDF5

The MPI library can be an any implementation such as OpenMPI (open-mpi.org) or one that is specific to a given supercomputer. Following the steps to install the library should allow it to work properly for compiling MCRaT. The OpenMP library (openmp.org) is automatically included with Intel and GNU compilers. The GNU Scientific Library (GSL) can be downloaded from gnu.org/software/gsl/. Following the included instructions should install GSL in the default directory, allowing it to be found by the compiler. The HDF5 library can be found at support.hdfgroup.org/HDF5/ where version 1.10 or greater should be downloaded and installed. The parallel portion of HDF5 needs to also be installed which is included in the instructions for installing HDF5.

2.2 Downloading and Compiling MCRaT

Once the required libraries have been installed, the current MCRaT code can be downloaded from github.com/lazzati-astro/MCRaT/. There is a Makefile which makes compiling the code significantly easier. In the directory that all the MCRaT files were downloaded into, simply typing `make` will compile the MCRaT code into a binary file called `MCRAT`. Typing `make MERGE` will compile the included MCRaT post-processing code, which will be covered in later in this section, into a binary file called `MERGE`.

The Makefile has to be configured for a variety of specifics related to the user's system. The `CC` option in the makefile can be changed to the user's compiler of choice such as gcc or icc. The `HDF_INSTALL` line is set to the directory that the HDF5 library is installed in. The `EXTLIB` line is set to the directories of any external libraries that the user may have. For example, if the GSL library cannot be found by the compiler, the user can put the directory of the library in this line of the Makefile in order to properly compile MCRaT. The `CFLAGS` feeds any compiler options to the compiler such as optimizations, and the switch to compile MCRaT with OpenMP capabilities, which is necessary to exploiting the hybrid parallelization available in the code. If the user is using gcc, the `-fopenmp` does not need to be changed, however, if the user is using icc, `-fopenmp` needs to be changed to `-qopenmp`. The `INCLUDE` line can be changed to include any `include/` directories to help the compiler, and the same for the `LIBSHDF` in relation to any shared libraries. The `LIB`, `DEPS`, and `OBJ` lines should not need to be changed. Once the lines in the Makefile appropriately reflect the users configurations, typing `make` and `make MERGE` should properly compile the MCRaT code as well as its post-processing code, `MERGE`.

2.3 The mcrat_input.h file

Once compilation is successful, the user will need to decide where MCRaT's output will go. The user needs to make a directory within the directory that contains the hydro simulation frames. This subdirectory is where the parameter file for MCRaT must be placed. This is the same directory in which MCRaT will do all of its work, including making its own directories and output files within the aforementioned directories. Once this directory is created, the user must modify the `mcrat_input.h` file in order for the code to know where to find all the hydro files and the MCRaT parameter file. The constant `FILEPATH` should be modified with the directory of the flash simulation files while the constant `FILEROOT` should contain the name of the hydro simulation files without any frame numbers. Since the FLASH and PLUTO-CHOMBO simulation files are typically hdf5 files, MCRaT not need the file extension explicitly. The constant `MC_PATH` should be the name of the directory, located within the `FILEPATH` directory, that holds the MCRaT parameter file named `mc.par`. An example file structure, with the values for each variable in MCRaT, is shown below:

```
/dir/to/hydro/simulation/ (FILEPATH)
├── dir_with_MCRaT_parameter_file/ (MC_PATH)
│   └── mc.par
└── example_hydro_file_name_0001 (FILEROOT=example_hydro_file_name_)
```

The other variable that can be changed in MCRaT is `SIMULATION_TYPE` which should be set as `SCIENCE`, if using MCRaT for a production run, or `SPHERICAL_OUTFLOW`/`CYLINDRICAL_OUTFLOW`/`ST`

which is for testing MCRaT with these types of outflows (which will be covered in a later section), or CUSTOM_OUTFLOW for imposing a user-defined analytic outflow (see Section 5.2). Once all the variables have been appropriately changed, by replacing the strings appropriately in the mcrat.h file, the code needs to be recompiled.

Additionally, there are a number of switches that the user can specify to the program to tell it whether they want to consider the effects of polarization, with the STOKES_SWITCH, and one that specifies whether the user wants MCRaT to save the comoving 4 momenta of each photon, with the COMV_SWITCH. These switches can be set to OFF, which means do not save the comoving 4 momenta and/or save and consider the effects of polarization, or they can be set to ON, which tells the program to consider polarization and/or save the stokes parameter associated with each photon or save each photon's 4 momentum.

The SIM_SWITCH tells MCRaT what hydro simulation is being used. The options are FLASH and PLUTO_CHOMBO, for the FLASH hydrodynamic code and the PLUTO hydro code with CHOMBO AMR respectively. The switch DIM_SWITCH tells MCRaT the dimensionality of the hydro simulation that is used. This switch can be set to TWO, for 2D, TWO_POINT_FIVE, for 2.5D, or THREE, for 3D. The geometry can also be specified through the GEOMETRY define statement. The options are SPHERICAL, CARTESIAN, CYLINDRICAL, and POLAR. The length, pressure, and density scales of the hydro simulation also has to be specified by setting the values of HYDRO_L_SCALE, HYDRO_P_SCALE, and HYDRO_D_SCALE, respectively, to the normalization that is used in the hydro simulation.

In order to enable the consideration of thermal synchrotron photons in the MCRaT simulation the CYCLOSYNCHROTRON_SWITCH needs to be set to ON, otherwise this switch should be set to OFF. If it is set to be ON, the user needs to define other variables. One variable that needs to be defined is CYCLOSYNCHROTRON_REBIN_E_PERC which provides MCRaT with an upper limit for the number of synchrotron photons to emit. This quantity is calculated as the percentage of the maximum number of photons the user wants injected (see the [MCRaT Photon Block] portion of the example mc.par file in section 2.4). If CYCLOSYNCHROTRON_REBIN_E_PERC is set to 0.5 and the value of the 17th line is 3000, the upper limit for the number of synchrotron emitted photons is 1500. This variable also defines the number of energy bins that MCRaT will create when it rebins the synchrotron photons, thus in the aforementioned example, the number of energy bins that the photons will be rebinned into is 1500 as well. MCRaT will also rebin the photons into spatial bins and the size of these bins are specified using the CYCLOSYNCHROTRON_REBIN_ANG variable which has units of degrees. In considering synchrotron emission, the user needs to specify how the magnetic field is calculated. This is done through the B_FIELD_CALC switch which can be set to: INTERNAL_E, where the magnetic energy density is set equal to the internal energy of the outflow, TOTAL_E, where the magnetic field is calculated from the total energy of the outflow, or SIMULATION where the code uses the magnitude of the magnetic field that is acquired from a simulation. If B_FIELD_CALC is set to TOTAL_E or INTERNAL_E, the user needs to set the EPSILON_B switch, which represents the fraction of the total outflow energy that is in the magnetic field. More information about the implementation of thermal synchrotron emission in the algorithm of the code can be found in section 3.2. More information regarding the physics of the implementation of thermal synchrotron emission and absorption can be found in section 4.5. MCRaT also provides a non-thermal synchrotron capability, enabled with SYNCHROTRON_SWITCH ON together with a power-law NONTHERMAL_E_DIST and the associated electron-distribution and scattering-bias macros. Because this capability is substantial, its switches and their required companions are documented together in Section 4.6.2; the parameter table below

lists them for completeness.

Another helpful switch is the `SAVE_TYPE` switch, which tells MCRaT to include the photon type in the saved datasets.

The parameters that can be set and the appropriate values that each parameter can be set as are as follows:

Parameter	Acceptable Values
SIMULATION_TYPE	SCIENCE, CYLINDRICAL_OUTFLOW, SPHERICAL_OUTFLOW, STRUCTURED_SPHERICAL_OUTFLOW
FILEPATH	"/dir/to/hydro/simulation/ "
FILEROOT	"example_hydro_file_name "
MC_PATH	"dir_with_MCRaT_parameter_file/"
SIM_SWITCH	FLASH, PLUTO_CHOMBO
STOKES_SWITCH	ON, OFF (Defaults to OFF)
COMV_SWITCH	ON, OFF (Defaults to OFF)
SAVE_TYPE	ON, OFF (Defaults to OFF)
DIMENSIONS	TWO, TWO_POINT_FIVE, or THREE
GEOMETRY	CARTESIAN, SPHERICAL, CYLINDRICAL (only in 2D), POLAR (only in 3D)
HYDRO_L_SCALE	$(-\infty, +\infty)$
HYDRO_D_SCALE	$(-\infty, +\infty)$
MCPAR	"mcpar_filename.par"
CYCLOSYNCHROTRON_SWITCH	ON, OFF (Defaults to OFF)
SYNCHROTRON_SWITCH	ON, OFF (Defaults to OFF)
If CYCLOSYNCHROTRON_SWITCH is ON:	
CYCLOSYNCHROTRON_REBIN_E_PERC	(0.0, 1.0] (Default is 0.1)
CYCLOSYNCHROTRON_REBIN_ANG	$(-\infty, +\infty)$ (Default is 0.5 degrees)
If SYNCHROTRON_SWITCH is ON (see Section 4.6.2):	
NONTHERMAL_E_DIST	POWERLAW BROKENPOWERLAW
POWERLAW_INDEX	$(1, +\infty)$ (single power law)
POWERLAW_INDEX_1, POWERLAW_INDEX_2	$(1, +\infty)$ (broken power law)
GAMMA_MIN, GAMMA_MAX	$[1, +\infty)$, with GAMMA_MIN $<$ GAMMA_MAX
GAMMA_BREAK	(GAMMA_MIN, GAMMA_MAX) (broken power law)
N_GAMMA	positive integer (number of Lorentz-factor subgroups)
TAU_CALCULATION	TABLE (required for the non-thermal path) or DIRECT

Parameter	Acceptable Values
SCATTERING_BIAS_SWITCH	ON, OFF
NONTHERMAL_CONVERSION_FACTOR	(0.0, 1.0] (Default is 0.1)
SCATTERING_BIAS_SCALING	$(0, +\infty)$ (Default is 1)
RR_WEIGHT_FRACTION	(0.0, 1.0] (Default is 10^{-2})
B_FIELD_CALC (used by CYCLOSYNCHROTRON_SWITCH, SYNCHROTRON_SWITCH, and custom outflows)	INTERNAL_E TOTAL_E (Default) SIMULATION
If B_FIELD_CALC is TOTAL_E or INTERNAL_E:	
EPSILON_B	(0.0, 1.0] (Default is 0.5)

2.4 The MCRaT Parameter File

MCRaT requires the user to provide information in the form of a parameter file named `mc.par` which should be placed in the appropriate directory, as is described in the previous section. An example `mc.par` file, named `sample_mc.par` is included in the MCRaT github repository. An example with a description of each line, is as follows:

```
[Hydro/MHD Simulation Block]

5.        # Number of frames per second of hydro simulation (likely always the same)
3000   # Last available hydro simulation frame  (get from the last file in the data folder)
0 5e12   # Max r0 coordinate limits of hydro simulation
0 2.5e13  # Max r1 coordinate limit of hydro simulation
0 2e13   # Max r2 coordinate limit of hydro simulation (if simulation is 3D)

[MCRaT Injection Angles Block]

0.                # The minimum off-axis angle to inject photons (in degrees)
6.                # The maximum off-axis angle to inject photons (in degrees)
3.         # Number of angle bins to consider
200 200 200       # Frame at which photon injection starts for each angle bin
199 199 299       # Number of frames for which photons are injected for each angle bin
1e11 1.5e12 2e12  # The radius at which the photons are injected for each angle bin

[MCRaT Photon Block]

b  # Type of spectrum we inject with, w=wien b=blackbody, c=custom spectrum
1000  # Min number of photons
5000  # Max number of photons

[Initialization/Continuation Block]

i  # Initialize or continue simulation (i=initialize (delete all files) c=continue)
```

The format of the `mc.par` file must follow what is shown above with each block in its respective order and each block title separated by a blank line from the contents of the

block. Furthermore, the coordinate limits for r2 must always be there, even if the hydro simulation that is being analyzed with MCRaT is in 2D. The entries for each line that takes multiple values, such as the coordinate limits, must have each value separated by a space.

The first block provides information on the hydro/MHD simulation that MCRaT is analyzing. The first line denotes the number of hydro frames per second, or fps. The second line provides the file number of the last hydro simulation frame available in the analysis. The next three lines provide the lower and upper limits of the hydro coordinate. Even if the hydro simulation that is being analyzed is in 2D, the last line must still be there. In the example above, the r0 coordinate extends from 0 to 5e12 in the units provided by `HYDRO_L_SCALE` or, in the case of angles, the units used by the hydro simulation. The table below denotes what coordinates r0, r1, and r2 correspond to for each coordinate system and number of dimensions as well as their corresponding units.

	2D	3D
Coordinate System	(r0, r1, r2)	(r0, r1, r2)
Cartesian	(x,y,-)	(x,y,z)
Spherical	(r,θ, -)	(r,θ, ϕ)
Polar	-	(r, ϕ, z)
Cylindrical	(r,z)	-

The second block, called the `[MCRaT Injection Angles Block]`, tells MCRaT what angle ranges (in degrees) it should consider when injecting photons, as provided by the first two lines of the block, and what radii to inject photons for each angle range. The third line of this block denotes the number of angle ranges. In this example, we want MCRaT to inject photons from 0∘-6∘ and we want there to be 3 angle bins. For each angle bin, we provide the frame from which the photon injections will start, $\mathrm{f_{start}}$, the number of frames that we will inject photons into, df, and the radius where the photons will be injected in the frame, R_{inj}. Thus, in the example provided, there have to be 3 numbers provided in each of the last 3 lines of the block. If we wanted N angle bins, then the next 3 lines should contain N numbers for each angle bin. Each number needs to be separated by a space.

These last 3 lines dictate what time in the simulation you begin injecting photons. This is calculated as

$$t_{\mathrm{start}} = \frac{\mathrm{f_{start}}}{\mathrm{fps}} - \frac{R_{\mathrm{inj}}}{c}$$

Building on the above formula, the time in which photons stop being injected are calculated as

$$t_{\mathrm{end}} = \frac{\mathrm{f_{start}} + \mathrm{df}}{\mathrm{fps}} - \frac{R_{\mathrm{inj}}}{c}$$

These calculations are the same for photons injected at all angles. The number of angle bins and the number of frames that photons will be injected into (for each angle bin) are important for MCRaT's parallelism. This will be covered in the next section.

In the next block, labeled `[MCRaT Photon Block]`, we specify the initial condition of the MCRaT simulation and the number of photons that we want MCRaT to consider. The first line denotes whether we want the injected photons to be drawn from a blackbody or Wien spectrum, based on the optical depth being either $\gtrsim 10^3$ or ~ 100, respectively. A third option selects the user-defined custom spectrum (Section 5.1); see that section for

the behaviour of `custom_photon_sampler`. The next two lines specify the minimum and maximum number of photons that MCRaT injects into a given frame, which helps to balance Poisson error from the Monte Carlo method with the run time of the MCRaT code.

The final block allows the user to continue the MCRaT code from a previous point, if the value is set to "c", or it allows the user to initalize (by erasing the full working directory of MCRaT) or start a simulation by setting it to "i". **This is an important parameter to check to prevent accidentally erasing work that has already been done.**

2.5 Understanding MCRaT's Parallelism

The first step that MCRaT takes in breaking up the problem is breaking it up into pieces based on the range of angles that photons will be injected into, which the user specifies in the `[MCRaT Injection Angles Block]` in the `mc.par` file. As the example `mc.par` above shows, MCRaT will break up the problem into 3 pieces initially. Then, for each angle range ($0^\circ - 2^\circ$, $2^\circ - 4^\circ$, etc.) the code will then break up the problem based upon how many times photons will be injected, which is 1 plus the values provided in the `mc.par` file; this is because the photons are injected in the frames $[\mathrm{f_{start}}, \mathrm{f_{start}} + 1, ..., \mathrm{f_{start}} + \mathrm{df} - 1, \mathrm{f_{start}} + \mathrm{df}]$, which gives us this n+1 behavior. These pieces of information are important for understanding how the processes are divided with MCRaT. An example will be given in the next section.

2.6 Running MCRaT

In this section we show how to run MCRaT and allocate the appropriate number of cores in order to optimize MCRaT's parallelism. We will use the example `mc.par` file in Section 2.4 for this section.

Running the MCRaT code is as simple as doing `mpiexec -np N ./MCRAT` where N is the total number of MPI processes that will be created. The OpenMP parallelism is controlled by setting the OpenMP environment variable `OMP_NUM_THREADS`. We recommended that `OMP_NUM_THREADS` be set to 1 for a newly started MCRaT simulation, which essentially turns off the OpenMP parallelism.

The total number of MPI processes, N, is initially divided by the set of angles that photons will be injected into. For the example `mc.par`, there are 3 sets of angles that will be considered which leads to the N MPI process initially being divided by 3. The $N_\theta = N/3$ MPI processes will then be used to inject photons in the user specified frames in parallel. The example `mc.par` file shows that for the first angle bin there will be $199 + 1$ photon injections, as was described in the last section. Thus, the number of frames that each MPI process will inject photons into becomes $\mathrm{f_{inj}} = (199 + 1)/N_\theta$. If we set $N = 150$, $N_\theta = 50$ and the number of frames that each MPI process will inject photons into becomes 4. Alternatively, if we want each MPI process to only inject photons into one frame, $N_\theta = 200$ which means that $N = 600$. Calculating N, by following these examples, shows how the problem is broken up between angles and photon injections. If the number of MPI processes available for the user to use does not divide well into the size of the problem, in terms of angle ranges and frames in which photons are injected, MCRaT will prioritize breaking up the problem evenly based on the set of angles. Thus, N should at least be evenly divisible by the number of angle ranges specified by the `mc.par`; Once

N_θ is calculated, MCRaT will distribute the number of frames in which photons will be injected accordingly to calculate f_{inj}.

Another option, when dealing with a limited number of available cores, is running MCRaT on a small portion of the problem. For example instead of doing 3 sets of angles at 2° intervals all at once, simply run 1 set of angles from $0° - 2°$, and when that simulation is complete run the second set from $2° - 4°$, etc. The increased parallelism in decreasing f_{inj} by using this method may speed up the simulation compared to running the whole problem at once. This is also advantageous due to the fact that as the MPI processes begin to complete their portion of the simulation, the user may restart MCRaT using only the appropriate number of MPI processes left that have calculations left to do. Thus, this feature lets a given MCRaT simulation (e.g. $0° - 2°$) use less MPI processes and resources over time, allowing another full simulation (e.g. $2° - 4°$) to start running using the rest of the available resources. We will go over this feature in a later section.

2.7 MCRaT Output Files

Once MCRaT is run, it will create subdirectories dedicated to the calculations and results of each angle range. Using the example `mc.par` file, the directory tree will now look like:

```
/dir/to/hydro/simulation/
├── example_hydro_file_name_0001
└── dir_with_MCRaT_parameter_file/
    ├── mc.par
    ├── 0.0-2.0/
    │   ├── mc_proc_i.h5
    │   ├── mc_chkpt_i.dat
    │   └── mc_output_i.log
    ├── 2.0-4.0/
    │   ├── mc_proc_i.h5
    │   ├── mc_chkpt_i.dat
    │   └── mc_output_i.log
    └── 4.0-6.0/
        ├── mc_proc_i.h5
        ├── mc_chkpt_i.dat
        └── mc_output_i.log
```

Within each angle subdirectory, each MPI process, labeled process i, creates 3 files. The mc_output_i.log file prints exactly what the MCRaT code is doing for that given MPI process. This allows us to monitor the progress of the MCRaT simulation. This can be easily done by using Linux commands such as `less mc_output_i.log`. The mc_proc_i.h5 is a HDF5 file that contains all of the calculation results of the MCRaT MPI process. The mc_chkpt_i.dat file contains the necessary data for each MPI process to continue from the last point if MCRaT is interrupted for any reason. This is covered in the next section.

When all of the MPI processes in a given angle subdirectory have completed their calculations, they will produce HDF5 files with the results of the calculations for each file in the simulation. The results start with the frame specified in the `[MCRaT Injection Angles Block]` of the `mc.par` file and go up to the frame specified in the next line of the

`mc.par` file block. These files will be named `mcdata_`f`.h5` where f is the frame number of the hydrodynamic simulation.

2.8 Restarting MCRaT

If the MCRaT code gets interrupted by running out of allocated time on a cluster or any other type of issue, the code is easily continued. It may also use less MPI processes when it is restarted based on the number of MPI processes in the original simulation run that still have calculations to complete.

In order to ensure that the code knows to continue a MCRaT simulation, the last block of the `mc.par` file must be set to "c". **This is important to check since not changing this parameter means that all of the simulation progress will be erased and a completely new simulation will be started, thus nullifying the attempt to save time and resources by continuing a MCRaT simulation.**

Once this parameter has been changed, the user needs to identify how many total MPI processes have completed their calculations. This is done by running"

```
tail -n 1 dir_with_MCRaT_parameter_file/angle_dir/mc_output_* |grep
"completed" |wc -l
```

in each angle range directory created by MCRaT. Add all the printed values and subtract from the original number of MPI processes, N, that the MCRaT simulation was originally started with. This is the number that has to replace N in the new run of MCRaT. From here, MCRaT automatically distributes the given number of processes among the MPI processes in each angle subdirectory that still need to run calculations.

This feature of only continuing the MCRaT simulation with the necessary number of MPI processes means that as the simulation progresses, fewer resources will be used which is invaluable for running jobs in a pipeline type of fashion and for conserving resources such as SBU allocations.

If a certain process is also taking a long time to conduct its simulation it is possible to modify the mc.par file to specify that the simulation should inject fewer photons, to speed up the calculation and force the process' simulation portion to restart. To force the restart, delete that process' mc_chkpt file and it's mc_proc file. This is exactly what the `mcrat_msp.sh` shell script does.

2.8.1 Using The mcrat_msp.sh Shell Script

The mcrat_msp.sh shell script determines which MCRaT processes have a very large number of scatterings and allows the user to delete those process' mc_chkpt and mc_proc files, if the process' photons have undergone a number of scatterings greater than some user specified number of scatterings. This would then force a new restart for those processes with less injected photons. It also allows the user to refine the mc.par file to set a new number of injected photons for these same processes that will be restarted the next time the user starts mcrat with the 'c' flag (to continue a simulation). **This is important to change once MCRaT has been restarted and the photons injected. If it isn't, other processes, which don't have so many scatterings, will inject photons within the new range specified by the mc.par file (if applicable, which it should be on a relatively small cluster where the user can't assign one photon injection frame to each process). To do this, once the simulation restarts and is running**

smoothly, the user has to end it, modify the minimum and maximum range of injected photons to the original numbers, and restart the simulation once again. (We are currently working on a better way of implementing this feature.) The user must remember to reset the restart flag in the mc.par file to 'c' otherwise all the simulation data files will be erased and all progress lost. Opening the mcrat_msp.sh file in a text editor will bring up different flags for the shell script and give examples of how to properly use the function.

2.9 Using Hybrid-Parellelization

As was mentioned earlier, it is not advised to enable hybrid parallelization at the start of a new MCRaT simulation, especially if limited resource availability is an issue. This is due to the fact that the OpenMP threads may interfere with one another and neighboring MPI processes, thus slowing the overall code. To prevent this, set `OMP_NUM_THREADS` to 1.

If MCRaT is being used to continue a simulation, there is a good chance that some of the MPI processes have finished and the remaining MPI processes are running slower than the rest due to a variety of factors, which will be covered later. This is where hybrid parallelization helps to ensure that MCRaT completes its calculations is a reasonable amount of time. These MPI processes can be sped up by using the OpenMP threads. The number of threads per MPI process can be set with the `OMP_NUM_THREADS` parameter. Even in this case, it is important to make sure that threads are not interfering with one another and undermining any potential performance gain.

2.10 Post-Processing MCRaT Data

If the user is simply running running a simulation with one angle range under consideration, there is no post-pocessing to be done. The MCRaT produced `mcdata_`f`.h5` files contain all the information for the photons (which will be covered in the next section).

If not we need to be able to merge all the simulation results for photons injected in all angle ranges; following the example `mc.par` file, we need to merge the data in angles $0° - 2°$, $2° - 4°$, etc. This is done is with the MERGE code that is included with MCRaT. To run merge, the user simply types `mpiexec -np N ./MERGE`. In this case N needs to be a multiple of the number of subdirectories the MCRaT code has created and it has to be greater than or equal to the number of subdirectories; of course, the larger the number of MPI processes used, the less work each process has to do, making the post-processing code faster. In the example `mc.par` file there are 3 directories created, so N has to be a multiple of 3 in this case. The minimum value of N can be 3; however, since the MERGE code is also parallelized with OpenMPI, the larger the number of processes used, the less time it takes to merge all of the MCRaT data. If $N = 6$, 3 processes will be allocated to merge the data from the first half of the frames from 200 to 3000, as specified in the `mc.par` file, and the other 3 will merge the data from the last half of the range of frames. If $N = 12$, the number of frames from 200 to 3000 will be divided into quarters and 3 processes will deal with the first quarter, while a second set of 3 processes will deal with the second quarter, so on and so forth.

The merged data is placed into another folder that the code creates named `ALL_DATA/`.

The standard work flow for MCRaT is as follows:

1. Compile the MCRaT and MERGE codes

2. Write the `mc.par` file in the correct directory

3. Run MCRaT

4. Use MERGE to post-process the MCRaT results

This work flow can be practiced using a spherical or cylindrical outflow problem that is included in MCRaT. These simulations can be run using any hydro simulation because MCRaT reads in the data from the hydro simulation and over writes it (in memory, not in the hydro file) with the correct values assuming a spherical or cylindrical outflow.

In order to run a spherical outflow simulation simply change the `SIMULATION_TYPE` parameter in `mcrat_input.h` file to `SPHERICAL_OUTFLOW`. Then recompile the code and run it as we outlined previously. In order to run a cylindrical outflow simulation, change the `SIMULATION_TYPE` parameter to `CYLINDRICAL_OUTFLOW`. These test simulations allow the user to become acquainted with MCRaT and test that MCRaT produces the expected results for these type of outflows. A third option, the custom outflow (Section 5.2), lets the user impose an arbitrary analytic outflow by the same overwrite-in-memory mechanism; this is selected with `SIMULATION_TYPE CUSTOM_OUTFLOW`.

3 The MCRaT Code

This section describes the algorithm and output of MCRaT. It will cover the steps that MCRaT takes to inject and scatter photons. A more in depth description of the physics is included later in this manual.

3.1 Assumptions about the Hydro Simulation

MCRaT has to make two important assumptions about the hydro simulation. These assumptions are that:

1. The velocity in the hydro simulation is normalized by the speed of light, c.

2. The pressure is radiation dominated. As such, we can calculate the temperature using $p = \frac{1}{3}aT^4$, where a is the radiation density constant.

3.2 Algorithm

MCRaT first reads the data from the `mc.par` file and then appropriately divides the MPI processes based on the number of angle ranges, if starting a new simulation, or, for continuing a simulation, reads the mc_chkpt file in each directory to determine which MPI processes still have work to complete and then distributes the MPI processes.

Each MPI process is assigned a range of frames in which the process injects photons into and then propagates through all the hydro simulation frames until the last hydro simulation frame, which is specified in the `mc.par` file. The photons have a `photon` data structure that fully describes each photon. This includes the photon's lab frame 4-momenta, the x, y, and z coordinates of the photon, the Stokes parameters of the photon, the weight of the photon, the number of times the photon has scattered, and the index of the hydro fluid element that the photon is located within.

In order to inject the photons, MCRaT reads in the first hydro frame, that the photons will be injected into, and chooses hydro elements that have radii, r, such that

$R_{\rm inj} - c/(2 * {\rm fps}) < r < R_{\rm inj} + c/(2 * {\rm fps})$, where c is the speed of light, $R_{\rm inj}$ is the injection radius and fps is the hydro simulation frames per second, both specified in `mc.par`. These hydro elements also have to be within the angle range that the MPI process is injecting photons within. Thus, we choose a slab of hydro elements to inject photons into. The number of photons that would be expected from the $i^{\rm th}$ hydro element, n_i, is calculated using the energy density of the element and by assuming a weight for each MCRaT photon. This is further explained in Section 4.2. Once the expected number of photons is calculated, the actual number is acquired by assuming that the photon injection is random, which means that we acquire the actual number of photons injected into the $i^{\rm th}$ hydro element, N_i by randomly drawing from a Poisson distribution with a mean of n_i. The total number of photons that would be injected across all the chosen hydro elements becomes $N = \sum N_i$. If N is within the minimum and maximum number of photons that the user wants, which is specified in the `mc.par` file, the calculation for the number of photons is complete. If not, then the code continues to recalculates n_i using a new weight for each photon, then recalculates N_i and N, and after rechecks so see if N is within the user specified number of photons again. This loop is exited when N falls within the user specified number of desired photons. If the code needs to inject more photons to meet this condition, it will adjust the weights to be smaller and vice versa.

Once N and N_i have been determined, the N_i photons are randomly placed within the $i^{\rm th}$ hydro element, with a random azimuthal angle uniformly distributed between 0 and 2π (since we assume cylindrical symmetry for the 2D hydro simulation), and they are given an energy based on the comoving temperature of the hydro element, thus the photons are injected in the comoving frame of the outflow. If the user specified that the photons should be injected with a Wien spectrum then the frequencies, ν, are assigned to the N_i photons by sampling from a Wien spectrum defined by the $i^{\rm th}$ fluid element's comoving temperature. The same thing is done with a Blackbody spectrum if the user specified that the photons should be injected as a Blackbody spectrum. This sampling is done by using the acceptance-rejection Monte Carlo sampling of the spectra. Alternatively, if the user has selected the custom spectrum, the photon energy (and optionally its direction and polarization) is instead assigned by the user-defined custom_photon_sampler routine described in Section 5.1; all other aspects of injection proceed identically.

For their 4-momenta, the photons are assigned a random azimuthal angle, ϕ, from a uniform distribution between 0 and 2π, as well as a polar angle, which is calculated as $\theta = \arccos(2\xi - 1)$, where ξ is a random number drawn from a uniform distribution between 0 and 1. The $i^{\rm th}$ injected photon 4-momentum becomes:

$$p_i^\mu = \frac{h\nu_i}{c} \begin{pmatrix} 1 \\ \sin(\theta_i)\cos(\phi_i) \\ \sin(\theta_i)\sin(\phi_i) \\ \cos(\theta_i) \end{pmatrix}$$

After the 4-momenta are set, the photons are boosted into the lab (observer) frame using the velocity of the fluid element that the photons are located within. The lab frame 4-momenta are saved to each photon's `photon` data structure, as well as the photon's position and weight. The number of scatterings is initialized to 0 and the stokes parameters are initialized to 0 except for s0 which is set to 1. These photons are also denoted as type 'i', since they are the original injected photons.

Once the photons have been injected into the simulation and their 4-momenta have been determined, the scattering process through the hydrodynamic (HD) simulation

occurs. MCRaT reads in a HD simulation frame, starting from the frame in which the photons were initially injected into up until the last HD simulation frame available, and determines which HD fluid elements each photon is in. MCRaT then calculates the scattering path length of each photon. In order to reduce memory and optimize the code, MCRaT determines the minimum and maximum radii of the photons, $R_{\rm min}$ and $R_{\rm max}$ respectively, and selects a slab of hydro fluid elements with radii, r_i such that

$$R_{\rm min} - \alpha \frac{c}{\rm fps} < r_i < R_{\rm max} + \alpha \frac{c}{\rm fps}$$

where α is a multiplicative factor that can be increased to ensure that the number of chosen hydro elements is greater than 0. The code finds which fluid element each photon is located within, and saves the index to the `photon` data structure. This data field only gets updated if the photon propagates outside of the fluid element. If this happens, then the code finds and saves the index of the new fluid element that the photon is located within. Sometimes, the code is not able to find a fluid block that a given photon is located within. This may happen due to a photon propagating outside of the range of radii of the chosen slab of hydro element, and is very rare. To deal with this, we set the scattering path length, l, to an arbitrary large value to ensure that the photon doesn't scatter, which would inaccurately change its energy. If the fluid element is correctly identified, the scattering path length is calculated as

$$l = -\frac{m_p}{\rho \sigma_{\rm T}(1 - \beta \cos\theta_{fp})} \ln \xi$$

where we have inverted the exponential probability distribution for a photon scattering in a medium. Here, ξ is a random number uniformly distributed between 0 and 1, m_p is the proton mass, $\sigma_{\rm T}$ is the Thompson cross section, ρ is the lab frame density of the hydro fluid element that the photon is located within, β is the velocity of the fluid element normalized by the speed of light (this should be already done in the hydro code), and θ_{fp} is the angle between the fluid element's and the photon's velocity vectors. From l, the time for each photon to scatter is calculated as $t_{\rm s} = l/c$, where c is the speed of light. The photon with the smallest t_s is assumed to scatter first, however that is not necessarily true since we use the full Klein-Nishina cross section in the electron rest frame to actually determine if the scattering occurs. The positions of all the photons are advanced by the time for the photon of interest to scatter, t_s.

In determining whether a photon will scatter or not, we first boost the chosen photon with the smallest t_s to the fluid rest frame and produce an electron by sampling the Maxwell-Boltzmann or Maxwell-Juttner distributions for non-relativistic and relativistic gases. The comoving temperature, T_i', of the fluid element is used to distinguish between which distribution is used. If $T_i' \geq 1 \times 10^7$ K the Maxwell-Juttner distribution is used, otherwise the Maxwell-Boltzmann distribution is used. When the Maxwell-Boltzmann distribution is used we choose 3 random x, y and z velocities from gaussian distributions with a standard deviation of $k_B T_i'/m_e$, where k_B is the Boltzmann constant and m_e is the mass of the electron. The velocities are normalized by the speed of light and used to calculate the Lorentz factor of the electron in the fluid frame. To get the full 4-mometum of the electron, the azimuthal angle of the electron's direction fo travel is drawn from a uniform distribution between 0 and 2π and the relative polar angle (between the electron velocity vector and the incoming photon's velocity vector) is acquired by acceptance-rejection sampling the distribution $(1 - \beta\cos(\theta))\sin(\theta)$. We then perform

rotations based on the velocity vector of the photon that will scatter to ensure that the electron velocity is oriented appropriately relative to the photon's velocity.

Once the electron has been initialized, the photon and the electron are boosted into the electron rest frame. Then the photon velocity vector is rotated such that it is moving directly along the x-axis towards the stationary electron at the origin of the coordinate system. We then use the Klein Nishina (KN) cross section including the effects of polarization, $\sigma_{\rm KN}$, to determine if the photon will actually scatter. The ratio $\sigma_{\rm KN}/\sigma_T$ is calculated and a random number, ξ, from a uniform distribution between 0 and 1 is drawn. If $\xi > \sigma_{\rm KN}/\sigma_T$ the photon does not scatter, otherwise it does scatter with the electron.

If the photon scatters with the electron, a random θ and ϕ are drawn from their respective distributions that are derived from the KN cross section including polarization. The photon's new 4-momentum and stokes parameters are calculated and then the scattered photon is rotated to its original orientation with respect to the previously stationary electron. The photon is then Lorentz boosted twice, back to the lab frame. The process of keeping track of the Stokes parameters and calculating them is covered in subsection 4.4.

If the photon does not scatter, then the next photon with the second shortest time to scatter is chosen and the process to create an electron and determine if the photon will scatter repeats. This loop continues until a photon scatters or all of the photons have been determined to not scatter.

While this loop is occurring, the photons in the simulation are being propagated incrementally by the change in scattering time for each photon until the time for the next hydro simulation frame is reached. The same thing occurs when a photon does scatter, the photons get propagated through space using the scattering time and the process of determining each photons' time to scatter, etc. repeats until the time corresponding to the next hydro frame is reached.

Once the time limit for the next hydro fluid frame is reached, MCRaT saves all the photons' information in the `photon` data structure into the HDF5 file under a group with the same HD frame number that the photons were being scattered in. Once the data is written, a checkpoint file is saved for MCRaT to be able to continue the simulation if it were interrupted. Then, the next simulation frame is loaded and the process of determining photons' time to scatter, actual probability of scattering and resulting scattered 4-momenta until the next HD simulation frame time is reached. This repeats until the last HD simulation frame is reached.

If the user is taking thermal synchrotron photons under consideration, then the thermal synchrotron photons are emitted into the flow in the frame after the initial set of photons are injected into the flow. The emission is done in the comoving frame of the flow. The number of photons emitted is calculated and iterated over in the same manner as the injection of photons. In the thermal synchrotron case the number of photons emitted in the $i^{\rm th}$ fluid element is calculated by integrating the low energy tail of the blackbody spectrum divided by $h\nu$, corresponding to the temperature of that fluid element, from 10 Hz to the cyclotron frequency of that cell. Another difference is that the total number of photons is limited by the `CYCLOSYNCHROTRON_REBIN_E_PERC` switch times the value of the $17^{\rm th}$ line in the `MCPAR` file. The last major difference is that the emitted synchrotron photons are set to have the energy corresponding to the cyclotron frequency of the $i^{\rm th}$ fluid element that they are placed in.

These synchrotron photons are placed at the center of their corresponding fluid elements. If, when MCRaT is calculating the scattering path length of each photon, one of these

synchrotron photons is determined to scatter, it is placed at a random position within its cell, then another synchrotron photon is created to replace it in the pool of photons at the center of the fluid element. After, the scattering process occurs.

In order to keep the number of photons manageable, MCRaT checks the number of scattered synchrotron photons every 1000 scatterings. If the number is smaller than the 17th line in the `MCPAR` file (the maximum number of photons the user wants injected in each frame) then nothing happens, but if the number if larger then MCRaT rebins the photons. The rebinning is done in energy and in space with the number of energy bins, N_E, calculated as `CYCLOSYNCHROTRON_REBIN_E_PERC`$\times N_{\text{max}}$, where `CYCLOSYNCHROTRON_REBIN_E_PERC` is a switch that the user sets in the `mcrat_input.h` file and N_{max} is the value of the 17th line in the `MCPAR` file. The number of spatial bins, N_θ, is calculated as $\Delta\theta$/`CYCLOSYNCHROTRON_REBIN_ANG`, where $\Delta\theta$/`CYCLOSYNCHROTRON_REBIN_ANG` is the angular difference between the largest photon location in theta and the smallest photon location in theta and `CYCLOSYNCHROTRON_REBIN_ANG` is is a switch that the user sets in the `mcrat_input.h` file. $\Delta\theta$/`CYCLOSYNCHROTRON_REBIN_ANG` will most likely be the range of angles that the photons are injected into so that use can anticipate this value a priori. If $N_E \times N_\theta > N_{\text{max}}$, then MCRaT will throw an error pointing this out. Otherwise the rebinning occurs by averaging photons' values within a given energy and spatial bin. The scattering and general steps of MCRaT continues and iterates until the time for the next hydro fluid frame is reached. Before the photons are saved, any photons that have energies that are smaller than the energy corresponding to the thermal synchrotron energy of the cell they are located in gets absorbed and becomes a null photon.

If the MPI process has more photons to inject, then the process of injecting/emitting the photons into the next frame and propagating those photons through the hydro simulation repeats. If all of the MPI processes have finished injecting and propagating photons for the hydro simulation frames assigned to them, the code combines all of the photons' data from each MPI process into `mcdata_`i`.h5` files where i is the hydro simulation frame number. The output is covered in the next section.

3.3 Saved Quantities

Each MCRaT MPI process outputs 3 files. These files are named `mc_proc_`i`.h5`, `mc_chkpt_`i`.dat`, and `mc_output_`i`.log` where i is the number for the MPI process producing those files. As the end of the simulation, MCRaT produces the actual data files named `mcdata_`f`.h5` where f is the hydro simulation file frame.

The `mc_chkpt_`i`.dat` file is a binary file that contains the data from the MPI process that allows it to continue the simulation if MCRaT gets interrupted for any reason. This information is: the MPI process' range of hydro frames to inject photons into, the frame that the current set of photons were injected in, the frame in which the photons were recently scattered in, the current time of the MCRaT simulation, the number of photons that the code is keeping track of, and the `photon` data structure for each photon in the simulation.

The `mc_proc_`i`.h5` file is a HDF5 file that contains all the data for the photons that the ith MPI process injects and propagates through the hydro simulation. The data is organized by the hydro simulation frame which constitutes a HDF5 group. Within the group, there are the following datasets, depending on if `STOKES_SWITCH`, `COMV_SWITCH` or `SAVE_TYPE` ids are set to ON or OFF.

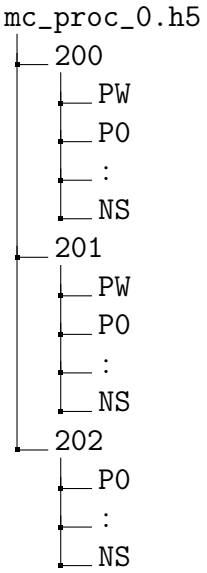
- PW - The photon weight for any photons that were injected into the frame. If there

were no photons injected, then this data set does not exist.

- P0 - the energy of the photons (in erg/c)
- P1 - the magnitude of the photons' 4-momentum in the x direction (in erg/c)
- P2 - the magnitude of the photons' 4-momentum in the y direction (in erg/c)
- P3 - the magnitude of the photons' 4-momentum in the z direction (in erg/c)
- COMV_P0 - the comoving energy of the photons (in erg/c)
- COMV_P1 - the magnitude of the photons' comoving 4-momentum in the x direction (in erg/c)
- COMV_P2 - the magnitude of the photons' comoving 4-momentum in the y direction (in erg/c)
- COMV_P3 - the magnitude of the photons' comoving 4-momentum in the z direction (in erg/c)
- R0 - the x position of the photons (in cm)
- R1 - the y position of the photons (in cm)
- R2 - the z position of the photons (in cm)
- S0 - the normalized I Stokes parameter for the photons
- S1 - the normalized Q Stokes parameter for the photons
- S2 - the normalized U Stokes parameter for the photons
- S3 - the normalized V Stokes parameter for the photons
- NS - the cumulative number of scattering each photon has undergone
- PT - the photon type of each photon (these are saved as ASCII decimal numbers that can be mapped to a character)

The photon types are 'i' (injected photons, including any injected via the custom spectrum sampler of Section 5.1), 's' (emitted synchrotron photons), 'p' (synchrotron pool photons), 'c' (previously-scattered cyclo-synchrotron photons carried across frames), 'k' (Comptonized photons), 'r' (rebinned photons), and 'N' (null photons).

These data sets include information for all of the photons after they were scattered through the given hydro simulation frame. The order of the data is based on the chronological order in which each set of photons were injected. For the example `mc.par` file in subsection 2.4, if the user uses N MPI processes such that each process injects photons into 2 frames, the `mc_proc_0.h5` file would look like:



where all the datasets in group 200 would have only n_0 elements, where n_0 is the number of photons first injected into hydro frame 200. If we let n_1 be the number of photons injected in frame 201, the dataset PW in group 201 will have n_1 elements while the rest of the datasets will have n_0 elements, corresponding to the information for the photons first injected in frame 200 that are propagated through frame 201, *followed by* n_1 elements for the newly injected photons in frame 201.

The last file produced by the MCRaT MPI processes are named `mc_output_`i`.log`. These are files that output the progress of the given process, allowing the user to keep track of how far along MCRaT is in the radiation transfer simulation and how much more work the code has left to complete. This file contains the number of scatterings that the code completes in increments of 1000 scatterings as well as the average photon energy in units of ergs, the comoving temperature, in Kelvin, of the hydro fluid element that was used in the most recent scattering, and the most recent time step for a photon to scatter and the current time in the simulation. When all the scatterings are completed for a given frame, the code prints out the average cumulative number of scatterings that each photon has undergone, as well as the the maximum and minimum number of scatterings for the photons.The code also prints out the average radius of the MCRaT photons, the simulation time and the frames left to inject photons into and the last hydro simulation frame.

When the MCRaT MPI process has completed the Monte Carlo scattering portion of the code it will print: "Process i has completed the MC calculation." where i is the ID of the MPI process. When all of the MPI processes are complete with the Monte Carlo scattering they will collectively produce the `mcdata_`f`.h5` files which contain the merged data from each MPI process within a given angle range.

These `mcdata_`f`.h5` files are formatted in the same way that the HDF5 groups are formatted in the `mc_proc_`i`.h5` files with the exception of the PW dataset being included in each frame, regardless of whether additional photons were injected into the simulation at that frame or not.

3.4 Post-processing Algorithm

When the user has completed running MCRaT for each angle range that they are interested in, the data for every simulated photon needs to be merged. The MERGE code that is included with the download of MCRaT uses parallel HDF5 I/O to accomplish and speed up this post-processing.

The code takes the MPI processes and distributes them evenly among the MCRaT produced angle subdirectories. If there are n_θ angle subdirectories, the MPI processes are grouped into sets of n_θ, with each MPI process operating withing a given subdirectory. Then, the code distributes the hydro simulation frames that need to be created among the groups of n_θ MPI processes. It also counts how many processes were originally used by MCRaT in each directory so if MCRaT is independently run 3 different times for 3 different ranges of angles, but with each of the 3 runs using a different number of MPI processes, the post-processing code will still work.

These processes open the `mc_proc_`i`.h5` files and tally how many photons will be merged to produce the final `mcdata_`f`.h5` file in the `ALL_DATA` directory. Using this information, the code constantly ensures that any files created by MERGE files that were created, before the code was terminated for any reason, are not corrupted; thus the MERGE code can be stopped and started again without worrying about corrupting data. The code iterates over the photons injected from first to last in the MCRaT simulation, by reading the data from the `mc_proc_`i`.h5` files. These photon data are then collectively appended to the final `mcdata_`f`.h5` file.

4 Physics Included in MCRaT

This section describes the physics of the MCRaT code injecting photons and scattering them through the hydro simulation. We also go into the physics of emitting thermal synchrotron photons into the outflow and how we rebin these photons to ensure that the number of photons doesn't increase without bound.

4.1 Background: Monte Carlo Radiation Transport

4.1.1 What is Monte Carlo radiation transport?

In astrophysics, we are often interested in how radiation propagates through a plasma: photons are emitted by electrons, scattered off other electrons, and absorbed. The exact solution to the radiative transfer equation in a complex, multi-dimensional, relativistically moving medium is computationally intractable. Monte Carlo (MC) methods provide a practical alternative: rather than solving a partial differential equation for the radiation field, we simulate the trajectories of a large number of representative photon *packets* and accumulate statistics from their histories.

4.1.2 Photon packets and Monte Carlo estimators

A photon packet represents many physical photons. Its weight w_k records the number of physical photons represented by packet k.

For an extensive observable, such as photon number, emitted energy, or a spectral-bin

contribution, the Monte Carlo estimator has the generic form

$$\widehat{F} = \sum_k w_k \, f_k, \tag{1}$$

where f_k is the contribution of one represented photon. Spectra, luminosities, and light curves use this form together with the relevant time-, solid-angle-, distance-, and bin-width normalizations.

A normalized weighted average is instead

$$\widehat{\langle f \rangle} = \frac{\sum_k w_k f_k}{\sum_k w_k}. \tag{2}$$

Equation (2) is appropriate only when a normalized mean is intended; it is not the generic estimator for an extensive radiative quantity.

4.1.3 Continuous absorption and weight attenuation

There are two philosophically different ways to handle absorption in a Monte Carlo code:

1. **Stochastic absorption**: a packet is either transmitted or destroyed at each interaction, determined by a random draw. This is unbiased but can be very noisy when the survival probability is high (nearly transparent medium).

2. **Continuous (deterministic) absorption**: the weight is reduced at every step by the survival probability $\exp(-d\tau)$, where $d\tau$ is the optical depth traversed. The packet is never stochastically destroyed; instead it carries a continuously decreasing weight. This is also unbiased and has lower variance, but produces packets with very small weights after traversing optically thick material.

MCRaT uses continuous absorption for synchrotron self-absorption. The consequence is that after traversing an optically thick region, a packet's weight may have fallen to a tiny fraction of its birth weight, making its statistical contribution negligible while still consuming computational resources. The Russian roulette algorithm described in Section 4.6.9 addresses this problem.

4.2 Injecting Photons

The code selects HD fluid elements at a given radius and calculates the expected number of photons in each element as

$$n_i = \frac{\xi T_i'^3 \Gamma_i}{w} dV_i \tag{3}$$

where ξ, the number density coefficient which is used to calculate the number density of photons as $n_\gamma = \xi T'^3$, is 20.29 for a Blackbody spectrum or 8.44 for a Wien spectrum, T_i' is the comoving temperature of the fluid element, Γ_i is the Lorentz factor of the element, w is the weighting factor of each injected photon, and dV_i is the volume of the fluid element.

The comoving temperature is calculated as

$$T_i' = \left(\frac{3p_i}{a}\right)^{\frac{1}{4}} \tag{4}$$

where p_i is the pressure of the fluid element, and a is the radiation density constant.

If the HD simulations used are 2D axis-symmetric, we assume cylindrical symmetry and calculate $dV_i = 2\pi x_i dA_i$ where x_i is the distance of the fluid element from the y-axis of the simulation and dA_i is the area of the element.

Once the number of photons is calculated, we draw a random number from a Poisson distribution using n_i as the average value in order to get the actual number of photons that will be injected into the ith HD element, N_i. The weight, w, for a given set of injected photons can be adjusted accordingly in order to inject more or less photons into the simulation; this allows us to conserve energy and increase photon statistics for time resolved spectra. This weight is essentially how many real photons each MCRaT photon represents.

The total number of photons that would be injected across all the chosen hydro elements becomes $N = \sum N_i$. If N is within the minimum and maximum number of photons that the user wants, which is specified in the `mc.par` file, the calculation for the number of photons is complete. If not, then the code continues to recalculates n_i using a new weight for each iteration, then recalculates N_i and N, and after rechecks to see if N is within the user specified number of photons again. This loop is exited when N falls within the user specified number of desired photons. If the code needs to inject more photons to meet this condition, it will adjust the weights to be smaller and vice versa.

Once N and N_i have been determined, the N_i photons are placed at the center of the ith hydro element, with a random azimuthal angle uniformly distributed between 0 and 2π (since we assume cylindrical symmetry for the 2D hydro simulation), and they are given an energy based on the comoving temperature of the hydro element, thus the photons are injected in the comoving frame of the outflow. If the user specified that the photons should be injected with a Wien spectrum then the frequencies, ν_i, are assigned to the N_i photons by sampling from a Wien spectrum defined by the ith fluid element's comoving temperature. The same thing is done with a Blackbody spectrum if the user specified that the photons should be injected as a Blackbody spectrum. This sampling is done by using the acceptance-rejection Monte Carlo sampling of the spectra.

For their 4-momenta, the photons are assigned a random azimuthal angle, ϕ, from a uniform distribution between 0 and 2π, as well as a polar angle, which is calculated as $\theta = \arccos(2\xi - 1)$, where ξ is a random number drawn from a uniform distribution. The ith injected photon 4-momentum becomes:

$$p_i^\mu = \frac{h\nu_i}{c} \begin{pmatrix} 1 \\ \sin(\theta_i)\cos(\phi_i) \\ \sin(\theta_i)\sin(\phi_i) \\ \cos(\theta_i) \end{pmatrix}$$

After the 4-momenta are set, the photons are boosted into the lab frame using the velocity of the fluid element that the photons are located within. The lab frame 4-momenta are saved to each photon's `photon` data structure, as well as the photon's position and weight. The number of scatterings is initialized to 0 and the stokes parameters are initialized to 0 except for s0 which is set to 1.

Alternatively, if the user has selected the custom spectrum, the photon energy (and optionally its direction and polarization) is instead assigned by the user-defined custom_photon_sampler routine described in Section 5.1; all other aspects of injection proceed identically.

4.3 Scattering Photons

In order to scatter the photons, we first need to determine the distance that each photon will propagate before scattering. Thus we calculate each photon's mean free path and choose a random number that will distribute it according to an exponential distribution. We calculate the distance each photon will propagate before scattering as:

$$l = -\frac{m_p}{\rho\sigma_{\rm T}(1-\beta\cos\theta_{fp})}\ln\xi$$

where, ξ is a random number uniformly distributed between 0 and 1, m_p is the proton mass, $\sigma_{\rm T}$ is the Thompson cross section, ρ is the lab frame density of the hydro fluid element that the photon is located within, β is the velocity of the fluid element normalized by the speed of light (this should be already done in the hydro code), and θ_{fp} is the angle between the fluid element's and the photon's velocity vectors. From l, the time for each photon to scatter is calculates as $t_{\rm s} = l/c$, where c is the speed of light. The photon with the smallest t_s is assumed to scatter first, however that is not necessarily true since we use the full Klein-Nishina cross section in the electron rest frame to actually determine if the scattering occurs. The positions of all the photons are advanced by the time for the photon of interest to scatter, t_s.

In determining whether a photon will scatter or not, we first boost the chosen photon with the smallest t_s to the fluid rest frame and produce an electron by sampling the Maxwell-Boltzmann or Maxwell-Juttner distributions for non-relativistic and relativistic gases. The comoving temperature, T_i', of the fluid element is used to distinguish between which distribution is used. If $T_i' \geq 1\times10^7$ K the Maxwell-Juttner distribution is used, otherwise the Maxwell-Boltzmann distribution is used. When the Maxwell-Boltzmann distribution is used we choose 3 random x, y and z velocities from gaussian distributions with a standard deviation of $k_B T_i'/m_e$, where k_B is the Boltzmann constant and m_e is the mass of the electron. The velocities are normalized by the speed of light and used to calculate the Lorentz factor of the electron in the fluid frame. To get the full 4-mometum of the electron, the azimuthal angle of the electron's direction of travel is drawn from a uniform distribution between 0 and 2π and the polar angle is acquired by acceptance-rejection sampling of the polar angle distribution $(1-\beta\cos(\theta))\sin(\theta)$. We then perform rotations based on the velocity vector of the photon that will scatter to ensure that the electron velocity is oriented relative to the photon's velocity.

Once the electron has been initialized, the photon and the electron are boosted into the electron rest frame. Then, the photon velocity vector is rotated such that it is directly along the x-axis with the stationary electron at the origin of the coordinate system. We then use the Klein Nishina (KN) cross section, $\sigma_{\rm KN}$, to determine if the photon will actually scatter. Using $\sigma_{\rm T}$ as the Thomson cross section, the ratio $\sigma_{\rm KN}/\sigma_T$ is calculated and a random number, ξ, from a uniform distribution between 0 and 1 is drawn. If $\xi > \sigma_{\rm KN}/\sigma_T$ the photon does not scatter, otherwise it does scatter with the electron.

If $\xi \leq \sigma_{\rm KN}/\sigma_{\rm T}$ then we sample the differential KN cross section to determine the angles, $\theta_{\rm sc}$ and $\phi_{\rm sc}$, that the photon will be scattered into. The differential cross section given by Lundman et al. (2014) is

$$\frac{{\rm d}\sigma_{\rm KN}}{{\rm d}\Omega}(\theta_{\rm sc},\phi_{\rm sc}) = \frac{r_0^2}{2}\left(\frac{\epsilon}{\epsilon_0}\right)^2 \times \left\{\frac{\epsilon_0}{\epsilon}+\frac{\epsilon}{\epsilon_0}-\sin^2\theta_{\rm sc}\left(1-q\cos2\phi_{\rm sc}+u\sin2\phi_{\rm sc}\right)\right\} \quad (5)$$

where r_0 is the classical electron radius, ϵ_0 is the incoming photon energy scaled by the electron rest mass, $\epsilon = \epsilon_0/(1+\epsilon_0[1-\cos\theta_{\rm sc}])$ is the scattered photons energy scaled by

the electron rest mass, and q and u are the stokes parameters discussed further in Section 4.4. The outgoing photon's $\theta_{\rm sc}$ is acquired by rejection sampling the differential cross section integrated over $\phi_{\rm sc}$ following the method outlined by Mathews (2013) for maximum efficiency. We acquire $\phi_{\rm sc}$ by choosing a random uniformly distributed value between [0, 2π] when $q = u = 0$, otherwise we apply a rejection sampling method to the differential KN cross section. The case of $q = u = 0$ removes the KN cross section's dependence on $\phi_{\rm sc}$, which allows us to choose a value between [0, 2π].

We outline our method of sampling Equation 5 to acquire $\phi_{\rm sc}$ when the incoming photon is polarized. The azimuthal angle that gives the maximum of the KN differential cross section, $\phi_{\rm m}$, can be solved as

$$\phi_{\rm m} = \frac{1}{2}|\arctan\left(\frac{-u}{q}\right)| \tag{6}$$

We then plug $\phi_{\rm m}$ and the previously acquired $\theta_{\rm sc}$ into the KN differential cross section to get the normalization for the rejection sampling method, $\frac{{\rm d}\sigma_{\rm KN}}{{\rm d}\Omega}(\theta_{\rm sc}, \phi_{\rm m})$. We proceed as follows:

(1) Draw a random number ξ uniformly distributed between 0 and 1

(2) Draw a random $\phi_{\rm sc}$ uniformly distributed between 0 and 2π

(3) Calculate $\frac{{\rm d}\sigma_{\rm KN}}{{\rm d}\Omega}(\theta_{\rm sc}, \phi_{\rm sc})$

(4) Determine if $\xi \leq [\frac{{\rm d}\sigma_{\rm KN}}{{\rm d}\Omega}(\theta_{\rm sc}, \phi_{\rm sc})][\frac{{\rm d}\sigma_{\rm KN}}{{\rm d}\Omega}(\theta_{\rm sc}, \phi_{\rm m})]^{-1}$

(5) If the above condition is met, accept the value of $\phi_{\rm sc}$, otherwise repeat the sampling

The photon's new 4-momentum and stokes parameters are calculated and then the scattered photon is rotated to its original orientation with respect to the previously stationary electron. The photon is then Lorentz boosted twice, back to the lab frame.

If the photon does not scatter, then the next photon with the second shortest time to scatter is chosen and the process to create an electron and determine if the photon will scatter repeats. This loop continues until a photon scatters or all of the photons have been determined to not scatter.

4.4 Polarization

The Stokes parameters are a vector, $S = (I, Q, U, V)$ that holds information about the polarization of electromagnetic radiation. I is the intensity of the electromagnetic radiation, Q and U describe the orientation of the polarization ellipse, and V describes the ratio of the principal axis of the polarization ellipse (Rybicki & Lightman, 1979). We follow the convention set by McMaster (1961) and Lundman et al. (2014) where $Q = +1$ is oriented with the y-axis of the Stokes plane and $Q = -1$ is oriented with the x-axis of the Stokes plane. The $+U$ axis is rotated 45° clockwise with respect to the $+Q$ axis and the $-U$ axis is rotated 45° clockwise with respect to the $-Q$ axis. Furthermore, we normalize the Stokes parameters such that $I = 1$ at all times giving us $s = (1, Q/I, U/I, V/I) = (1, q, u, v)$. In the code, it is stored as $s = (S0, S1, S2, S3)$. In our simulations, we only consider linear polarization which means that we ignore any contribution by v (S0). This is appropriate since we assume that electron spins, which directly affect v, are isotropically distributed.

In MCRAT all photons are initialized to have no polarization; thus, we set $s = (1, 0, 0, 0)$. Each photon becomes 100% polarized from the very first scattering that it undergoes, which does not bias the results.

In order to deal with polarization, we follow the prescription given by Lundman et al. (2014). We lorentz boost the photons' 4 momenta from the lab frame to the fluid frame, then from the fluid frame to the electron rest frame. In the electron rest frame we conduct our scattering using the full Klein Nishina (KN) cross section and then scatter the Stokes parameters using Fanno's Matrix. Afterwards, we deboost the photons from the electron rest frame and the fluid rest frame back to the lab frame.

In the lab frame, the Stokes plane is oriented such that the $+Q$ axis is pointing in the $-\hat{\phi}$ direction and the $-Q$ axis is pointing in the $-\hat{\theta}$ direction. The $+U$ axis is rotated 45° clockwise with respect to the $+Q$ axis and the $-U$ axis is rotated 45° clockwise with respect to the $-Q$ axis. χ is measured clockwise from the $+Q$ axis towards the $+U$ axis. $\hat{\phi}$ and $\hat{\theta}$ are the orthonormal azimuthal and polar unit vectors in a spherical coordinate system where the radial unit vector is parallel to the photon's momentum vector.

Each boost to another reference frame entails rotating the Stokes plane using the Muller rotation matrix (McMaster, 1961) given by

$$\mathbf{M}[\phi] = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos 2\phi & -\sin 2\phi & 0 \\ 0 & \sin 2\phi & \cos 2\phi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{7}$$

where the angle of rotation, ϕ, corresponds to the rotation that orients the y-axis of the Stokes plane perpendicular to the photon three momentum and the velocity vector of the frame that the photon will be boosted into. The equation for ϕ is given in Appendix B of Lundman et al. (2014). After each boost, we rotate the Stokes plane again to ensure that the y-axis of the Stokes plane is perpendicular to the plane formed by the reference frame's z-axis and the photon's three momentum.

We conduct a scattering of the photon's 4 momenta as outlined in section 4.3. Lorentz boosting to all the proper frames and sampling the KN cross section.

Once the scattering is completed, and still in the electron rest frame after undoing all of the rotations that placed the photon along the x axis moving towards the electron at the origin of the coordinate system, we conduct the scattering of the Stokes parameters. First, we use the Muller matrix to rotate the Stokes plane such that the Stokes plane's y-axis is perpendicular to the plane formed by the incoming and outgoing photon three momenta (McMaster, 1961; Lundman et al., 2014). Then, we use Fanno's matrix, $T[\theta_{\rm sc}, \epsilon_0, \epsilon]$, (McMaster, 1961) to determine the resulting polarization, $\tilde{s} = T[\theta_{\rm sc}, \epsilon_0, \epsilon]s$. Fanno's matrix is

$$T[\theta_{\rm sc}, \epsilon_0, \epsilon] = \begin{pmatrix} 1 + \cos^2\theta_{\rm sc} + (\epsilon_0 - \epsilon)(1 - \cos\theta_{\rm sc}) & \sin^2\theta_{\rm sc} & 0 \\ \sin^2\theta_{\rm sc} & 1 + \cos^2\theta_{\rm sc} & 0 \\ 0 & 0 & 2\cos\theta_{\rm sc} \end{pmatrix} \tag{8}$$

where, similar to Krawczynski (2011), we have excluded the factors in front since they cancel out when we normalize the scattered Stokes parameters by I as mentioned above. We have also excluded the fourth row and column of the matrix since we only consider linear polarization. In this case, $\theta_{\rm sc}$ is acquired by calculating the angle between the photon's original incoming 3 momentum and the new outgoing 3 momentum. We calculate this as:

$$\theta_{\rm sc} = \arccos(\overrightarrow{p_{\rm in}} \cdot \overrightarrow{p_{\rm out}}) \tag{9}$$

Once the new stokes parameters are acquired, they are normalized by dividing by $S0$. Then, the stokes parameters are rotated to ensure that the y-axis of the Stokes plane is perpendicular to the plane formed by the electron rest frame's z-axis and the outgoing photon's 3 momentum. After the photon is deboosted back to the lab frame and the stokes parameters are rotated for each lorentz boost.

We reproduce Depaola's (2003) result, as shown in Figure 1, which verifies the sampling algorithm of the differential KN cross section. Figure 1 shows the resulting modulation curve when our Klein Nishina (KN) cross section is monte carlo sampled in black. The analytic profile of the cross section as a function of $\phi_{\rm sc}$ calculated by Depaola (2003) is shown as the solid blue line. They sample the KN cross section for a photon beam with 100% polarization along the $+Q$ direction, with an energy of 100 keV and $85^\circ < \theta_{\rm sc} < 90^\circ$. Accounting for the differences in the positive stokes parameters in the convention used, we are able to use our method of sampling the stokes parameter to acquire the proper distribution.

Additionally, reproducing Krawczynski's (2011) results, in Figure 2, tests the Lorentz transform portion of MCRaT. Krawczynski (2011) scattered a beam of photons, 100% polarized along the $+Q$ axis with frequency $\omega = 10^{12}$ Hz, with a beam of electrons moving at a Lorentz factor of $\gamma = 100$. Following their setup, we produce distributions of the resulting lab frame quantities which we show in Figure 2. These distributions are identical, with the exception of normalization, to the distribution acquired by Krawczynski (2011) in their Figure 6. The difference in the signs of Q and U in our distribution with respect to Krawczynski's (2011) Figure 6 is due to the differences in the orientation of the $+Q$ and $+U$ axis of the Stokes plane.

In order to test the code globally, we reproduce the results of Lundman et al. (2014). To do so, we imposed the analytic jet structure provided by Lundman et al. (2014) on some FLASH HD simulation frames. This is similar to Lazzati (2016) simulating a variety of outflows by imposing an analytic solution onto hydrodynamic simulation files. The domain of this simulation is 2.5×10^{13} cm along the direction of the jet and 5×10^{12} cm along the x axis. We used $\sim 6 \times 10^5$ photons to conduct our code validation for a wide structured jet with $\theta_{\rm j} = 0.1$ radians ($\sim 5.7^\circ$), $\Gamma_0 = 100$ and $L = 3 \times 10^{50}$ erg/s. This is the same case exhibited in Lundman et al.'s (2014) wide jet with the exception of the value of L that we use, which was chosen to maximize the number of photons that reached the photosphere before they approached the edge of the domain of the hydrodynamic simulation. There are two major differences between the simulation conducted by Lundman et al. (2014) and the MCRaT simulation: 1) MCRaT uses the full Klein Nishina Cross section to determine if photons scatter while Lundman et al.'s (2014) simulation uses the Thomson cross section and 2) The photons in MCRaT are not permitted to immediately escape to infinity if the randomly drawn optical depth is small enough while photons in Lundman et al.'s (2014) simulation are allowed to do so.

In Figure 3 we show the results of our validation. The polarization acquired by Lundman et al. (2014) is shown as the red dotted line, the black points with 1σ error bars show the polarization acquired by MCRaT, and the grey dotted line shows $\Pi = 0\%$ for reference. In addition to finding that the Stokes u parameter vanishes ($\sum u \approx 0$), which is expected for an axis symmetric jet (Ito et al., 2014; Lundman et al., 2014), we find that MCRaT is able to recover Lundman et al.'s (2014) polarization profile relatively well. We are also able to recover the change in the sign of the stokes Q parameter at $\theta_{\rm v}/\theta_{\rm j} \sim 1.2$ that Lundman et al. (2014) find in their results. The MCRaT result is slightly lower than the polarization acquired by Lundman et al. (2014) due to the fact that the analytic

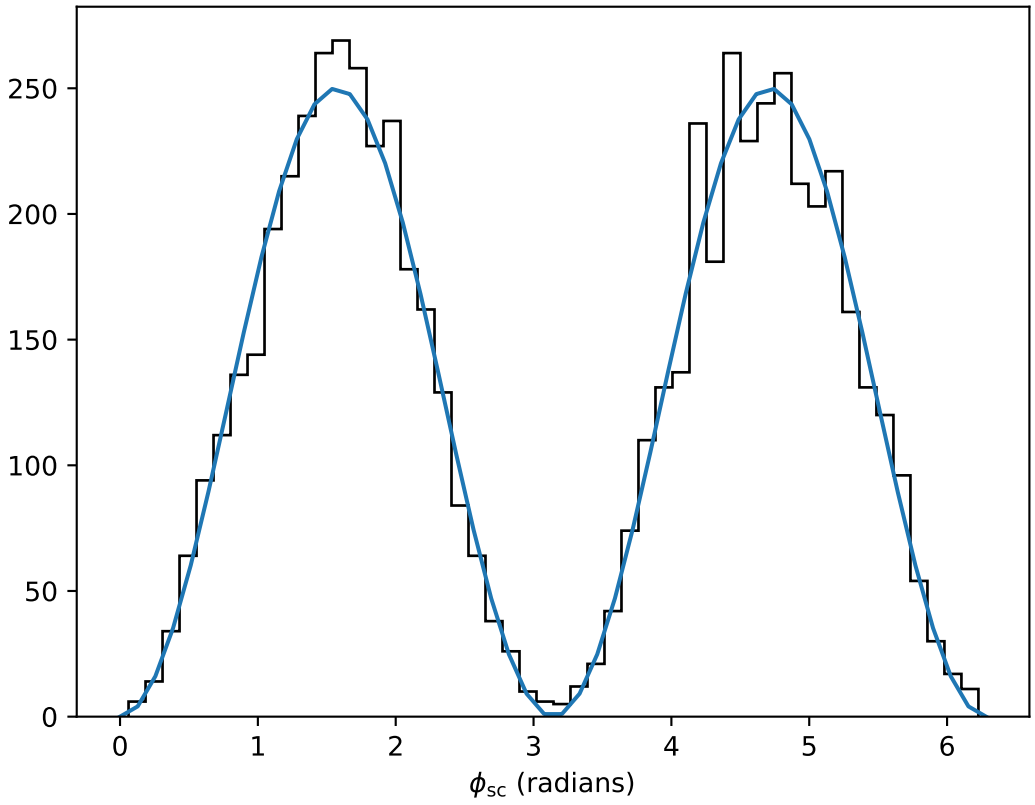


Figure 1: Plot of the distributed ϕ_{sc} from sampling the Klein Nishina cross section compared to the analytic cross section acquired by Depaola (2003). The distribution is shown in black and the analytic profile is shown by the blue curve.

jet profile is mapped onto a discretized grid; this has the effect of decreasing the gradients in the jet profile that would contribute to a larger polarization. Furthermore, Lundman et al.'s (2014) polarization profile contains a minimum of 200 photons in each angle bin, while the results of the MCRAT validation contain at least ~ 10000 photons in each angle bin, increasing the general precision of the MCRAT results.

For $\theta_v/\theta_j \gtrsim 1.6$ ($\theta_v \approx 9°$) the MCRAT polarization begins to decrease again, coming into strain with what is expected. This is due to the fact that the simulation files that we impose the analytic jet equations onto have a finite domain. Even when the photons reach the edge of the domain (2.5×10^{13} cm) at $\theta_v \gtrsim 9°$, they haven't fully decoupled from the photosphere (which would be located at $r \sim 1 \times 10^{14}$ cm (Lundman et al., 2014)) thus decreasing the detected polarization.

4.5 Cyclo-synchrotron Radiation

4.5.1 Emitting Cyclo-synchrotron (CS) Photons

The emission of the cyclo-synchrotron (CS) photons is similar to injecting photons into the outflow. We iteratively calculate the number of photons to emit as synchrotron photons in the frame and assign them random 4 momenta with an energy corresponding to the local cyclotron frequency. We consider the source of CS photons to be constant in time so

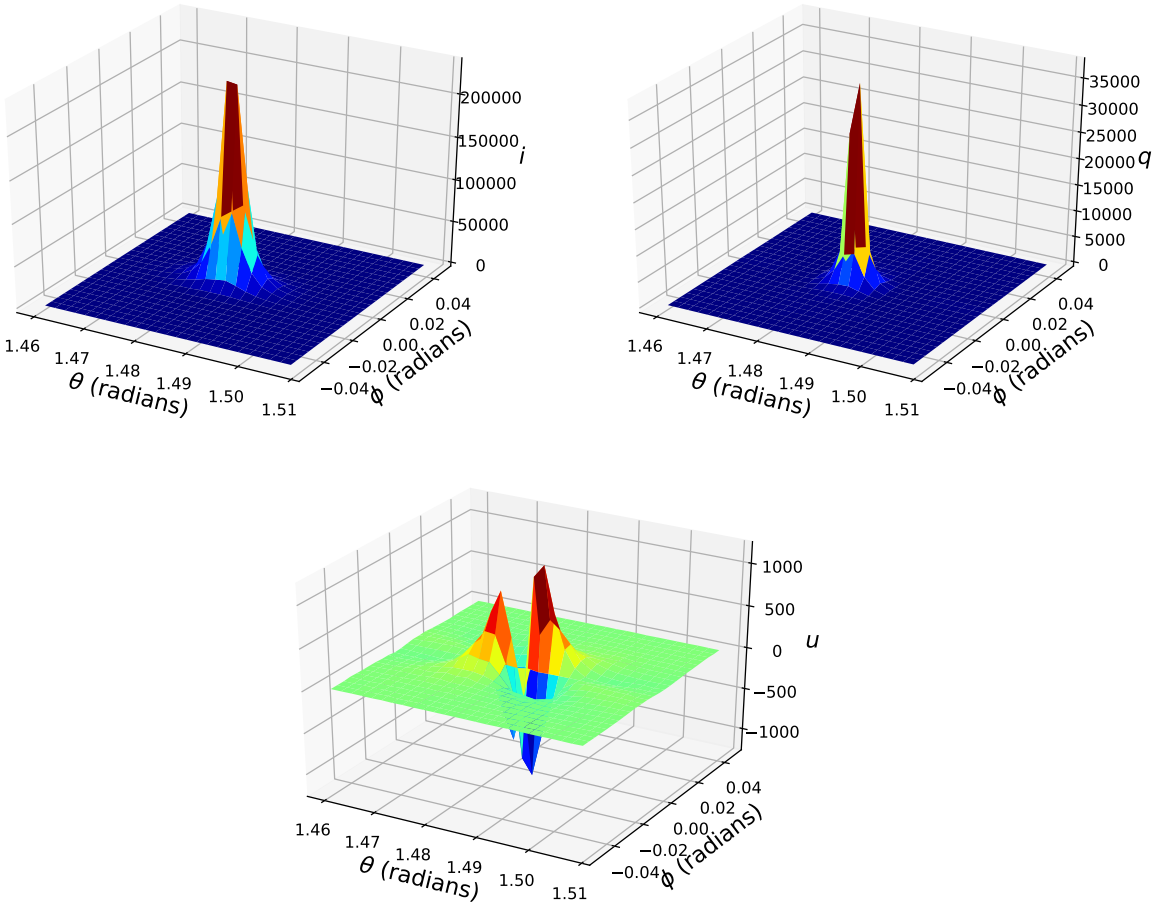


Figure 2: The distribution of the stokes parameters as a function of the scattered photon's θ and ϕ values. These distributions are morphologically similar to the distribution acquired by Krawczynski (2011) in thier Figure 6.

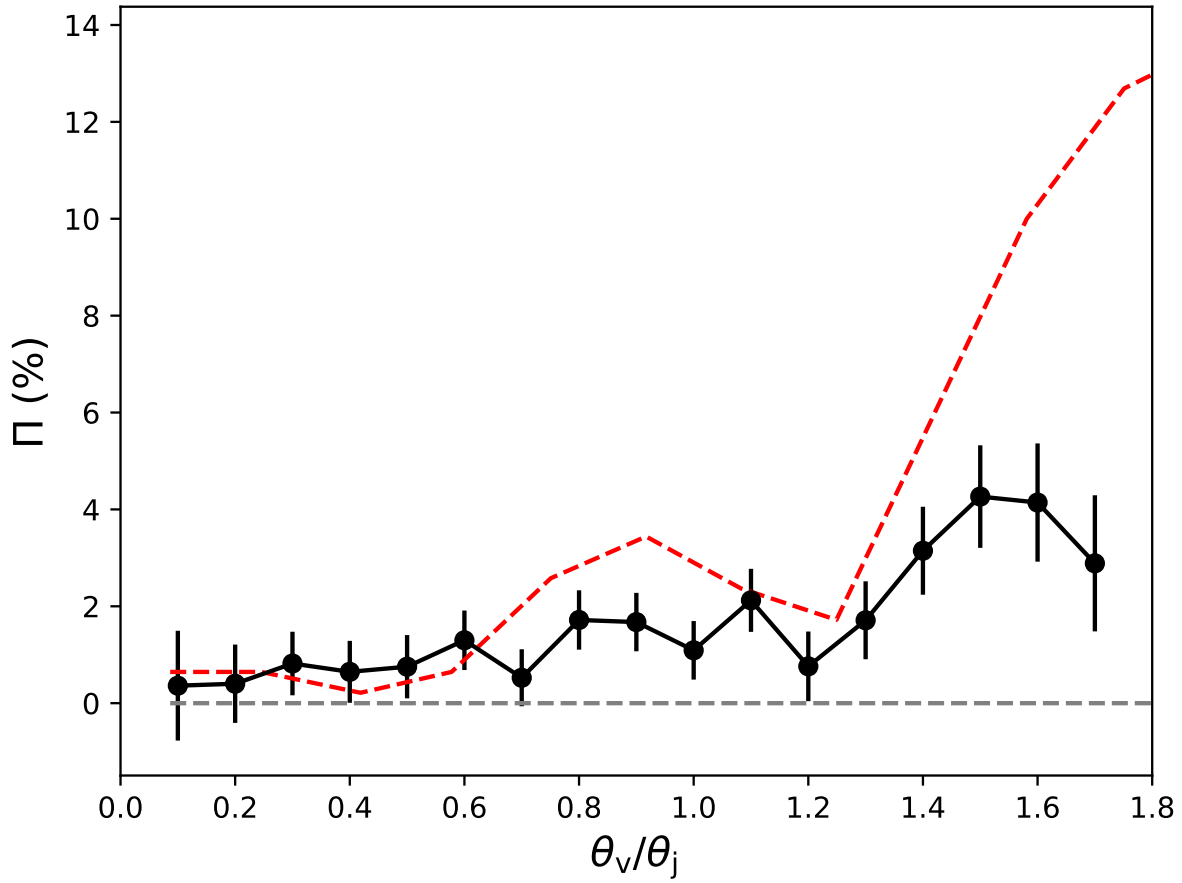


Figure 3: A comparison between the polarization acquired by Lundman et al. (2014) for a structured jet with $\theta_j = 0.1$ radians $\Gamma_0 = 100$ and $L = 3 \times 10^{50}$ erg/s, shown by the red dashed line, and the McRaT acquired polarization shown by the black points with 1σ error bars. The dashed grey line denotes $\Pi = 0\%$ for reference. We find agreement until $\theta_v/\theta_j \approx 1.6$ ($\theta_v \approx 9°$) where the McRaT photons are no longer decoupled from the flow by the time they reach the edge of the simulation domain.

we emit a constant number and if any synchrotron photon gets upscattered, we replace it with another synchrotron photon.

These photons are emitted into a region that the original injected photons are passing through so it accounts for the CS photons that would be upscattered in the flow during the time of interest of the original injected photons.

The maximum number of photons that are emitted, $N_{e,max}$, are calculated as the percentage of the maximum number of photons the user wants injected (see line 17 of the example mc.par file in section 2.4). For example, if `CYCLOSYNCHROTRON_REBIN_E_PERC` is set to 0.5 and the value of the 17th line is 3000, the upper limit for the number of synchrotron emitted photons is 1500. The number density of CS photons, $n_{s,i}$, in a given cell is acquired by integrating the blackbody photon number spectrum up to the cyclotron frequency, ν_B. We calculate the number density as

$$n_{s,i} = \int_0^{\nu_B} \frac{u_\nu}{h\nu} d\nu \frac{dV_i}{w} = \int_0^{\nu_B} \frac{8\pi\nu^2}{c^3(e^{\frac{h\nu}{kT_i'}} - 1)} d\nu \frac{dV_i}{w} \tag{10}$$

where w is the weight of each CS photon, dV_i is the volume of the HD cell, T_i is the temperature of the same cell, h is Planck's constant and

$$\nu_B = \frac{eB}{2\pi m_e c} \tag{11}$$

where e is the electron charge and B is the magnetic field within the cell. Once $n_{s,i}$ is calculated, we draw a random number from a Poisson distribution with a mean of $n_{s,i}$ to acquire the total actual number of photons that are emitted into the cell. We get the total number of emitted photons, N_e, by summing over each cell's $n_{s,i}$. If $N_e > N_{e,max}$, then the w is made larger and then the aforementioned calculations are made. If $N_e \leq N_{e,max}$ then the code proceeds to assign the CS photons a random 4 momentum and assigns them to be at the center of their respective hydrodynamic cell.

When a CS photon gets scattered, it is placed randomly within its HD cell and the scattering proceeds as it normally would. Then another CS photon gets added to the pool of photons at the center of their respective HD cell with the proper ν_B.

4.5.2 Calculating B

MCRaT provides 2 ways to calculate the magnetic field in the outflow, in order to calculate ν_B. It either uses the internal energy fo the flow to calculate B, when the `B_FIELD_CALC` switch is set to `INTERNAL_E`, or it uses the total energy of the outflow to calculate B, when the `B_FIELD_CALC` switch is set to `TOTAL_E`.

- Total Energy

  For a given plasma, the luminosity from matter and radiation can be written as $L = 4\pi(\rho c^2 + 4p)\Gamma^2 c r^2$, where c is the speed of light, ρ is the plasma density, p is the pressure, Γ is the bulk Lorentz factor, and r is the radius of the plasma away from its point of origin. The luminosity due to the magnetic field is $L_B = 4\pi(B^2/8\pi)\Gamma^2 c r^2$ where B is the magnetic field of the plasma. The fraction of energy between the magnetic outflow luminosity and the total outflow luminosity can be written as

$$\epsilon_B = \frac{L_B}{L} = \frac{B^2}{8\pi(\rho c^2 + 4p)} \tag{12}$$

If the user supplies ϵ_B to MCRaT via the `EPSILON_B` parameter then the code uses the above formula to solve for the magnetic field in each HD cell.

- Internal Energy

  In the internal energy case, the magnetic field energy density is equated to the electron thermal kinetic energy.

$$\frac{B^2}{8\pi} = \frac{3}{2} n k T' \epsilon_B \tag{13}$$

  where n is the number density of electrons, k is the boltzmann constant and T is the plasma temperature.

MCRaT can also read in the magnetic field from a magneto-hydrodynamic simulation and use the magnitude of the magnetic field in calculating ν_B. This option is used when `B_FIELD_CALC` is set to `SIMULATION`.

4.5.3 Rebinning The Scattered CS Photons

The MCRaT code spatially rebins the CS photons in order to keep the number of photons that it needs to track down to a reasonable amount, which helps decrease computational time. In order to rebin the photons, MCRaT finds the photon with the smallest position in the polar direction, $\theta_{\rm min}$, and the photon with the largest position in the polar direction, $\theta_{\rm max}$. These typically correspond to the values that were specified in the `mc.par` file that is read in by MCRaT. Then the size of each spatial bin, $\Delta\theta$ is set by the user through the `CYCLOSYNCHROTRON_REBIN_ANG` parameter in the `mcrat_input.h` file. The number of spatial bins, N_θ is then is determined as

$$N_\theta = \frac{\theta_{\rm max} - \theta_{\rm min}}{\Delta\theta} \tag{14}$$

The photons are also binned into energy bins, where the size of the energy bins, ΔE is determined as

$$\Delta E = \frac{\log(E_{\rm max}) - \log(E_{\rm min})}{N_E} \tag{15}$$

where $E_{\rm max}$ and $E_{\rm min}$ are the minimum and maximum energies of the CS photons and N_E is the number of energy bins acquired by multiplying the `CYCLOSYNCHROTRON_REBIN_E_PERC` value times the value of the $17^{\rm th}$ line of the `mc.par` file, N_{17}.

To ensure that the rebinning is efficient, MCRaT checks that $N_\theta N_E < N_{17}$. If it is not, the the code throws an error. The user can then change either parameter to ensure that the aforementioned condition is met.

When rebinning the photons in polar angle and energy, the following values of photons that fall within a given energy and θ bin are averaged and then assigned to a new rebinned photon (the quantities for this rebinned photon are represented with a tilde). For a given energy bin i and θ bin j, the new rebinned photon would have the following four momentum

$$\tilde{p}^\mu_{i,j} = \frac{< E >}{c} \begin{pmatrix} 1 \\ \sin(<\theta>)\cos(<\phi>) \\ \sin(<\theta>)\sin(<\phi>) \\ \cos(<\theta>) \end{pmatrix}$$

where θ and ϕ are the directional angles of the 4 momentum and the $< \cdots >$ denote weighted averages of the various quantities over the photons that fall within bin (i, j). The stokes parameters for the rebinned photon becomes $\tilde{s} = (1, < q >, < u >, < v >)$. The photon weight becomes $\tilde{w} = \sum w_{i,j}$. The position vector of the rebinned photon becomes

$$\overrightarrow{\tilde{r}}_{i,j} =< r > \begin{pmatrix} \sin(< \theta_p >) \cos(< \phi > - < \Delta\tilde{\phi}_p >) \\ \sin(< \theta_p >) \sin(< \phi > - < \Delta\tilde{\phi}_p >) \\ \cos(< \theta_p >) \end{pmatrix}$$

where $< r >$ is the average radius of the photons that lie in bin (i, j), $< \theta_p >$ is the average polar angle of the photons with the given bins and , for these same photons, $< \Delta\tilde{\phi}_p >$ is the average displacement angle taken across each photon's 4-momentum azimuthal angle, ϕ, and its positional azimuthal angle, ϕ_p, that is $\Delta\tilde{\phi}_p = \phi - \phi_p$. This consideration of the displacement in the positional and 4-momentum azimuthal angles is necessary to ensure that the rebinned photons are still in appropriate equilibrium with the flow.

4.5.4 Absorption of CS Photons

At the end of each HD frame, MCRaT proceeds to absorb any photons that have a frequency $\nu \leq \nu_B$ of the HD cell that they are located within. As a result, all of the CS photons that were emitted and never upscattered become absorbed and any photons that have been scattered but still satisfy $\nu \leq \nu_B$ are also absorbed.

4.6 Non-Thermal Synchrotron Emission, Self-Absorption, and Scattering Bias

The preceding section described the *thermal* cyclo-synchrotron treatment (Section 4.5), which is enabled by the `CYCLOSYNCHROTRON_SWITCH` and implemented in `mc_cyclosynch.c`; there, photons are emitted at the local cyclotron frequency, rebinned, and absorbed below ν_B. This section describes a distinct, more detailed *non-thermal* synchrotron capability, enabled by the `SYNCHROTRON_SWITCH` together with a power-law `NONTHERMAL_E_DIST` and implemented in `mc_synchrotron.c`, which models emission, self-absorption, and biased Compton scattering for a power-law electron population by sampling a universal, precomputed frequency CDF.

Implementation note. The thermal cyclo-synchrotron path (Section 4.5) and the non-thermal synchrotron path described here are *mutually exclusive*: MCRaT will refuse to build if both `CYCLOSYNCHROTRON_SWITCH` and `SYNCHROTRON_SWITCH` are enabled simultaneously. The two paths use different source modules (`mc_cyclosynch.c` versus `mc_synchrotron.c`) and different emission prescriptions (emission at ν_B versus sampling of the universal photon-number CDF). Both perform periodic Russian-roulette and rebinning operations that share the "every 1000 scatterings" cadence referenced in Section 3.2, but they act on different photon populations; the cadence statements in the two sections therefore describe analogous but independent machinery and are not in conflict.

Before working through the detailed physics, Section 4.1 gives a brief, self-contained primer on Monte Carlo radiation transport for readers new to the method.

4.6.1 Roadmap: the life cycle of a synchrotron packet

Before working through the detailed physics in the remainder of this section, it is useful to have a bird's-eye view of everything that happens to a single synchrotron photon packet. Section numbers point to where each step is described in detail.

1. **Emission** (Section 4.6.5): a cell with non-thermal electrons and $B > 0$ emits a Poisson-distributed number of packets per hydro frame, with a frequency sampled from a universal, precomputed CDF and an isotropic-in-intent (but see the known defect in Section 4.6.5) comoving direction.

2. **Transport** (Section 4.6.6): the packet's 4-momentum is Lorentz-boosted to the lab frame and it is propagated in straight lines between scattering/absorption events.

3. **Continuous self-absorption** (Section 4.6.6): at every transport sub-step, the packet's weight is multiplied by $\exp(-d\tau_{\nu_f}^{(a)})$, where $d\tau_{\nu_f}^{(a)}$ includes the full relativistic frame correction $\Gamma(1 - \beta\cos\theta)$.

4. **Biased Compton scattering** (Section 4.6.8): the mean free path to the next scattering is drawn using an artificially enhanced ("biased") optical depth so that rare, high-energy non-thermal electron subgroups are adequately sampled; the packet is split into a scattered and an unscattered component with weights corrected to preserve unbiasedness.

5. **Russian roulette** (Section 4.6.9): every 1000 scatterings, and once more at the end of each *eligible intermediate* hydro frame (the pass is skipped for a packet batch's initial scattering frame and for the final hydro frame; see Section 4.6.10), packets whose weight has fallen far below the population median

6. **Rebinning and output**: surviving packets are merged if the photon list approaches capacity (Section 4.6.9), and their weighted contribution to spectra, light curves, or images is ultimately accumulated via the estimator (1).

4.6.2 Enabling and Initializing the Module

The nonthermal synchrotron module is controlled primarily through compile-time settings in `mcrat_input.h`. A complete rebuild is required after changing these macros.

Required switches At minimum, enable the synchrotron module and select one supported nonthermal electron distribution:

```
#define SYNCHROTRON_SWITCH ON

/* Choose exactly one of these distributions. */
#define NONTHERMAL_E_DIST POWERLAW
/* or:
#define NONTHERMAL_E_DIST BROKENPOWERLAW
*/
```

For a single power law, specify

```
#define POWERLAW_INDEX ...
#define GAMMA_MIN ...
#define GAMMA_MAX ...
```

For a broken power law, specify

```
#define POWERLAW_INDEX_1 ...
#define POWERLAW_INDEX_2 ...
#define GAMMA_MIN ...
#define GAMMA_BREAK ...
#define GAMMA_MAX ...
```

The nonthermal scattering implementation requires tabulated cross sections and scattering bias:

```
#define TAU_CALCULATION TABLE
#define SCATTERING_BIAS_SWITCH ON
```

The magnetic field must be obtained either from the hydro simulation or from one of MCRaT's parameterized prescriptions:

```
#define B_FIELD_CALC SIMULATION
/* or INTERNAL_E / TOTAL_E with the corresponding EPSILON_B setting */
```

Useful optional settings include

```
#define NONTHERMAL_CONVERSION_FACTOR ...
#define SCATTERING_BIAS_SCALING ...
#define RR_WEIGHT_FRACTION ...
#define N_GAMMA ...
```

Required parameter ordering and runtime validation The compile-time checks verify primarily that the required macros exist; they do not validate every numerical relationship needed by the physics. Before a production run, the user should verify

$$1 \leq \gamma_{\min} < \gamma_{\max}, \tag{16}$$

and, for a broken power law,

$$\gamma_{\min} < \gamma_{\rm br} < \gamma_{\max}. \tag{17}$$

For the physical applications described in this guide, the power-law indices should normally satisfy

$$p > 1, \qquad p_1 > 1, \qquad p_2 > 1, \tag{18}$$

unless the corresponding logarithmic normalization limits are explicitly intended and tested.

The following quantities must also be finite and positive:

$$\text{fps}, \qquad B, \qquad V_i, \qquad n_{e,\rm nth}, \qquad w_0. \tag{19}$$

The requested packet interval should satisfy

$$0 \leq N_{\min} \leq N_{\max}. \tag{20}$$

Known issue. The current `photonEmitSynch` weight-tuning loop has no explicit iteration bound and no early exit for a zero or non-finite total physical emission rate. A configuration with

$$\sum_i W_i \leq 0 \tag{21}$$

and a positive `min_photons` has no feasible solution. Such a configuration can cause the tuning loop to fail to converge. Production runs should therefore validate the total emission rate before entering the loop.

Initialization order The main program initializes the required shared tables before photon transport:

1. `initalizeHotCrossSection` loads or creates the thermal and nonthermal scattering tables.
2. `initSynchTables` constructs $F(x)$, $G_a(x;p)$, the legacy inverse CDFs, and the universal $\tilde{\nu}$ photon-number CDF.
3. `initGlobalThreadRNG` creates one random-number generator per OpenMP thread.

These initialization routines must complete before any call to `photonEmitSynch`, `synchAlphaNu`, or `calculateOpticalDepthSSA`.

At shutdown, the associated resources are released by `cleanupInterpolationData`, `freeGlobalThreadRNG`, and `freeSynchTables`.

Runtime sequence For each hydro frame, MCRaT:

1. reads the hydrodynamic cell data;
2. calculates the nonthermal electron density;
3. emits synchrotron packets in the selected shell;
4. finds the containing hydro cell for every live packet;
5. computes scattering and absorption opacities;
6. advances all packets to the next event or frame boundary;
7. applies continuous SSA attenuation during every propagation step;
8. performs biased Compton scattering when selected;
9. periodically applies Russian roulette and rebinning;
10. writes packet data and a restart checkpoint.

Implementation note. The universal synchrotron tables depend on compile-time electron parameters such as p, $\gamma_{\min}$, $\gamma_{\max}$, and $\gamma_{\rm br}$. Changing any of these values requires rebuilding the executable and reconstructing the tables.

4.6.3 Non-Thermal Electron Distribution

Synchrotron radiation is produced by relativistic electrons spiralling around magnetic field lines. The electron distribution in each fluid cell is specified as a power law in the Lorentz factor γ (the ratio of the electron's total energy to its rest-mass energy $m_e c^2$).

Single power law The simplest model is a single power law:

$$N(\gamma) = A\,\gamma^{-p}, \qquad \gamma_{\min} \le \gamma \le \gamma_{\max}, \tag{22}$$

where $N(\gamma)\,d\gamma$ is the number of non-thermal electrons per unit volume with Lorentz factors between γ and $\gamma + d\gamma$. The normalisation constant A is fixed by requiring $\int_{\gamma_{\min}}^{\gamma_{\max}} N(\gamma)\,d\gamma = n_{e,\mathrm{nth}}$:

$$A = n_{e,\mathrm{nth}}\,A_{\mathrm{norm}}, \qquad A_{\mathrm{norm}} \equiv \frac{1-p}{\gamma_{\max}^{1-p} - \gamma_{\min}^{1-p}}, \qquad p \neq 1. \tag{23}$$

Key result. Notation: throughout the rest of this document (e.g. Eq. (46) and Section 4.6.5), A_{norm} always denotes the *dimensionless*, $n_{e,\mathrm{nth}}$-independent piece of the normalisation defined here, i.e. $N(\gamma) = n_{e,\mathrm{nth}}\,A_{\mathrm{norm}}\,\gamma^{-p}$. The code function `powerLawNorm` (`electron.c`) takes no $n_{e,\mathrm{nth}}$ argument and returns A_{norm}, *not* the full, $n_{e,\mathrm{nth}}$-dependent A above; the $n_{e,\mathrm{nth}}$ factor is applied separately by every caller (e.g. `n_e_nth` in `synchAlphaNu`, or $n_{e,\mathrm{nth}}^{(i)}$ in W_i, Eq. (48)).

The power-law index $p > 1$ is a compile-time constant set by the user via `POWERLAW_INDEX`.

Broken power law A more physically motivated model allows the slope of the distribution to change at a break Lorentz factor γ_{br}:

$$N(\gamma) = A \begin{cases} \gamma^{-p_1}, & \gamma_{\min} \le \gamma \le \gamma_{\mathrm{br}}, \\ \gamma_{\mathrm{br}}^{p_2-p_1}\,\gamma^{-p_2}, & \gamma_{\mathrm{br}} < \gamma \le \gamma_{\max}. \end{cases} \tag{24}$$

The dimensionless normalization is obtained from

$$\int_{\gamma_{\min}}^{\gamma_{\max}} A_{\mathrm{norm}}^{(\mathrm{br})} f_{\mathrm{br}}(\gamma)\,d\gamma = 1, \tag{25}$$

where $f_{\mathrm{br}}(\gamma)$ denotes the two unnormalised segments in Eq. (24). For $p_1 \neq 1$ and $p_2 \neq 1$,

$$A_{\mathrm{norm}}^{(\mathrm{br})} = \left[\frac{\gamma_{\mathrm{br}}^{1-p_1} - \gamma_{\min}^{1-p_1}}{1-p_1} + \gamma_{\mathrm{br}}^{p_2-p_1} \frac{\gamma_{\max}^{1-p_2} - \gamma_{\mathrm{br}}^{1-p_2}}{1-p_2} \right]^{-1}. \tag{26}$$

The full density-dependent coefficient is

$$A = n_{e,\mathrm{nth}} A_{\mathrm{norm}}^{(\mathrm{br})}. \tag{27}$$

The logarithmic limiting forms implemented by `brokenPowerLawNorm` must be used when either power-law index is equal, or sufficiently close, to unity. The factor $\gamma_{\mathrm{br}}^{p_2-p_1}$ ensures the distribution is continuous at $\gamma = \gamma_{\mathrm{br}}$ (Kawashima et al., 2023, Raikou Eqs. A4–A5). Writing $A = n_{e,\mathrm{nth}}\,A_{\mathrm{norm}}^{(\mathrm{br})}$ analogously to the single power-law case, it is the dimensionless piece $A_{\mathrm{norm}}^{(\mathrm{br})}$ that is computed by `brokenPowerLawNorm` in `electron.c`; like `powerLawNorm`, this function takes no $n_{e,\mathrm{nth}}$ argument.

Determining the total nonthermal electron number density For a custom outflow, the user may assign $n_{e,\mathrm{nth}}$ directly. In that case the custom-outflow implementation is responsible for populating `hydro_data->nonthermal_dens` consistently in every cell.

Otherwise, MCRaT derives the nonthermal electron density from the local magnetic energy density. The magnetic field may be read from an MHD simulation or inferred from a parameterized fraction of the internal or total energy density, depending on `B_FIELD_CALC`.

Let f_e = `NONTHERMAL_CONVERSION_FACTOR` be the fraction of the magnetic energy density assigned to the nonthermal electron population. MCRaT imposes

$$n_{e,\mathrm{nth}}\langle\gamma\rangle m_e c^2 = f_e \frac{B^2}{8\pi}. \tag{28}$$

The mean Lorentz factor is

$$\langle\gamma\rangle = \frac{1}{n_{e,\mathrm{nth}}} \int_{\gamma_{\min}}^{\gamma_{\max}} \gamma N(\gamma)\, d\gamma = \int_{\gamma_{\min}}^{\gamma_{\max}} \gamma A_{\mathrm{norm}} f(\gamma)\, d\gamma, \tag{29}$$

where $A_{\mathrm{norm}} f(\gamma)$ is normalized to unit integral.

Solving Eq. (28) gives

$$n_{e,\mathrm{nth}} = \frac{f_e B^2/(8\pi)}{\langle\gamma\rangle m_e c^2}. \tag{30}$$

In addition to the electron-energy assignment in Eq. (28), the current implementation applies a more stringent baryonic-loading constraint. Assuming one proton is associated with each accelerated electron, it requires

$$n_{e,\mathrm{nth}} m_p c^2 < \frac{B^2}{8\pi}. \tag{31}$$

Combining Eqs. (28) and (31) gives

$$f_e < \langle\gamma\rangle \frac{m_e}{m_p}. \tag{32}$$

This is the condition checked by `calculateNonthermalElectronDens`. Because $m_p \gg m_e$, it is more restrictive than requiring the nonthermal electron energy alone to remain below the magnetic energy density.

Implementation note. The current source always applies the baryonic form Eq. (32); it does not select m_e or m_p from a runtime composition setting. For a purely leptonic or pair-dominated outflow, the custom-outflow capability may be used to provide $n_{e,\mathrm{nth}}$ directly, or the source guard must be generalized to include an explicit composition model.

4.6.4 Universal Spectral Tables

Many of the integrals that appear in synchrotron physics depend only on a dimensionless frequency ratio x, not on the specific values of B or ν_f in a given cell. MCRaT exploits this by precomputing two universal functions once at startup and storing them as spline-interpolated lookup tables, avoiding expensive numerical integration at every transport step.

$F(x)$ **? single-electron synchrotron spectrum** The power radiated by a single relativistic electron in a magnetic field at a frequency characterised by the dimensionless ratio x is proportional to (Kawashima et al., 2023; Rybicki & Lightman, 1979, RAIKOU Eq. B13; R&L79 Eq. 6.31):

$$F(x) = x \int_x^\infty K_{5/3}(\xi)\, d\xi, \tag{33}$$

where $K_{5/3}$ is the modified Bessel function of order 5/3. The argument x is the ratio of the photon frequency to the characteristic frequency of an electron with Lorentz factor γ:

$$x = \frac{\nu_f}{\gamma^2\, \nu_c}, \qquad \nu_c = \frac{3\, e\, B}{4\pi\, m_e\, c}, \tag{34}$$

where ν_c is the pitch-angle–averaged critical frequency (Rybicki & Lightman, 1979, R&L79 Eq. 6.17c, pitch-angle averaged). Note that RAIKOU Eq. B7 gives the distinct *thermal* characteristic frequency $\nu_s \propto \Theta_e^2 \sin\theta_B$ and is not the quantity used here; the non-thermal $\nu_c = 3eB/(4\pi m_e c)$ adopted by MCRAT is the standard Rybicki–Lightman form.

$F(x)$ peaks near $x \approx 0.3$ and decays exponentially for $x \gg 1$, so most synchrotron emission comes from electrons whose critical frequency matches the photon frequency. The function is tabulated on a log-spaced grid of `SYNCH_N_X` points over $x \in$ [`SYNCH_X_MIN`, `SYNCH_X_MAX`] using GSL adaptive quadrature. In the current `mc_synchrotron.h` these default to `SYNCH_N_X`= 2000, `SYNCH_X_MIN`= 10^{-5}, and `SYNCH_X_MAX`= 50; a reader changing these macros should treat the numerical bounds here as illustrative of the defaults only.

$G_a(x;\, p)$ **? absorption spectral integral** The self-absorption coefficient involves a second universal function (Kawashima et al., 2023, RAIKOU Eq. C3):

$$G_a(x;\, p) = \int_x^\infty z^{(p-2)/2}\, F(z)\, dz, \tag{35}$$

which is the integral of $F(z)$ weighted by $z^{(p-2)/2}$ from x to infinity. For a single power law one table is built at the index p; for a broken power law two tables are built at p_1 and p_2 respectively.

Key result. Both $F(x)$ and $G_a(x;p)$ are evaluated at runtime not by simple clamping at the table boundaries [`SYNCH_X_MIN`, `SYNCH_X_MAX`], but by an *analytic continuation* below $x_0 \equiv$ `SYNCH_X_MIN`. This continuation uses the physical low-frequency asymptote of the single-electron spectrum (Rybicki & Lightman, 1979, R&L79 Eq. 6.31b),

$$F(x) \sim F(x_0)\, (x/x_0)^{1/3}, \qquad x \ll x_0, \tag{36}$$

implemented in `evalF`. For $x < x_0$, split the definition of G_a at the lower table boundary,

$$G_a(x;p) = G_a(x_0;p) + \int_x^{x_0} z^{(p-2)/2} F(z)\, dz, \tag{37}$$

and substitute the asymptote (36) into the added segment:

$$G_a(x;p) - G_a(x_0;p) \simeq \frac{F(x_0)}{x_0^{1/3}} \int_x^{x_0} z^{(p-2)/2+1/3}\, dz = \frac{F(x_0)}{x_0^{1/3}\, q}\left(x_0^{\,q} - x^{\,q}\right), \qquad q \equiv \frac{p-2}{2} + \frac{1}{3} + 1 = \frac{p}{2} + \frac{1}{3}, \tag{38}$$

where the final +1 in the exponent arises because integrating z^a produces $z^{a+1}/(a+1)$. For $x < x_0$ this gives

$$G_a(x;p) = G_a(x_0;p) + \frac{F(x_0)}{x_0^{1/3}\,q}\Big(x_0^{\,q} - x^{\,q}\Big), \qquad q = \frac{p}{2} + \frac{1}{3}, \qquad x < x_0. \tag{39}$$

This is implemented by `evalGa_p` (the evaluator actually called by `synchAlphaNu`), which computes exactly `q = 0.5 * p + (1.0/3.0)`, matching the derivation above. Since $q > 0$ for all physical $p > 1$, $x^q \to 0$ as $x \to 0$, so $G_a(x;p)$ grows smoothly and remains finite as $x \to 0$ rather than being held flat at its tabulated boundary value. This is what allows $\alpha_{\nu_f}^{(f)}$ to exhibit the correct soft SSA turnover at low ν_f (rather than an artificially flattened one) whenever $x_{\min} = \nu_f/(\gamma_{\max}^2\nu_c)$ falls below `SYNCH_X_MIN` ? a realistic occurrence for high-B, low-frequency cells. In this finite-electron-bound regime the continuation also recovers the expected index-independent scaling

$$\alpha_{\nu_f}^{(f)} \;\propto\; \left(\frac{\nu_f}{\nu_c}\right)^{-(p+4)/2} x_{\max}^{\,q} \;\propto\; \nu_f^{-(p+4)/2}\,\nu_f^{\,q} \;=\; \nu_f^{-(p+4)/2+p/2+1/3} \;=\; \nu_f^{-5/3}, \tag{40}$$

where $q = p/2 + 1/3$ from Eq. (38) and the explicit p-dependence cancels, leaving the index-independent exponent $-5/3$.

Thread Safety and Per-Thread Accelerators Several of the precomputed tables in MCRaT ? the synchrotron universal spectral tables (Section 4.6.4) and the hot cross section tables (Section 4.6.7) ? are evaluated at runtime via GSL spline interpolation. GSL spline evaluation requires a `gsl_interp_accel` object, which caches the most recently used interval index to accelerate repeated lookups via bisection. These accelerator objects are *not* thread-safe: if two OpenMP threads share a single accelerator and call `gsl_spline_eval` (or `gsl_spline2d_eval`) concurrently, the cached state is corrupted, producing incorrect interpolated values or a segmentation fault.

MCRaT resolves this by allocating one independent set of accelerator objects per OpenMP thread at initialisation time. Two patterns are used, one for each table family:

Synchrotron tables (`SynchThreadAccel`). The struct `SynchThreadAccel` holds a complete set of `gsl_interp_accel` pointers for every *spline-based* lookup in the synchrotron universal tables: the $F(x)$ spline, the $G_a(x)$ spline (or splines, for the broken power law), and the inverse single-electron-x CDF spline and inverse pitch-angle CDF spline used by `synchSampleX` and `synchSampleAlpha` respectively. The latter two samplers draw a single-electron emission angle/frequency directly from $F(x)$ and are retained primarily for use by the cyclosynchrotron module (`mc_cyclosynch.c`); they are *not* called along the stratified non-thermal emission path described in Section 4.6.5 of this document.

Key result. The stratified frequency sampler `sampleSynchFrequency`, which *is* the sampler used by `emitCellPackets` in the emission path documented in Section 4.6.5, does *not* use any GSL spline or any `SynchThreadAccel` field. It performs a manual binary search directly on the raw universal-CDF arrays `nu_cdf_u`/`nu_cdf_log_nu_tilde` built by `buildUniversalNuCDF`, followed by linear

interpolation between the two bracketing points. Because this search operates on plain arrays rather than a `gsl_spline` object, it requires no accelerator at all and is trivially thread-safe without any per-thread state.

One `SynchThreadAccel` is allocated per thread in a file-scope array `synch_thread_accels` during `initSynchTables`. The helper function `getSynchAccel` returns a pointer to the accelerator for the calling thread:

```
static SynchThreadAccel *getSynchAccel(void)
{
 int thread_id = 0;
 #if defined(_OPENMP)
 thread_id = omp_get_thread_num();
 #endif
 if (thread_id >= synch_num_threads_allocated)
 thread_id = 0;
 return &synch_thread_accels[thread_id];
}
```

Every call to `gsl_spline_eval` or `gsl_spline2d_eval` on a synchrotron table (i.e. evaluations of $F(x)$, $G_a(x)$, or the legacy single-electron x/pitch-angle inverse CDFs) passes `getSynchAccel()->...` as the accelerator argument, ensuring no two threads share state.

Hot cross-section tables (`ThreadLocalAccel`). On MPI rank zero, `initalizeHotCrossSectionInte` constructs the thermal and nonthermal interpolation splines and initializes one `ThreadLocalAccel` structure per OpenMP thread.

Non-root MPI ranks reconstruct the interpolation splines after `broadcastInterpolationData`. In the current source they do not subsequently call `initThreadLocalAccelerators`. Their interpolation functions therefore use the safe fallback path, which allocates temporary accelerators for an out-of-range or unavailable thread-local slot. This fallback is thread-safe, but it is slower than persistent per-thread accelerators.

For consistent performance on every MPI rank, `initThreadLocalAccelerators(omp_get_max_threa` should be called after the interpolation structures have been constructed on each rank.

Global RNG pool (`global_thread_rng`). The same per-thread pattern is applied to the GSL random number generators. `initGlobalThreadRNG` allocates one `gsl_rng` per thread, seeding each from the master RNG. All functions that require random numbers inside a parallel region retrieve their thread-local generator via `global_thread_rng[omp_get_thread_nu` avoiding contention on a shared generator.

Key result. All shared read-only state (spline coefficients, table data) is accessed concurrently by multiple threads without locks, since GSL spline evaluation is read-only with respect to the spline object itself. All mutable per-evaluation state (accelerator caches, RNG state) is strictly private to each thread via the per-thread allocation described above. No `#pragma omp critical` sections are required for table lookups or random number generation.

4.6.5 Photon Emission

New synchrotron photon packets are injected into the simulation at each hydro frame by `photonEmitSynch` (`mc_synchrotron.c`). The procedure proceeds in four stages, corresponding to Sections 4.6.5–4.6.5 below: (1) computing the physical photon production rate and Poisson-drawing packet counts per cell (§4.6.5–4.6.5); (2) sampling a dimensionless frequency from the universal, stratified photon-number CDF (§4.6.5); (3) filling in the packet's comoving 4-momentum, Lorentz boost, birth position, and Stokes parameters (§4.6.5); and (4) appending the resulting packets to `photon_list` for transport.

Physical photon emission rate and cell weights The starting point for non-thermal synchrotron emission is the pitch-angle?averaged single-electron spectrum $F(x)$ introduced in Section 4.6.4.

For a power-law electron distribution written as

$$N(\gamma) = n_{e,\mathrm{nth}} A_{\mathrm{norm}} \gamma^{-p}, \qquad \gamma_{\mathrm{min}} \le \gamma \le \gamma_{\mathrm{max}}, \tag{41}$$

the pitch-angle-averaged synchrotron energy emissivity in the fluid frame has the schematic form

$$j^{(f)}_{\nu_f} \propto n_{e,\mathrm{nth}} A_{\mathrm{norm}} B \int_{\gamma_{\mathrm{min}}}^{\gamma_{\mathrm{max}}} \gamma^{1-p} F\!\left(\frac{\nu_f}{\gamma^2 \nu_c}\right) d\ln\gamma. \tag{42}$$

where $\nu_c = 3eB/(4\pi m_e c)$ is the pitch-angle?averaged critical frequency. Here A_{norm} is dimensionless. The full normalization introduced in Section 4.6.3 is $A = n_{e,\mathrm{nth}} A_{\mathrm{norm}}$ and must not be multiplied by $n_{e,\mathrm{nth}}$ a second time. The exact prefactors and detailed form of the integral are spelled out in RAIKOU; here we focus on how MCRAT turns this emissivity into a *photon-number* production rate that can be used for Monte Carlo emission.

The *photon-number* emissivity ? the rate at which photons of frequency ν_f are produced per unit volume, per unit frequency ? is obtained by dividing by the energy per photon $h\nu_f$:

$$\dot{n}^{(f)}_{\nu_f} = \frac{j^{(f)}_{\nu_f}}{h\nu_f} \quad \left[\text{photons cm}^{-3}\ \text{s}^{-1}\ \text{Hz}^{-1}\right]. \tag{43}$$

MCRAT uses the fact that the spectrum depends only on the dimensionless ratio $\tilde{\nu} = \nu_f/\nu_c$ and the power-law index p, so the *shape* of the photon-number emissivity can be precomputed once and reused in all cells.

The total number of photons produced per unit time in cell i, integrated over all frequencies, is

$$\dot{N}_i = V_i \int_0^\infty \dot{n}^{(f)}_{\nu_f}\, d\nu_f = \frac{V_i}{h} \int_0^\infty \frac{j^{(f)}_{\nu_f}}{\nu_f}\, d\nu_f, \tag{44}$$

where V_i is the cell volume. After substituting Eq. (42) and changing variables to $\tilde{\nu} = \nu_f/\nu_c$, the cell-dependent factors separate cleanly and the photon production rate per simulation frame $\Delta t_{\mathrm{fps}} = 1/\mathrm{fps}$ can be written as

$$\Lambda_i \equiv W_i = K_{\mathrm{phys}}\, n^{(i)}_{e,\mathrm{nth}}\, B_i\, V_i \quad \left[\text{photons frame}^{-1}\right]. \tag{45}$$

Here K_{phys} is a cell-independent emission-rate prefactor evaluated once per call to `photonEmitSynch`. It depends on the compile-time electron-distribution parameters,

the universal table normalisation constructed at startup, and the current value of `hydro_data->fps`:

$$K_{\rm phys} = \frac{\sqrt{3}\,e^3}{m_e c^2 h}\, A_{\rm norm}\, \ln(10)^2\, \nu_{\rm cdf_norm}\, \Delta t_{\rm fps}, \qquad \Delta t_{\rm fps} \equiv \frac{1}{\rm fps}. \tag{46}$$

The absence of an additional factor of 1/2 reflects the convention used by the updated implementation: the single-electron synchrotron spectral power is pitch-angle averaged and integrated over emission solid angle. In this convention,

$$P_{\nu_f}(\gamma) = \frac{\sqrt{3}\,e^3 B}{m_e c^2} F\left(\frac{\nu_f}{\gamma^2 \nu_c}\right), \tag{47}$$

and the cell photon-production rate follows directly from integrating $P_{\nu_f}/(h\nu_f)$ over photon frequency and the electron distribution.

The remaining factors in Eq. (46) have the following origins:

- $A_{\rm norm}$ is the dimensionless electron-distribution normalisation, defined by $N(\gamma) = n_{e,\rm nth} A_{\rm norm} f(\gamma)$.
- $\nu_{\rm cdf_norm}$ is the unnormalised numerical double integral over $\log_{10}\tilde{\nu}$ and $\log_{10}\gamma$, constructed by `buildUniversalNuCDF`.
- The factor $\ln(10)^2$ converts both base-10 logarithmic integration measures to natural-logarithmic measures.
- $\Delta t_{\rm fps} = 1/\rm fps$ converts photons per second to photons per simulation frame.

The resulting physical photon count emitted by cell i during one simulation frame is

$$W_i = K_{\rm phys}\, n_{e,\rm nth}^{(i)}\, B_i\, V_i \qquad [\text{photons frame}^{-1}]. \tag{48}$$

The implementation evaluates the absolute photon-production rate without an arbitrary proportionality constant. Its numerical value is determined by the adopted pitch-angle/solid-angle convention and by the finite $\tilde{\nu}$ and γ integration grids used to construct `nu_cdf_norm`. Convergence should therefore be checked with respect to `SYNCH_N_NU`, `SYNCH_N_GAMMA_INT`, and the table boundaries.

Key result. $K_{\rm phys}$'s final factor converts a rate in photons $\rm s^{-1}$ to a rate in photons $\rm frame^{-1}$. This requires *dividing* by `fps`, i.e. multiplying by $\Delta t_{\rm fps} = 1/\rm fps$, as written in Eq. (46) above and exactly as computed by `photonEmitSynch` (`K_phys ... / hydro_data->fps`).

Key result. $W_i = K_{\rm phys}\, n_{e,\rm nth}^{(i)}\, B_i\, V_i$ is the *expected physical photon count* emitted by cell *i per simulation frame* [photons $\rm frame^{-1}$] — i.e. the per-second production rate already multiplied by the frame duration $\Delta t_{\rm fps} = 1/\rm fps$ — not a mere proportionality constant. We reserve the word "rate" for per-second quantities and "count" for the per-frame W_i.

Note that W_i scales *linearly* with B, not B^2: one power of $\nu_c \propto B$ in the numerator of $j_{\nu_f}^{(f)}$ is cancelled by one power of $\nu_f \propto \nu_c$ in the denominator of $j_{\nu_f}^{(f)}/(h\nu_f)$.

Each photon packet in MCRAT carries a weight w_0 [photons packet^{-1}] representing the number of real photons it stands in for. Since W_i already has units of photons frame^{-1}, the expected number of packets drawn from cell i is simply

$$\lambda_i = \frac{W_i}{w_0} \quad \left[\text{packets frame}^{-1}\right]. \tag{49}$$

No additional fps factor is needed elsewhere in the emission procedure. A Poisson draw with mean λ_i gives the actual packet count n_i for cell i; the combination of Eqs. (48) and (49) guarantees that the *expected* total weight carried by all packets from cell i equals the physical number of photons Λ_i emitted per frame.

Universal photon-number emissivity The implementation is most transparently derived from the pitch-angle-averaged, solid-angle-integrated single-electron spectral power,

$$P_{\nu_f}(\gamma) = \frac{\sqrt{3}\, e^3 B}{m_e c^2} F\left(\frac{\nu_f}{\gamma^2 \nu_c}\right), \qquad \nu_c = \frac{3eB}{4\pi m_e c}. \tag{50}$$

This convention is the one used to construct the updated physical prefactor in `photonEmitSynch`.

For any normalized nonthermal electron distribution written as

$$N(\gamma) = n_{e,\mathrm{nth}} A_{\mathrm{norm}} f(\gamma), \qquad \int_{\gamma_{\min}}^{\gamma_{\max}} A_{\mathrm{norm}} f(\gamma)\, d\gamma = 1, \tag{51}$$

the energy emitted per unit volume, time, and frequency is

$$\mathcal{P}_{\nu_f} = n_{e,\mathrm{nth}} A_{\mathrm{norm}} \frac{\sqrt{3}\, e^3 B}{m_e c^2} \int_{\gamma_{\min}}^{\gamma_{\max}} f(\gamma) F\left(\frac{\nu_f}{\gamma^2 \nu_c}\right) d\gamma. \tag{52}$$

The photon-number emissivity per unit linear frequency is obtained by dividing by the photon energy:

$$\dot{n}_{\nu_f} = \frac{\mathcal{P}_{\nu_f}}{h\nu_f}. \tag{53}$$

The explicit factor $1/\nu_f$ disappears only when the integration is written in logarithmic frequency:

$$\dot{n}_{\nu_f}\, d\nu_f = \frac{\mathcal{P}_{\nu_f}}{h}\, d\ln \nu_f. \tag{54}$$

Define the dimensionless frequency

$$\tilde{\nu} \equiv \frac{\nu_f}{\nu_c}. \tag{55}$$

Since $d\ln \nu_f = d\ln \tilde{\nu}$, all magnetic-field dependence separates from the dimensionless spectral shape.

For a single power law,

$$f(\gamma) = \gamma^{-p}, \tag{56}$$

and the dimensionless photon-number kernel per unit $\ln \tilde{\nu}$ is

$$\Phi(\tilde{\nu}) = \int_{\ln \gamma_{\min}}^{\ln \gamma_{\max}} \gamma^{1-p} F\left(\frac{\tilde{\nu}}{\gamma^2}\right) d\ln \gamma. \tag{57}$$

For a broken power law,

$$f(\gamma) = \begin{cases} \gamma^{-p_1}, & \gamma_{\min} \le \gamma \le \gamma_{\rm br}, \\ C_{\rm cont}\gamma^{-p_2}, & \gamma_{\rm br} < \gamma \le \gamma_{\max}, \end{cases} \qquad C_{\rm cont} = \gamma_{\rm br}^{p_2-p_1}, \tag{58}$$

the corresponding kernel is

$$\begin{aligned} \Phi_{\rm br}(\tilde{\nu}) = & \int_{\ln\gamma_{\min}}^{\ln\gamma_{\rm br}} \gamma^{1-p_1} F\!\left(\frac{\tilde{\nu}}{\gamma^2}\right) d\ln\gamma \\ & + C_{\rm cont}\int_{\ln\gamma_{\rm br}}^{\ln\gamma_{\max}} \gamma^{1-p_2} F\!\left(\frac{\tilde{\nu}}{\gamma^2}\right) d\ln\gamma. \end{aligned} \tag{59}$$

The code evaluates the appropriate kernel numerically on a $\log_{10}\tilde{\nu}$–$\log_{10}\gamma$ grid. Its unnormalised double integral is stored as `nu_cdf_norm`. Therefore,

$$\int d\ln\tilde{\nu}\,\Phi(\tilde{\nu}) = \ln(10)^2\,\nu_{\rm cdf_norm}, \tag{60}$$

with the same relation applying to the broken-power-law kernel.

The photon production rate from cell i during one hydro frame is then

$$W_i = K_{\rm phys} n_{e,\rm nth}^{(i)} B_i V_i, \tag{61}$$

where $K_{\rm phys}$ is given by Eq. (46).

Implementation note. The analytic functions $G(x;p)$ used in closed-form presentations of the power-law synchrotron emissivity are useful for comparison with RAIKOU, but they are not evaluated by the MCRAT emission path. MCRAT constructs the photon-number kernel directly from $F(x)$ in `buildUniversalNuCDF`. This avoids ambiguity between per-solid-angle and solid-angle-integrated emissivity conventions.

Packet weight, weight-tuning loop, and Poisson draw Each photon packet carries a weight w_0 [photons packet^{-1}] that represents the number of real photons it stands in for. Because W_i is already the physical photon production rate per frame (48), the expected number of packets drawn from cell i is given by Eq. (49), repeated here for convenience:

$$\lambda_i = \frac{W_i}{w_0} \quad [\text{packets frame}^{-1}].$$

No additional fps factor is needed anywhere in the emission procedure.

Because w_0 is a free parameter, it is adjusted iteratively before the Poisson draw so that the total expected packet count lies within user-specified bounds $[N_{\min}, N_{\max}]$:

$$N_{\min} \;\le\; \sum_i \frac{W_i}{w_0} \;\le\; N_{\max}. \tag{62}$$

The loop multiplies w_0 by 10 when $\sum_i \lambda_i > N_{\max}$ and by 0.5 when $\sum_i \lambda_i < N_{\min}$, until (62) is satisfied. The converged value is stored as `ph_weight_adjusted`.

Once w_0 has converged, the actual packet count per cell is drawn from a Poisson distribution:

$$n_i \sim \text{Poisson}\left(\frac{W_i}{w_0}\right), \tag{63}$$

The Poisson draw produces the Monte Carlo fluctuation associated with the finite number of sampled packets. Because $\mathbb{E}[n_i w_0] = W_i$, the emitted photon-number estimator is unbiased. This packet-count variance should not automatically be identified with the physical photon shot noise of the source, because one packet represents many physical photons. The total number of packets to fill is $N_{\rm tot} = \sum_i n_i$.

Degenerate emission fallbacks. Two edge cases in `emitCellPackets` are handled without contributing physical weight to the simulation:

- **Nonpositive magnetic field.** `emitCellPackets` contains a local fallback that would assign zero weight and a placeholder frequency if called with $B_{\rm cell} \leq 0$. In the normal emission path, however, such a cell has $W_i = 0$ and normally draws no packets. If all active cells have zero physical emission while `min_photons` is positive, the preceding weight-tuning loop has no feasible solution and may fail to converge. Users must therefore ensure that cells counted as active have finite, positive B and finite, positive W_i.

  A source-level guard should test

  $$W_{\rm total} = \sum_i W_i \tag{64}$$

  before entering the tuning loop and return without emission when $W_{\rm total} \leq 0$.

- If a stratum k has zero probability mass, $p_k = 0$ (66) (i.e. the universal CDF has no support in that frequency range), the packets allocated to that stratum are instead emitted as zero-weight packets at the stratum's log-frequency midpoint, $\tilde{\nu} = 10^{(\log \tilde{\nu}^{\min,k} + \log \tilde{\nu}^{\max,k})/2}$, converted to a physical comoving frequency via $\nu_f = \tilde{\nu}\, \nu_c(B_{\rm cell})$ exactly as in the normal sampling path (Section 4.6.5), rather than being silently dropped. This preserves the exact packet count n_i requested by the Poisson draw (63).

Photon-number CDF and stratified frequency sampling For emission, we need to sample photon frequencies according to the *photon-number* spectrum, i.e. proportional to $j^{(f)}_{\nu_f}/(h\nu_f)$, not to the energy spectrum $j^{(f)}_{\nu_f}$ alone. In log-frequency space this photon-number spectrum can be written schematically as

$$\text{pdf}(\tilde{\nu}) \propto \int_{\gamma_{\min}}^{\gamma_{\max}} \gamma^{1-p}\, F\left(\frac{\tilde{\nu}}{\gamma^2}\right) d\ln\gamma, \tag{65}$$

where $\tilde{\nu} = \nu_f/\nu_c$. The factor γ^{1-p} comes from combining the electron power law $N(\gamma) \propto \gamma^{-p}$, the single-electron spectrum F, and the kinematic Jacobian that relates photon frequency to electron Lorentz factor in the pitch-angle?averaged formalism.

MCRaT exploits the fact that Eq. (65) depends only on $\tilde{\nu}$, p, and the electron distribution parameters, not on the cell?s magnetic field B or volume V_i. At startup, `buildUniversalNuCDF` evaluates Eq. (65) on a grid of $\log_{10}\tilde{\nu}$ values and constructs a

single, universal CDF $\mathrm{CDF}(\tilde{\nu})$ and its normalisation `nu_cdf_norm`. This CDF encodes the photon-number emissivity shape in dimensionless frequency and is reused in every cell.

To improve sampling of the spectral tails, the log-frequency axis is divided into $K =$ `SYNCH_N_STRATA` equal intervals (*strata*). Let $\tilde{\nu}^{\min,k}$ and $\tilde{\nu}^{\max,k}$ be the edges of stratum k, and define its probability mass

$$p_k = \mathrm{CDF}(\tilde{\nu}^{\max,k}) - \mathrm{CDF}(\tilde{\nu}^{\min,k}), \qquad k = 0, \ldots, K-1. \tag{66}$$

These p_k are computed once from the universal CDF and stored in `tables->strata_p_k[k]`.

For a given cell with n_i packets, `emitCellPackets` divides the packets as equally as possible across strata (with any remainder distributed to the lowest-indexed strata). Each packet in stratum k is assigned an importance weight

$$w_k = w_0 \, p_k \, K, \tag{67}$$

where w_0 is the baseline packet weight and K is the number of strata. The factor $p_k K$ corrects for the forced equal allocation across strata: although we emit roughly n_i/K packets per stratum, each packet in stratum k is up-weighted by $p_k K$ so that the *weighted* spectrum matches the true photon-number CDF.

Within each stratum, `sampleSynchFrequency` draws a uniform variate $u \sim \mathcal{U}(\mathrm{CDF}(\tilde{\nu}^{\min,k}), \mathrm{CDF}(\tilde{\nu}^{\max}$ inverts the universal CDF to obtain $\log_{10} \tilde{\nu}$, and then rescales by the cell?s critical frequency $\nu_c(B_i)$ to get the physical comoving frequency $\nu_f = \tilde{\nu} \, \nu_c(B_i)$.

Known issue. The importance weight

$$w_k = w_0 p_k K \tag{68}$$

is exact when every stratum receives the same packet count, for example when n_i is divisible by the number of strata K. The current implementation distributes the integer remainder to the lowest-indexed strata:

$$n_k = \left\lfloor \frac{n_i}{K} \right\rfloor + \mathbf{1}_{k < n_i \bmod K}. \tag{69}$$

Consequently, the emitted cell weight is

$$W_{\mathrm{MC}} = w_0 K \sum_k n_k p_k, \tag{70}$$

which is not generally identical to $n_i w_0$.

If $n_i < K$, some strata receive no packets. Any nonzero probability mass in those unsampled strata cannot be recovered by importance weighting. The present scheme should therefore be described as an approximately weight-corrected stratification when the packet allocation is nearly equal and all relevant strata are populated.

For exact cell-level weight conservation in every populated stratum, one may instead use

$$w_k = w_0 \frac{n_i p_k}{n_k}, \tag{71}$$

provided every stratum with $p_k > 0$ has $n_k > 0$.

Single-photon kinematics (`synchFillPhoton`) Once ν_f and w_k are determined, `synchFillPhoton` fills the full photon-packet state:

1. **Comoving 4-momentum.** The comoving-frame photon energy-momentum magnitude is $|\mathbf{p}| = h\nu_f/c$. An emission direction $(\theta_{\rm ph}, \phi_{\rm ph})$ is sampled by calling `samplePhotonTheta` and `samplePhotonPhi`. The azimuthal angle $\phi_{\rm ph}$ is drawn uniformly from $[0, 2\pi)$. The polar angle $\theta_{\rm ph}$ is drawn from a distribution

$$f(\mu) \propto \tfrac{1}{2}\big(1 + \beta\,\mu\big), \qquad \mu = \cos\theta_{\rm ph}, \tag{72}$$

   where $\beta = v/c$ is the magnitude of the cell?s fluid velocity and the proportionality constant ensures proper normalisation over $\mu \in [-1, 1]$. This distribution slightly favours emission along the fluid velocity when $\beta > 0$; it is motivated by earlier work on anisotropic injection in hot flows. The helper `samplePhotonTheta` is intended to draw $f(\mu) = \frac{1}{2}(1 + \beta\mu)$. As documented below, the current call order and direction construction do not reliably realize this distribution relative to the local fluid velocity. A purely isotropic emission model in the fluid frame would instead use $f(\mu) = \frac{1}{2}$ (uniform in $\cos\theta_{\rm ph}$).

   In the current implementation, this mild anisotropy is applied even for synchrotron emission. It is therefore important to remember that while the *microphysics* (pitch-angle?averaged emissivity and absorption) assumes an isotropic electron distribution, the *emission* direction sampling in `synchFillPhoton` is slightly biased with respect to the fluid velocity.

2. **Lorentz boost.** The comoving 4-momentum $p^\mu_{\rm comv} = (h\nu_f/c)(1, \hat{\mathbf{n}}_{\rm comv})$ is boosted to the lab frame using the exact Lorentz transformation for a fluid element with velocity $\mathbf{v}_{\rm fluid}$. The same fluid velocity that enters the angular sampling is used in the boost, ensuring consistent kinematics between emission and transport.

3. **Birth position.** The code samples each native hydro coordinate uniformly across the coordinate bounds of the emitting cell and then converts the sampled point to MCRaT Cartesian coordinates. This is exactly uniform in physical volume for Cartesian cells. In curvilinear coordinates it is uniform in the coordinate variables, not generally in the physical volume element; see the known issue below.

4. **Stokes parameters.** The packet is born unpolarised, with Stokes vector $(i, q, u, v) = (1, 0, 0, 0)$. Synchrotron polarisation is not yet modelled.

5. **Bookkeeping.** The packet type is set to `SYNCH_PHOTON`; the flag `recalc_properties` is set to 1 so that scattering and absorption opacities are computed on the first transport step; and `absorption_opacity` is initialised to zero.

Known issue. For spherical, cylindrical, and polar grids, uniform sampling of the native coordinates does not produce a uniform physical-volume distribution because the volume element contains a coordinate Jacobian.

For example, a spherical cell has

$$dV = r^2 \sin\theta\, dr\, d\theta\, d\phi. \tag{73}$$

Exact uniform-volume sampling would require

$$r = \left[r_{\min}^3 + u_r \left(r_{\max}^3 - r_{\min}^3\right)\right]^{1/3}, \tag{74}$$

$$\cos\theta = \cos\theta_{\min} + u_\theta \left(\cos\theta_{\max} - \cos\theta_{\min}\right), \tag{75}$$

$$\phi = \phi_{\min} + u_\phi \left(\phi_{\max} - \phi_{\min}\right), \tag{76}$$

with independent uniform variates $u_r, u_\theta, u_\phi \sim \mathcal{U}(0,1)$.

The current coordinate-uniform sampling is an approximation whose error decreases for sufficiently small cells. It should not be described as exactly uniform in physical volume for curvilinear geometries.

Known issue. In earlier code revisions, `synchFillPhoton` (`mc_synchrotron.c`) and the legacy blackbody/Wien photon-injection routine `photonInjection` (`mclib.c`) both called `samplePhotonTheta(boost,...)` before `boost` had been populated by `hydroVectorToCartesian`, so that the polar angle was sampled using an undefined value of the fluid speed $\beta = \|\texttt{boost}\|$. **This use-before-initialization defect has been fixed**: both functions now call `hydroVectorToCartesian` to populate the local fluid-velocity vector *before* the corresponding call to `samplePhotonTheta`, so β is now a well-defined, correctly computed quantity at the point of sampling.

A second, independent limitation remains in the current code. `samplePhotonTheta` uses only the *magnitude* $\beta = \|\texttt{boost}\|$ of the supplied velocity, not its direction. The direction subsequently constructed as

$$\hat{\mathbf{n}}_{\rm comv} = (\sin\theta\cos\phi,\ \sin\theta\sin\phi,\ \cos\theta) \tag{77}$$

is expressed relative to the code's Cartesian z-axis, not relative to $\hat{\mathbf{v}}_{\rm fluid}$. Consequently, even with β now correctly defined, the sampled direction does not generally realize the intended distribution $f(\mu) = \frac{1}{2}(1+\beta\mu)$ with $\mu = \hat{\mathbf{n}}_{\rm comv} \cdot \hat{\mathbf{v}}_{\rm fluid}$, except in the degenerate case where the local fluid velocity happens to be aligned with the z-axis. Fully realizing the intended anisotropic distribution would additionally require rotating the sampled direction into an orthonormal basis whose polar axis is the local fluid-velocity direction. Alternatively, if strictly isotropic emission in the fluid frame is intended (as assumed by the pitch-angle-averaged microphysics of Section 4.6.5), `samplePhotonTheta` should instead sample $\mu = \cos\theta_{\rm ph} \sim \mathcal{U}(-1,1)$ independently of β.

Key result. The synchrotron microphysics in MCRaT (emissivity and self-absorption) assumes a pitch-angle?averaged, isotropic electron population in the fluid frame. The current emission-angle sampler introduces a mild anisotropy tied to the fluid velocity via $f(\mu) \propto 1 + \beta\mu$. If strictly isotropic emission in the fluid frame is desired, the implementation of `samplePhotonTheta` should be modified to sample $\cos\theta_{\rm ph}$ uniformly in $[-1,1]$ instead.

Modelling assumptions for synchrotron emission For clarity, we summarize the main physical assumptions underlying the synchrotron emission and absorption

implementation in MCRAT:

- **Pitch-angle?averaged, isotropic electrons.** The non-thermal electron distribution in each cell depends only on Lorentz factor γ and is assumed isotropic in pitch angle. All synchrotron emissivity and absorption formulas are taken in their pitch-angle?averaged, angle-integrated form.

- **Photon-number, not energy, sampling.** Emission is sampled from the photon-number spectrum $j_{\nu_f}^{(f)}/(h\nu_f)$ in log-frequency space, using a universal CDF over $\tilde{\nu} = \nu_f/\nu_c$. Packet weights represent real photon counts.

- **Mild anisotropy tied to fluid velocity (partially implemented).** The angular sampler `samplePhotonTheta` is intended to draw $f(\mu) \propto 1 + \beta\mu$ relative to the fluid velocity. The use-before-initialization defect that previously left β undefined at the point of sampling has been fixed (Section 4.6.5): `boost` is now populated before `samplePhotonTheta` is called. However, the sampler still does not rotate the sampled direction into a basis aligned with the fluid-velocity direction, so the intended β-weighted anisotropy relative to $\hat{\mathbf{v}}_{\rm fluid}$ is still not reliably realized; see Section 4.6.5 for the remaining defect and the corresponding fix needed.

- **Continuous self-absorption.** Synchrotron self-absorption is treated via continuous weight attenuation with the full relativistic frame correction $\Gamma(1 - \beta\cos\theta)$ applied to the comoving absorption coefficient. SSA acts only during transport, not at packet birth.

- **Importance sampling and bias correction.** Scattering events are biased to improve sampling in optically thin regions and rare non-thermal electron subgroups. Packet weights are split and corrected so that all observables remain statistically unbiased **provided every per-subgroup bias satisfies** $b_k \geq 1$. As detailed in the erratum in Section 4.6.8, the $\max(1, \ldots)$ floor that enforces $b_k \geq 1$ is currently disabled in `calculateNonthermalScatteringBias`, so this unbiasedness guarantee does not strictly hold for the default runtime behaviour as shipped; it holds exactly once the floor is restored, or in any cell/subgroup combination where $\tau_k \leq b_{\rm ref}\,\tau_{\rm ref}$ so that the unclamped ratio $b_{\rm ref}\tau_{\rm ref}/\tau_k$ already exceeds unity.

Built-in emission bookkeeping diagnostics After creating a batch of synchrotron packets, `photonEmitSynch` prints two bookkeeping diagnostics.

Packet-weight consistency. The code compares

$$W_{\rm packets} = \sum_i w_i \tag{78}$$

with

$$W_{\rm nominal} = N_{\rm tot} w_0^{\rm (adj)}. \tag{79}$$

For exactly equal stratum allocation these quantities agree by construction. With the current integer-remainder allocation, a finite deterministic discrepancy can remain; it is not purely Monte Carlo noise. A large discrepancy nevertheless remains useful for detecting packet-allocation or importance-weighting failures.

Energy-bookkeeping consistency. The emitted comoving energy is

$$E_{\text{emitted}} = \sum_i w_i h\nu_{f,i}. \tag{80}$$

The comparison quantity printed as `P_analytic` uses the physical photon-number rate together with the emission-weighted mean photon energy estimated from the same emitted packet batch:

$$P_{\text{analytic}} = \frac{\left(\sum_j W_j\right) \langle h\nu_f \rangle_{\text{sampled}}}{V_{\text{total}} \Delta t_{\text{fps}}}. \tag{81}$$

This is a useful internal bookkeeping check, but it is not an independent analytic validation of the synchrotron spectral kernel or of the absolute power normalization, because $\langle h\nu_f \rangle_{\text{sampled}}$ is derived from the same packet sample used to compute E_{emitted}.

4.6.6 Synchrotron Self-Absorption

Synchrotron self-absorption (SSA) occurs when synchrotron photons are reabsorbed by the same electron population that emitted them. It is important at low frequencies (below the SSA turnover frequency) where the plasma is optically thick to its own synchrotron radiation. In MCRaT, SSA is treated via continuous weight attenuation rather than stochastic absorption (see Section 4.1).

Absorption coefficient For a nonthermal electron distribution, the SSA absorption coefficient is obtained from the generalized detailed-balance relation between emission and absorption in continuum electron states, rather than from the thermal Kirchhoff relation alone (Rybicki & Lightman, 1979; Kawashima et al., 2023). Physically, $\alpha_{\nu_f}^{(f)}$ is the inverse absorption mean free path at comoving frequency ν_f.

Single power law (Kawashima et al., 2023, Raikou Eq. C2).

$$\alpha_{\nu_f}^{(f)} = \frac{(p-1)(p+2)\, n_{e,\text{nth}}\, e^2}{4\sqrt{3}\, m_e\, c\, (\gamma_{\min}^{1-p} - \gamma_{\max}^{1-p})\, \nu_c} \left(\frac{\nu_f}{\nu_c}\right)^{-(p+4)/2} \left[G_a(x_{\min}) - G_a(x_{\max})\right], \tag{82}$$

where

$$x_{\min} = \frac{\nu_f}{\gamma_{\min}^2\, \nu_c}, \qquad x_{\max} = \frac{\nu_f}{\gamma_{\max}^2\, \nu_c}. \tag{83}$$

Because $G_a(x) = \int_x^\infty z^{(p-2)/2} F(z)\, dz$ is a monotonically *decreasing* function of x and $x_{\min} < x_{\max}$, the bracket in Eq. (82) is negative which cancels out the negative from $(\gamma_{\min}^{1-p} - \gamma_{\max}^{1-p})$, as required for a physical absorption coefficient. This is the operand order used by `synchAlphaNu`:

```
double delta_Ga = evalGa_p(x_min, tables->Ga_spline, p)
- evalGa_p(x_max, tables->Ga_spline, p);
```

The implementation uses the single-power-law expression obtained by reducing the broken-power-law absorption coefficient (Kawashima et al., 2023, Raikou Eq. C4) to a common index $p_1 = p_2 = p$. This form is also straightforward to compare with standard nonthermal synchrotron-absorption derivations (e.g. Dexter et al., 2012).

The expression adopted in Eq. (82) has dimensions of inverse length in Gaussian cgs units. We therefore do not use RAIKOU Eq. C2 verbatim when establishing the normalization; instead, we use the dimensionally explicit limiting form of Eq. C4. Any apparent difference with Eq. C2 may involve a typographical or convention-related rearrangement of powers of ν_c, so comparisons must use the complete definitions of x, $F(x)$, and $G_a(x;p)$ rather than the displayed prefactor alone.

Broken power law (Kawashima et al., 2023, RAIKOU Eq. C4). The two power-law segments contribute independently:

$$\alpha_{\nu_f}^{(f)} = \frac{A_{\rm norm}^{(\rm br)}\, n_{e,\rm nth}\, e^2}{4\sqrt{3}\, m_e c\, \nu_c} \left[(p_1{+}2) \left(\frac{\nu_f}{\nu_c}\right)^{-(p_1+4)/2} \Delta G_{a,p_1} + (p_2{+}2)\gamma_{\rm br}^{p_2-p_1} \left(\frac{\nu_f}{\nu_c}\right)^{-(p_2+4)/2} \Delta G_{a,p_2} \right], \tag{84}$$

where

$$\Delta G_{a,p_1} = G_a(x_{\rm br};\, p_1) - G_a(x_{\rm min};\, p_1), \tag{85}$$

$$\Delta G_{a,p_2} = G_a(x_{\rm max};\, p_2) - G_a(x_{\rm br};\, p_2), \tag{86}$$

$$x_{\rm br} = \frac{\nu_f}{\gamma_{\rm br}^2\, \nu_c}. \tag{87}$$

As in the single power-law case, each ΔG subtracts the value of G_a at the *larger* x argument from its value at the *smaller* x argument (recalling $x_{\rm min} < x_{\rm br} < x_{\rm max}$), matching `synchAlphaNu`:

```
double delta_Ga_p1 = evalGa_p(x_br,  tables->Ga_spline_p1, p1)
- evalGa_p(x_max, tables->Ga_spline_p1, p1);

double delta_Ga_p2 = evalGa_p(x_min, tables->Ga_spline_p2, p2)
- evalGa_p(x_br,  tables->Ga_spline_p2, p2);
```

Key result. Both $x_{\rm min}$ and $x_{\rm br}$ (and, for high-B cells, even $x_{\rm max}$) can fall below `SYNCH_X_MIN` for realistic parameter choices. In these cases `synchAlphaNu` does not clamp G_a at the tabulated boundary; instead it relies on the analytic low-x continuation of $G_a(x;p)$ described in Section 4.6.4, Eq. (38), evaluated by `evalGa_p`. This continuation is what produces the physically correct soft SSA turnover in $\alpha_{\nu_f}^{(f)}$ at low ν_f; a naive clamped evaluator (such as `evalGa`, which exists in the code but is not used by `synchAlphaNu`) would instead flatten $\alpha_{\nu_f}^{(f)}$ artificially once any x argument reaches the table boundary. In the regime where both finite-electron integration boundaries satisfy $x < x_0$, the corrected continuation gives $\alpha_{\nu_f}^{(f)} \propto \nu_f^{-5/3}$, as derived in Eq. (40).

Key result. A natural pitfall is to reverse the operand order in ΔG_a (writing $G_a(x_{\rm max}) - G_a(x_{\rm min})$ instead of $G_a(x_{\rm min}) - G_a(x_{\rm max})$). Because G_a is monotonically decreasing in x, that ordering would evaluate to a negative absorption coefficient. Equations (82)–(84) give the code-consistent ordering used by `synchAlphaNu`.

Key result. $G_a(x;\, p)$ depends only on the dimensionless argument x and the power-law index p, not on B or ν_f individually. A single precomputed 1D table therefore serves all cells in the simulation, with the physical prefactors (which do depend on B, $n_{e,\mathrm{nth}}$, and ν_f) evaluated analytically at runtime. This is what makes the table approach computationally efficient.

This is implemented in `synchAlphaNu` (`mc_synchrotron.c`).

Relativistic frame correction Photon transport in MCRaT is performed in the lab frame, while $\alpha_{\nu_f}^{(f)}$ is defined in the fluid rest frame. Connecting the two requires the relativistic radiative transfer equation for a moving medium.

Following Abramowicz et al. (1991), the optical depth element along a ray in the lab frame for a fluid element moving with bulk Lorentz factor Γ is:

$$d\tau_{\nu_f}^{(a)} = \Gamma\,(1 - \beta\cos\theta)\,\alpha_{\nu_f}^{(f)}\left(n_{e,\mathrm{nth}}^{(\mathrm{comv})}\right)\,dl, \tag{88}$$

where dl is the lab-frame path length element, $\beta = v/c$ is the fluid speed, θ is the angle between the photon direction and the fluid velocity in the lab frame, and $\alpha_{\nu_f}^{(f)}$ is evaluated using the *comoving-frame* electron number density $n_{e,\mathrm{nth}}^{(\mathrm{comv})}$ as stored in `hydro_data->nonthermal_dens`.

To understand where the factor $\Gamma(1 - \beta\cos\theta)$ comes from physically: the $(1 - \beta\cos\theta)$ term is a Doppler factor that accounts for the fact that the photon sees a different frequency in the fluid frame than in the lab frame; the Γ factor accounts for the Lorentz contraction of path lengths between frames.

In MCRaT, the full factor $\Gamma(1-\beta\cos\theta)$ is assembled inside `calculateOpticalDepthSSA` in two steps:

1. **The $(1 - \beta\cos\theta)$ factor** is the *fluid factor* computed in `calculateOpticalDepth` from the dot product of the lab-frame fluid velocity and photon direction:

$$\mathrm{fluid_factor} = 1 - \beta\,\hat{\mathbf{v}}\cdot\hat{\mathbf{n}} = 1 - \beta\cos\theta, \tag{89}$$

   which is passed into `calculateOpticalDepthSSA` as a parameter.

2. **The Γ factor** is read directly from the local fluid Lorentz factor stored in the hydro data frame, `gamma = (hydro_data->gamma)[cell_idx]`, and multiplied onto the absorption coefficient explicitly inside `calculateOpticalDepthSSA`:

$$\texttt{ph->absorption_opacity} = \Gamma \times \mathrm{fluid_factor} \times \alpha_{\nu_f}^{(f)}\left(n_{e,\mathrm{nth}}^{(\mathrm{comv})}\right). \tag{90}$$

   This choice ? multiplying Γ onto the absorption *coefficient* rather than onto the comoving electron density ? follows the frame-transformation argument of Abramowicz et al. (1991) and is applied at exactly the same point in the code as the scattering-opacity frame correction, keeping all frame-correction arithmetic in one place.

Key result. Edge case: if the local fluid element is static ($\beta = 0$), the direction $\hat{\mathbf{v}}$ in Eq. (89) is undefined (a zero vector has no direction). `calculateOpticalDepth` handles this explicitly: rather than computing the dot product $\hat{\mathbf{v}} \cdot \hat{\mathbf{n}}$ from an undefined unit vector, the code sets `fluid_factor` $= 1$ directly whenever $\beta = 0$, which is the correct limit of Eq. (89) as $\beta \to 0$ regardless of $\hat{\mathbf{n}}$. Both the scattering opacity and the SSA absorption coefficient therefore reduce smoothly to their fluid-rest-frame values in a static medium.

The value stored in `ph->absorption_opacity` is therefore the *complete* Abramowicz et al. (1991)-corrected absorption coefficient. For cross-referencing convenience elsewhere in this document we denote this stored quantity explicitly as:

$$\texttt{ph->absorption_opacity} = \underbrace{\Gamma}_{\text{step 2}} \times \underbrace{\text{fluid_factor}}_{\text{step 1, Eq. (89)}} \times \alpha_{\nu_f}^{(f)}\left(n_{e,\text{nth}}^{(\text{comv})}\right), \tag{91}$$

which is algebraically identical to Eq. (90) above.

Key result. The lab-frame conversion of the non-thermal electron density,

$$n_{e,\text{nth}}^{(\text{lab})} = \Gamma\, n_{e,\text{nth}}^{(\text{comv})}, \tag{92}$$

is used by `calculateOpticalDepth` to form the *scattering* optical depth for non-thermal electrons (`nonthermal_n_dens_lab = nonthermal_dens[i] * gamma[i]` in `optical_depth.c`, cf. Eq. (114)). This is a *different* mechanism from SSA: for SSA, Γ is *not* used to boost the electron density into the lab frame before calling `synchAlphaNu`. Instead, `synchAlphaNu` is evaluated at the raw *comoving*-frame density $n_{e,\text{nth}}^{(\text{comv})}$, and Γ is multiplied directly onto the resulting absorption *coefficient* inside `calculateOpticalDepthSSA`, as shown in Eq. (91). The two mechanisms are algebraically distinct even though both ultimately introduce a single factor of Γ: one rescales a number density, the other rescales an opacity. Table 4 and the summary in Section 4.6.10 describe the SSA-specific mechanism only; Eq. (92) above documents the separate, scattering-specific mechanism for completeness.

Implementation in `calculateOpticalDepth` Whenever a photon enters a new cell or scatters (`recalc_properties`$= 1$), `calculateOpticalDepth` (`optical_depth.c`) recomputes all photon-transport coefficients. The fluid factor (89) is computed once and applied to both the scattering opacity and the SSA absorption coefficient within the same function call:

$$\text{fluid_factor} = 1 - \beta\, \frac{\mathbf{v}_{\text{fluid}} \cdot \mathbf{p}_{\text{ph}}}{|\mathbf{v}_{\text{fluid}}||\mathbf{p}_{\text{ph}}|}. \tag{93}$$

Implementation note. The code variable `fluid_factor` contains only

$$\texttt{fluid_factor} = 1 - \beta\cos\theta. \tag{94}$$

It does not include the bulk Lorentz factor Γ.

For scattering, Γ enters through the lab-frame electron density, for example

$$n_{e,\mathrm{nth}}^{(\mathrm{lab})} = \Gamma n_{e,\mathrm{nth}}^{(\mathrm{comv})}. \tag{95}$$

For synchrotron self-absorption, the coefficient is first evaluated at the comoving density and Γ is then multiplied directly onto the coefficient:

$$\texttt{ph->absorption_opacity} = \Gamma\, \texttt{fluid_factor}\, \alpha_{\nu_f}^{(f)}. \tag{96}$$

The SSA coefficient is then computed via:

```
calculateOpticalDepthSSA(ph, hydro_data, fluid_factor, fPtr);
```
(97)

The result is stored in `ph->absorption_opacity` (91) and held there until the photon moves to a new cell or scatters.

Key result. A stale comment in `optical_depth.c` itself, stated that `calculateOpticalDepthSSA` only acts on packets of type `SYNCH_PHOTON`, with `ph->absorption_opacity` set to zero for all other types. In the current implementation this type check is explicitly disabled:

```
void calculateOpticalDepthSSA(...)
{
 ph->absorption_opacity = 0.0;

 //this can operate on all photon types
 //if (ph->type != SYNCH_PHOTON)
 //    return;
 ...
}
```

SSA absorption is therefore now computed and applied to *every* photon type (`INJECTED_PHOTON`, `COMPTONIZED_PHOTON`, etc.) that passes through a cell with $B > 0$ and non-thermal electrons present, not only synchrotron-seeded packets. This has a direct consequence for Russian roulette (Section 4.6.9): because `INJECTED_PHOTON` and `COMPTONIZED_PHOTON` packets can now also accumulate continuous SSA weight loss, they are separately rouletted in addition to `SYNCH_PHOTON` (see Section 4.6.9).

Continuous weight attenuation (`applyabsorption`) During photon transport, `applyAbsorption` is called at each sub-step of length dl within `updatePhotonPosition`. Because the complete frame correction $\Gamma(1 - \beta\cos\theta)$ is already folded into `ph->absorption_opacity` (91), the weight update requires no additional frame arithmetic:

$$w_{\mathrm{new}} = w_{\mathrm{old}} \times \exp(-\,\texttt{ph->absorption_opacity} \times dl)\,. \tag{98}$$

Expanding `ph->absorption_opacity` using (91) recovers the full Abramowicz et al. (1991) expression:

$$w_{\rm new} = w_{\rm old} \times \exp\!\Big(-\,\Gamma\,(1-\beta\cos\theta)\;\alpha_{\nu_f}^{(f)}\Big(n_{e,\rm nth}^{(\rm comv)}\Big)\;dl\Big). \tag{99}$$

Two numerical guards are applied before the weight update, exactly as implemented in `applyAbsorption`:

```
double tau = ph->absorption_opacity * dl;

if (tau > 200.0 || ph->weight < FLT_MIN)
{
 ph->weight = DBL_MIN;
 ph->nearest_block_index = -1;
 ph->time_to_scatter     = FLT_MAX / C_LIGHT;
 ph->recalc_properties   = 1;
 return;
}
ph->weight *= exp(-tau);
```

- If $dl \le 0$ or `ph->absorption_opacity` ≤ 0, the function returns immediately (no-op); this guard is checked earlier in the function and is unchanged from the previous description.
- If $\tau \equiv$ `ph->absorption_opacity` $\times\, dl > 200$ *or* the packet's weight has already fallen below `FLT_MIN`, the packet is not multiplied by $\exp(-\tau)$ (which would risk floating-point underflow for large τ). Instead its weight is set to the sentinel value `DBL_MIN` ? a small but strictly positive value, chosen so the packet can still be distinguished from a true `NULL_PHOTON` and, if desired, still be written to output ? rather than being set to exactly zero.
- In the same branch, the packet is also marked as having left the active transport domain: `nearest_block_index` is set to -1, `time_to_scatter` is set to the sentinel FLT_MAX$/c$, and `recalc_properties` is set to 1. This mirrors the sentinel scheme used elsewhere in MCRaT for photons that leave the hydro grid (Section 4.6.6), ensuring the packet is skipped by `photonEvent` and `calcMeanFreePath` for the remainder of the current frame.

Key result. The implemented deep-absorption guard triggers at $\tau > 200$ (not at the naïvely large value one might expect from pure underflow avoidance), sets the weight to the sentinel `DBL_MIN` rather than to exactly zero, additionally triggers on an independent `weight` $<$ `FLT_MIN` condition, and invalidates the packet for further transport within the current frame (`nearest_block_index` $= -1$, `time_to_scatter` $=$ FLT_MAX$/c$, `recalc_properties` $= 1$), exactly as detailed above and in `applyAbsorption` (`mc_synchrotron.c`).

Known issue. Packets that trigger the deep-absorption guard are assigned the exact sentinel weight `DBL_MIN`. The Russian-roulette weight-collection pass includes only packets satisfying

```
ph->weight > DBL_MIN
```

and therefore excludes packets whose weight is exactly `DBL_MIN` when calculating the population median.

If at least one ordinary packet of the same photon type is present, the later roulette pass still sees the `DBL_MIN` packet and will almost certainly kill it. If all remaining packets of that type have weight exactly `DBL_MIN`, however, the collection pass finds no eligible weights and returns without removing them. Consequently, `DBL_MIN` packets are not unconditionally converted into `NULL_PHOTON` packets by the current implementation.

SSA absorption is applied *only* during transport ? never at the birth of a photon packet ? preserving the physical interpretation of the packet weight as the number of real photons at the moment of emission.

4.6.7 Hot Scattering Cross Sections

Why the Thomson cross section is not enough In the simplest treatment of Compton scattering, every electron presents the same target area to every photon regardless of their energies: the Thomson cross section $\sigma_T = 6.65 \times 10^{-25}\,\mathrm{cm}^2$. This is valid only when both the photon energy $h\nu$ and the electron kinetic energy $(\gamma - 1)m_e c^2$ are much smaller than the electron rest-mass energy $m_e c^2 = 511\,\mathrm{keV}$. In the hot, relativistic plasmas modelled by MCRaT neither condition is generally satisfied:

- **Hot thermal electrons** in a fluid with dimensionless temperature $\Theta = k_B T/(m_e c^2) \gtrsim 0.01$ have significant relativistic bulk motion. At sufficiently low photon energy in every relevant electron rest frame, the effective cross section approaches σ_T. Relativistic electron motion can Doppler-boost the photon into the Klein–Nishina regime in the electron rest frame, reducing the distribution-averaged cross section below the Thomson value.
- **Energetic photons** with $h\nu \gtrsim m_e c^2$ experience Klein–Nishina suppression of the cross section even for cold electrons.
- **Non-thermal power-law electrons** with $\gamma \gg 1$ interact with photons through the Klein–Nishina cross section in the electron rest frame, which differs substantially from σ_T for most photon–electron energy combinations of interest.

Key result. Using σ_T in a hot or relativistic plasma over- or under-estimates the scattering rate, biasing both the energy exchange and the escape of radiation. MCRaT uses a tabulated *hot cross section* that correctly accounts for the full Klein–Nishina energy dependence and the relativistic electron distribution.

Definition of the hot cross section The effective scattering cross section for a photon with comoving-frame energy $\epsilon = h\nu_f/(m_e c^2)$ propagating through a plasma with electron distribution $f(\gamma, \mu)$ is obtained by averaging the Klein–Nishina cross section over the

electron distribution and over the angle $\mu = \cos\theta_{e\gamma}$ between the photon and electron momenta (Dolence et al., 2009; Kawashima et al., 2023; Canfield et al., 1987):

$$\sigma(\epsilon,\, T) = \frac{1}{2}\int_{1}^{\gamma_{\max}} \int_{-1}^{1} f_{\mathrm{MJ}}(\gamma,\, \Theta)\, \sigma_{\mathrm{KN}}(\epsilon')\, (1-\mu\beta)\, d\mu\, d\gamma, \tag{100}$$

where:

- $f_{\mathrm{MJ}}(\gamma,\, \Theta)$ is the Maxwell–Jüttner distribution of thermal electrons at dimensionless temperature $\Theta = k_B T/(m_e c^2)$,
- $\beta = \sqrt{1-1/\gamma^2}$ is the electron speed,
- $\epsilon' = \epsilon\,\gamma(1-\mu\beta)$ is the photon energy in the electron rest frame,
- The total Klein–Nishina cross section used by the code is written in normalized form as

$$\frac{\sigma_{\mathrm{KN}}(\epsilon')}{\sigma_T} = \frac{3}{4}\left[\frac{2}{\epsilon'^2} + \left(\frac{1}{2\epsilon'} - \frac{1+\epsilon'}{\epsilon'^3}\right)\ln(1+2\epsilon') + \frac{1+\epsilon'}{(1+2\epsilon')^2}\right], \tag{101}$$

  where ϵ' is the photon energy in the electron rest frame in units of $m_e c^2$ (Dolence et al., 2009).

  Direct evaluation of Eq. (101) suffers from cancellation at very small ϵ'. The implementation therefore uses the low-energy approximation

$$\frac{\sigma_{\mathrm{KN}}}{\sigma_T} \simeq 1 - 2\epsilon' \tag{102}$$

  when

$$\epsilon' < \texttt{THOMSON_LIMIT_ENERGY_RATIO} = 10^{-3}. \tag{103}$$

  The same transition energy is used by the total-cross-section evaluator and the differential scattering sampler. And

- the factor $(1-\mu\beta)$ is the relative velocity of the photon and electron in the fluid frame.

The result (100) is normalised by σ_T so that the code stores σ/σ_T: a dimensionless correction factor that is multiplied by σ_T when computing optical depths.

For nonthermal electrons, MCRAT partitions the full electron Lorentz-factor range into `N_GAMMA` subgroups. The table entry for subgroup k is

$$\frac{\sigma_k(\epsilon)}{\sigma_T} = \frac{1}{2}\int_{\gamma_k^{\min}}^{\gamma_k^{\max}} d\gamma \int_{-1}^{1} d\mu\, f_{\mathrm{nth}}(\gamma)\frac{\sigma_{\mathrm{KN}}(\epsilon')}{\sigma_T}\,(1-\mu\beta_e)\,, \tag{104}$$

where $f_{\mathrm{nth}}(\gamma)$ is normalized over the full interval $[\gamma_{\min}, \gamma_{\max}]$. It is not renormalized independently within each subgroup.

Consequently, σ_k/σ_T already includes the fractional abundance of electrons that lies in subgroup k. At runtime it is multiplied by the total nonthermal electron density. The sum of the subgroup contributions therefore reconstructs the full nonthermal scattering opacity.

Table construction and interpolation The double integrals (100) and (104) are expensive to evaluate at runtime, so MCRaT precomputes them once on a grid and uses bilinear interpolation during the simulation.

Thermal table. The thermal cross section is tabulated on a 2D grid of (`N_PH_E`+1) × (`N_T`+1) points:

- Photon energy axis: $\log_{10}(\epsilon)$ uniformly spaced over [`LOG_PH_E_MIN`, `LOG_PH_E_MAX`].
- Temperature axis: $\log_{10}(\Theta)$ uniformly spaced over [`LOG_T_MIN`, `LOG_T_MAX`].

Each table entry is $\log_{10}(\sigma/\sigma_T)$. The double integral is evaluated using the GSL plain Monte Carlo integrator with 5×10^5 calls per point. The resulting table is stored to a file (`HOT_THERMAL_X_SECTION_FILE`) so it can be reused across simulation runs without recomputation.

Non-thermal table. When `NONTHERMAL_E_DIST` $\neq$ `OFF`, a second table of dimension (`N_PH_E`+1) × `N_GAMMA` is built, where `N_GAMMA` is the number of Lorentz-factor subgroups. Each entry $[i][k]$ is $\log_{10}(\sigma_k/\sigma_T)$ for photon energy grid point i and subgroup k, evaluated with 10^6 Monte Carlo calls. The gamma axis of the table is placed at the log-space midpoint of each subgroup's Lorentz-factor range. A sufficiently large value of `N_GAMMA` is required to ensure that the interpolation across subgroup boundaries is accurate; with too few subgroups the cross section is poorly sampled near the boundary between adjacent subgroups, producing artefacts in the scattered spectrum at the corresponding photon frequencies.

MPI distribution. Table construction is performed by MPI rank 0 only. The tables are then broadcast to all other ranks via `broadcastInterpolationData` before the simulation begins. This avoids redundant computation and ensures all ranks use identical tables.

Runtime interpolation. During the simulation, `interpolateThermalHotCrossSection` and `interpolateSubgroupNonThermalHotCrossSection` use the prebuilt GSL `gsl_spline2d` bilinear interpolation objects to evaluate σ/σ_T at the photon's comoving energy and the cell temperature in $O(1)$ time. Thread safety is ensured by the same per-thread `ThreadLocalAccel` pattern used for the synchrotron tables (Section 4.6.4).

4.6.8 Scattering Bias and Importance Sampling

The problem: rare scatterings in low-optical-depth regions In optically thin regions, a photon packet may travel many mean free paths before scattering. If the simulation is run for a fixed time interval $\Delta t_{\rm fps} = 1/\text{fps}$, most packets in such regions will never scatter, contributing nothing to the scattering statistics. Meanwhile, optically thick regions produce enormous numbers of scatterings that are individually unimportant statistically. This imbalance makes the unbiased estimator (1) very high-variance.

The solution is *importance sampling* of the scattering interaction: force photon packets to scatter more often than they naturally would, then correct for this artificial enhancement by adjusting the weight. This is the scattering bias scheme used in MCRaT, following Kawashima et al. (2023); Dolence et al. (2009).

Opacity versus optical depth The photon fields `scattering_opacity[k]`, `total_scattering_opac` and `absorption_opacity` store coefficients with dimensions of inverse length. They are opacities, not dimensionless optical depths.

We denote the unbiased scattering opacity of electron population k by

$$\kappa_k = n_{e,k}\sigma_k\,(1-\beta\cos\theta) \qquad [\mathrm{cm}^{-1}], \tag{105}$$

with any required bulk Lorentz factor already included through the lab-frame density. The optical depth accumulated over a path dl is

$$d\tau_k = \kappa_k\,dl. \tag{106}$$

The total biased scattering opacity is

$$\kappa_{\mathrm{tot}}^{(b)} = \sum_k b_k\kappa_k, \tag{107}$$

and the sampled propagation distance is

$$\lambda = \frac{-\ln u}{\kappa_{\mathrm{tot}}^{(b)}}, \qquad u\sim\mathcal{U}(0,1). \tag{108}$$

Scattering bias for thermal electrons The thermal scattering opacity is

$$\kappa_{\mathrm{th}} = n_{e,\mathrm{th}}^{(\mathrm{lab})}\sigma_{\mathrm{th}}(\epsilon,T)\,(1-\beta\cos\theta) \qquad [\mathrm{cm}^{-1}], \tag{109}$$

where the bulk Lorentz factor is already included in the lab-frame electron density. The optical depth accumulated across a path dl is

$$d\tau_{\mathrm{th}} = \kappa_{\mathrm{th}}\,dl. \tag{110}$$

Without biasing, the mean free path (MFP) to the next scattering is:

$$\lambda = \frac{-\ln u}{\tau_0/l}, \qquad u\sim\mathcal{U}(0,1), \tag{111}$$

which in a thin cell may greatly exceed $c\,\Delta t_{\mathrm{fps}}$, meaning the packet never scatters.

The thermal scattering bias parameter b_{th} is defined so that the effective MFP used for sampling is reduced by a factor b_{th}:

$$\lambda_{\mathrm{biased}} = \frac{\lambda}{b_{\mathrm{th}}}, \tag{112}$$

forcing scatterings to occur b_{th} times more frequently. The bias parameter is computed by `calculateThermalScatteringBias` (Kawashima et al., 2023, Raikou Eq. 37):

$$b_{\mathrm{th}} = \max\!\left(1,\ \frac{\alpha\,\Theta_{\mathrm{cell}}^2}{\bar{\Theta}^2\,\tau_{\max}}\right), \tag{113}$$

where:

- α = `SCATTERING_BIAS_SCALING` is a dimensionless tuning parameter set by the user,

- $\Theta_{\rm cell} = k_B T_{\rm cell}/(m_e c^2)$ is the dimensionless temperature of the current cell,
- $\bar{\Theta}$ is the simulation-averaged dimensionless temperature `average_dimless_theta`,
- $\tau_{\rm max} = \tau_0 \times c\,\Delta t_{\rm fps}$ is the maximum optical depth the photon could accumulate before the next hydro frame is loaded.

The $\max(1, \ldots)$ ensures that $b_{\rm th} \geq 1$: the bias can only increase the scattering rate, never decrease it below the physical rate. Hot cells (large $\Theta_{\rm cell}$) and optically thin regions (small $\tau_{\rm max}$) receive the largest bias.

Key result. The bias (113) is proportional to $\Theta_{\rm cell}^2/(\bar{\Theta}^2\,\tau_{\rm max})$. Hot, optically thin cells are biased strongly (many forced scatterings); cool, optically thick cells approach the natural rate ($b_{\rm th} \to 1$).

Scattering bias for non-thermal electron subgroups When non-thermal electrons are present (`NONTHERMAL_E_DIST` $\neq$ `OFF`), the non-thermal electron population is divided into `N_GAMMA` Lorentz-factor subgroups $k = 1, \ldots,$ `N_GAMMA`. For nonthermal subgroup k, the corresponding opacity is

$$\kappa_k = n_{e,\rm nth}^{(\rm lab)} \sigma_k(\epsilon)\,(1 - \beta\cos\theta)\,. \tag{114}$$

where $\sigma_k(\epsilon)$ is the non-thermal cross section for subgroup k from the precomputed table (104).

The goal of the per-subgroup bias scheme is to ensure that every subgroup ? including rare, high-energy subgroups with very small τ_k ? is sampled with comparable frequency in the simulation, while the scattered photon weights are corrected to preserve the physical scattering rates. This is achieved by assigning each subgroup a bias b_k such that all subgroups contribute equally to the total biased optical depth.

Following Kawashima et al. (2023, RAIKOU Eq. 48), the bias assigned to nonthermal subgroup k is

$$b_k = b_{\rm ref}\frac{\kappa_{\rm ref}}{\kappa_k}, \tag{115}$$

where κ_k is the subgroup scattering opacity and the reference population depends on whether thermal electrons are present.

Known issue. The current packet-splitting implementation requires

$$0 \leq w_{\rm scatt} \leq w \tag{116}$$

so that

$$w_{\rm unscatt} = w - w_{\rm scatt} \tag{117}$$

remains nonnegative. With

$$w_{\rm scatt} = w\frac{1 - \exp(-\kappa_k dl)}{1 - \exp(-b_k \kappa_k dl)}, \tag{118}$$

a value $b_k < 1$ can produce $w_{\rm scatt} > w$. The current `photonEvent` implementation detects a negative unscattered weight and terminates the simulation.

Thus the raw RAIKOU subgroup-bias ratio is the intended bias definition, but its compatibility with MCRAT's current event-driven packet-splitting implementation requires either $b_k \geq 1$ in all encountered states or a revised weight-correction algorithm that remains valid for $b_k < 1$. Users should monitor the minimum subgroup bias during validation runs.

Mixed thermal and non-thermal case. When both thermal and non-thermal electrons are present, the reference quantities are those of the thermal population:

$$b_{\mathrm{ref}} = b_{\mathrm{th}}, \qquad \tau_{\mathrm{ref}} = \tau_0. \tag{119}$$

This is exactly the RAIKOU prescription (Kawashima et al., 2023, RAIKOU Eq. 48): each non-thermal subgroup is biased so that its contribution $b_k\tau_k$ to the total biased optical depth equals the thermal contribution $b_{\mathrm{th}}\tau_0$. The result is that the thermal population and every non-thermal subgroup are each selected with equal probability $1/(1+\texttt{N_GAMMA})$ during a scattering event.

Purely non-thermal case. When no thermal electrons are present, there is no natural reference population. In this case the reference optical depth is set to the *equal share* of the total non-thermal optical depth:

$$\tau_{\mathrm{ref}} = \frac{\tau_{\mathrm{nth,total}}}{\texttt{N_GAMMA}}, \qquad \tau_{\mathrm{nth,total}} = \sum_{k=1}^{\texttt{N_GAMMA}} \tau_k, \tag{120}$$

and the reference bias is computed from $\tau_{\mathrm{nth,total}}$:

$$b_{\mathrm{ref}} = \max\left(1,\ \frac{\alpha}{\tau_{\mathrm{nth,total}} \times c\,\Delta t_{\mathrm{fps}}}\right). \tag{121}$$

Substituting (120) into (115) gives:

$$b_k = \max\left(1,\ b_{\mathrm{ref}}\,\frac{\tau_{\mathrm{nth,total}}}{\texttt{N_GAMMA}\,\tau_k}\right), \tag{122}$$

so that $b_k\tau_k = b_{\mathrm{ref}}\,\tau_{\mathrm{nth,total}}/\texttt{N_GAMMA}$ for every subgroup (where the floor is not active), and every subgroup is again selected with equal probability $1/\texttt{N_GAMMA}$.

When the raw equalising bias ratios are used and all denominators are nonzero, the total biased scattering opacity is

$$\kappa_{\mathrm{tot}}^{(b)} = \begin{cases} (1+\texttt{N_GAMMA})\,b_{\mathrm{th}}\kappa_{\mathrm{th}}, & \text{thermal and nonthermal electrons,} \\ b_{\mathrm{ref}}\kappa_{\mathrm{nth,total}}, & \text{purely nonthermal electrons,} \end{cases} \tag{123}$$

where

$$\kappa_{\mathrm{nth,total}} = \sum_{k=1}^{\texttt{N_GAMMA}} \kappa_k. \tag{124}$$

The factor $1+\texttt{N_GAMMA}$ applies only in the mixed case, where the thermal population and all nonthermal subgroups are active. In the purely nonthermal case, the thermal slot is zero and only N_GAMMA populations contribute.

Key result. The per-subgroup bias (115) equalises the selection probability across all electron populations, boosting the sampling of rare high-energy subgroups. The weight correction at each scattering event (Section 4.6.8) then ensures that the statistically enhanced scatterings carry correctly reduced weights, so that the simulation remains unbiased.

Photon weight modification due to biased scattering Because biased sampling forces scatterings to occur more frequently than they physically should, the photon packet weight must be adjusted at each forced scattering event to preserve the unbiasedness of the estimator (1). The key insight is that a single packet travelling through the biased medium must be split into two components at the moment of scattering:

1. A **scattered component** that has physically interacted with an electron and carries the weight corresponding to the true probability of scattering over the path traversed.
2. An **unscattered component** that has not physically interacted and continues propagating with the remaining weight, representing the fraction of real photons that would have passed through unaffected.

The true probability of scattering off subgroup j over a path of optical depth $\tau_j \cdot dl$ is:

$$P_{\text{scatt,j}} = 1 - e^{-\tau_j \cdot dl}, \tag{125}$$

while the forced (biased) scattering probability for subgroup j over the same path is:

$$P_{\text{biased,j}} = 1 - e^{-b_j\, \tau_j \cdot dl}. \tag{126}$$

The scattered packet should carry the fraction of the total weight that corresponds to the *true* scattering probability of subgroup j, corrected for the *biased* sampling rate of that same subgroup. This is implemented in `scatteredPhotonWeight` (`mcrat_scattering.c`):

$$w_{\text{scatt}} = w \times \frac{1 - e^{-\tau_j \cdot dl}}{1 - e^{-b_j\, \tau_j \cdot dl}}, \tag{127}$$

where $\tau_j \cdot dl = \tau_j \times c\,\Delta t_{\text{scatt}}$ is the true optical depth of the scattering subgroup j over the time step, and $b_j\, \tau_j \cdot dl$ is the biased optical depth of that same subgroup.

Key result. The denominator of (127) uses the biased optical depth of the *specific subgroup that scattered*, $b_j\, \tau_j \cdot dl$, not the total biased optical depth $\tau_{\text{total}} \cdot dl$. This is the critical distinction from a naive implementation: using τ_{total} in the denominator would incorrectly mix the contributions of different subgroups and produce systematically wrong weights. Each subgroup's scattering rate was artificially enhanced by its own factor b_j, so each must be corrected by its own b_j independently.

In the optically thin limit ($\tau_j \cdot dl \ll 1$), the exponentials in (127) expand as $1 - e^{-x} \approx x$, and the expression simplifies to:

$$w_{\text{scatt}} \approx \frac{w}{b_j}, \tag{128}$$

which is the form used in the Dolence et al. (2009) grmonty code. The full expression (127) is used in MCRAT since it remains valid in both optically thin and thick regimes.

The unscattered component retains the remaining weight:

$$w_{\text{unscatt}} = w - w_{\text{scatt}}. \tag{129}$$

In practice, the original photon struct is first updated to carry w_{unscatt} and a copy is made with w_{scatt}. The copy is then added to the photon list as a new packet and its 4-momentum is updated by the Compton scattering kinematics. The original packet continues propagating unscattered with the reduced weight, representing the fraction of real photons that did not interact.

Key result. The split conserves the parent packet weight algebraically:

$$w_{\text{scatt}} + w_{\text{unscatt}} = w. \tag{130}$$

When $b_j \geq 1$, the scattered weight also satisfies $0 \leq w_{\text{scatt}} \leq w$, so both child weights are nonnegative.

The intended statistical correction follows the RAIKOU importance-sampling prescription. Exact unbiasedness of the complete implementation additionally requires consistency among the sampled total biased opacity, subgroup selection probability, and subgroup-specific weight correction. This should be verified with convergence tests, particularly for mixed populations and for any state in which b_j approaches or falls below unity.

Note that when the scattering bias equals 1 for a given subgroup (i.e. the MFP has not been artificially reduced for that subgroup, as occurs in optically thick regions where the $\max(1, \ldots)$ floor is active), the packet is scattered without splitting: the weight is unchanged and the unscattered component has zero weight. This is checked explicitly in the code to avoid unnecessary packet proliferation.

Electron subgroup selection When a scattering event occurs, `generateSingleElectron` (`electron.c`) must determine which electron population ? thermal or one of the `N_GAMMA` non-thermal subgroups ? interacts with the photon. The selection probability for each population j is proportional to its contribution to the total biased optical depth:

$$P(j) = \frac{b_j\,\tau_j}{\tau_{\text{total}}}, \tag{131}$$

where $\tau_{\text{total}} = \sum_j b_j\,\tau_j$ (123). By construction of the per-subgroup biases, $b_j\,\tau_j$ is the same constant for every population (where the $\max(1, \ldots)$ floor is not active), so (131) gives equal selection probability across all populations.

The selection is implemented via a forward CDF scan: a uniform random variate $u \sim \mathcal{U}(0, 1)$ is drawn and the cumulative sum $\sum_{j'=0}^{j} b_{j'}\tau_{j'}/\tau_{\text{total}}$ is accumulated until it exceeds u. The subgroup j at which the threshold is crossed is the selected population. The numerator of the selection sum, $\sum_{j'=0}^{j} b_{j'}\tau_{j'}$, is accumulated locally inside `generateSingleElectron` from the per-subgroup opacities and biases stored on the photon struct. The denominator, however, is *not* recomputed locally: it is read directly from the field `ph->total_scattering_opacity`,

```
double total_tau = (ph->total_scattering_opacity);
double cumulative = 0.0;
for (int i = 0; i < N_GAMMA + 1; i++)
{
 cumulative += (ph->scattering_bias)[i] * (ph->scattering_opacity)[i];
 if ((result == -1) && (random_num <= cumulative / total_tau))
 result = i;
}
```

which was itself populated earlier by `calculateOpticalDepth` via Eq. (132).

Key result. A subtle but important implementation detail: only the numerator of the subgroup-selection CDF is computed locally; the denominator is read from `ph->total_scattering_opacity`, which must therefore have been correctly (re-)computed by a prior call to `calculateOpticalDepth` for the selection probabilities (131) to be consistent. Any code path that mutates `ph->scattering_opacity` or `ph->scattering_bias` without also updating `ph->total_scattering_opacity` risks a numerator/ denominator mismatch in this function.

Once the subgroup is selected, an individual electron is sampled from the appropriate distribution: the Maxwell–Jüttner distribution for thermal electrons via `singleThermalElectron`, or the power-law distribution restricted to the Lorentz-factor range of the selected subgroup via `singleNonThermalElectron` and `calculateGammaSubgroup`.

The total optical depth and photon storage The total effective optical depth used to sample the next scattering event is the sum over all components, each weighted by its bias:

$$\tau_{\text{total}} = \sum_{j=0}^{\texttt{N_GAMMA}} b_j \, \tau_j, \tag{132}$$

where $j = 0$ is the thermal component (zero if no thermal electrons are present) and $j = 1, \ldots, \texttt{N_GAMMA}$ are the non-thermal subgroups. This biased total optical depth is stored in `ph->total_scattering_opacity` and used to compute the MFP via:

$$\text{mfp} = \frac{-\ln u}{\tau_{\text{total}}}, \qquad u \sim \mathcal{U}(0,1). \tag{133}$$

Implementation note. Two indexing conventions coexist and must not be confused. The *hot cross-section table* (104) has exactly `N_GAMMA` nonthermal columns, indexed $k = 1, \ldots, \texttt{N_GAMMA}$, with no thermal entry. The *photon-struct opacity and bias arrays* (`ph->scattering_opacity`, `ph->scattering_bias`) have $1 + \texttt{N_GAMMA}$ slots, indexed $j = 0, \ldots, \texttt{N_GAMMA}$, where slot $j = 0$ is the thermal population (zero when no thermal electrons are present) and $j = 1, \ldots, \texttt{N_GAMMA}$ map onto nonthermal subgroup $k = j$. Hence the selection and total-τ loops run to $\texttt{N_GAMMA} + 1$ entries, as in the quoted `generateSingleElectron` code.

The individual optical depths `ph->scattering_opacity[j]` and bias parameters `ph->scattering_bias[j]` are stored on each photon struct so that when a scattering event occurs, the correct subgroup can be selected via (131) and the weight correction (127) can be applied using the correct per-subgroup quantities.

Summary of the optical depth calculation The full optical depth calculation in `calculateOpticalDepth` (`optical_depth.c`) proceeds as follows each time `recalc_properties`= 1:

1. Compute `fluid_factor` $= 1 - \beta\cos\theta$ from the lab-frame photon and fluid directions. The factor Γ is not included in this variable: it enters through the lab-frame electron density for scattering and is multiplied separately in `calculateOpticalDepthSSA` for absorption.

2. Look up the normalised cross section σ/σ_T from the precomputed table via `getCrossSection`, which calls `getThermalCrossSection` and (if applicable) `getNonThermalCrossSection`.

3. *If thermal electrons are present:* compute the thermal optical depth $\tau_0 = n_{e,\mathrm{th}}^{(\mathrm{lab})}\,\sigma_T\,(\sigma/\sigma_T)$ fluid_factor compute the thermal bias $b_0 = b_{\mathrm{th}}$ via (113) using τ_0 as $\tau_{\max}$; store in `ph->scattering_opacity[0]` and `ph->scattering_bias[0]`. Set $\tau_{\mathrm{ref}} = \tau_0$ and $b_{\mathrm{ref}} = b_{\mathrm{th}}$ for use in step 4.

4. *If no thermal electrons are present:* compute $\tau_{\mathrm{nth,total}} = \sum_k \tau_k$ (120); compute b_{ref} from $\tau_{\mathrm{nth,total}}$ via (121); set $\tau_{\mathrm{ref}} = \tau_{\mathrm{nth,total}}/$`N_GAMMA` (120). Set `ph->scattering_opacity[0]` $= 0$ and `ph->scattering_bias[0]` $= 0$.

5. For each non-thermal subgroup $k = 1, \ldots,$ `N_GAMMA`, compute τ_k (114) and bias b_k (115) using b_{ref} and τ_{ref} from the appropriate branch above; store in `ph->scattering_opacity[k]` and `ph->scattering_bias[k]`.

6. Compute τ_{total} (132) and store in `ph->total_scattering_opacity`.

7. Compute the SSA absorption coefficient and store in `ph->absorption_opacity` (91) (Section 4.6.6).

All seven steps are triggered by a single flag `recalc_properties`= 1, which is set whenever the photon enters a new cell or undergoes a scattering event. After a biased scattering, the scattered copy of the packet has `recalc_properties`= 1 set so that its optical depths and bias parameters are freshly computed in the new cell it finds itself in after the scattering event.

4.6.9 Russian Roulette for Low-Weight Packets

The problem: exponentially small weights As a `SYNCH_PHOTON` packet propagates through an optically thick region, the continuous absorption in `applyAbsorption` reduces its weight exponentially:

$$w(l) = w_0 \exp\!\left(-\int_0^l \Gamma(1-\beta\cos\theta)\,\alpha_{\nu_f}^{(f)}\,dl'\right). \tag{134}$$

After traversing sufficient optical depth the weight may fall many orders of magnitude below its birth value w_0. Such a packet contributes negligibly to any observable yet still occupies a slot in `photon_list` and is processed at every scattering and transport step ? a waste of computational resources.

Russian roulette: theory Russian roulette is the standard Monte Carlo technique for culling low-weight particles while preserving the statistical unbiasedness of the simulation. The essential constraint is:

> **Key result.** The expected total weight of the photon population must be conserved by the roulette procedure: $\mathbb{E}[w_{\text{after}}] = w_{\text{before}}$.

Given a weight threshold w_{thresh}, for any packet with $w < w_{\text{thresh}}$ we define the survival probability:

$$P_{\text{surv}} = \frac{w}{w_{\text{thresh}}} \in (0,\, 1). \tag{135}$$

A uniform random variate $u \sim \mathcal{U}(0, 1)$ is drawn:

- If $u < P_{\text{surv}}$: the packet **survives** and its weight is boosted to w_{thresh}.
- If $u \geq P_{\text{surv}}$: the packet is **killed** and removed from the simulation.

The expected weight after the procedure is:

$$\mathbb{E}[w_{\text{after}}] = P_{\text{surv}} \cdot w_{\text{thresh}} + (1 - P_{\text{surv}}) \cdot 0 = \frac{w}{w_{\text{thresh}}} \cdot w_{\text{thresh}} = w. \quad \checkmark \tag{136}$$

This is exactly unbiased for any choice of w_{thresh}. All surviving packets emerge with weight exactly w_{thresh}, producing a common weight among packets that survive that roulette pass. The procedure trades a reduced packet count for additional sampling variance while preserving the expected packet weight.

Adaptive threshold via the median weight Rather than using a fixed threshold, MCRAT computes w_{thresh} adaptively from the current population of live `SYNCH_PHOTON` packets. The *median* weight w_{median} of all live `SYNCH_PHOTON` packets is computed, and the threshold is set as:

$$w_{\text{thresh}} = \varepsilon_{\text{RR}} \times w_{\text{median}}, \tag{137}$$

where ε_{RR} = `RR_WEIGHT_FRACTION` is a small dimensionless parameter set at compile time (typical value 10^{-2}).

The median is chosen over the mean for two reasons:

1. It is **robust to outliers**: rebinning events (see Section 4.6.9) produce a small number of high-weight packets that would inflate the mean but do not shift the median.
2. It is **self-adapting to the absorption history**: in strongly absorbed regions the median weight is already depressed, so the threshold is correspondingly lower and fewer packets are culled than in weakly absorbed regions.

The threshold w_{thresh} is computed *once* from a snapshot of the current population before any packet is modified, so it is a fixed constant for the entire roulette pass. This is essential for unbiasedness: if w_{thresh} were updated during the pass, the expected weight conservation (136) would no longer hold.

Relationship to rebinning The `photon_list` structure has a fixed capacity. When the number of live packets approaches a user-set maximum, the existing rebinning mechanism merges groups of packets into fewer representative packets by computing weighted averages of their momenta and positions, with the merged packet carrying the sum of the original weights. Rebinning is a many-to-one operation that controls the list size but introduces discretisation error in the phase-space distribution.

Russian roulette is complementary: it is a one-to-zero operation that removes individual low-weight packets, freeing slots for future injections without any reallocation. The two mechanisms operate on opposite ends of the weight distribution:

	Russian roulette	**Rebinning**
Targets	Low-weight packets	All active packets
Operation	Kill or boost individually	Merge groups
Statistical bias	None (exactly unbiased)	Small (phase-space error)
Triggered by	$w < w_{\text{thresh}}$	List near capacity

Russian roulette is applied *before* the rebinning check at the same checkpoint (every 1000 scatterings). By continuously removing low-weight packets, roulette reduces the pressure on the list capacity, meaning the rebinning trigger is reached less frequently or not at all in strongly absorbed regions.

Implementation (`applyRussianRoulette`) The per-population roulette logic (median computation, threshold, and kill/survive decision) is implemented in `applyRussianRouletteByType`, located in `mclib.c` alongside the `global_thread_rng` pool it uses for OpenMP-parallel random number generation:

```
int applyRussianRouletteByType(struct photonList *photon_list,
double             epsilon_rr,
char               photon_type,
FILE              *fPtr);
```

This function implements exactly the three-step procedure described below (weight collection, median/threshold computation, and the roulette pass) for a single, caller-specified `photon_type`, and returns the number of packets of that type killed.

The wrapper function actually invoked from the main transport loop, `applyRussianRoulette`, calls `applyRussianRouletteByType` *three times*, once for each of the photon types whose weight can be driven to very small values by continuous SSA absorption or by scattering into low-weight populations:

```
int applyRussianRoulette(struct photonList *photon_list,
double             epsilon_rr,
FILE              *fPtr)
{
 #if SYNCHROTRON_SWITCH == ON
 int n_killed=0;
 n_killed = applyRussianRouletteByType(photon_list, epsilon_rr,
```

```
 SYNCH_PHOTON, fPtr);
 n_killed = applyRussianRouletteByType(photon_list, epsilon_rr,
 INJECTED_PHOTON, fPtr);
 n_killed = applyRussianRouletteByType(photon_list,
 1e-8*epsilon_rr,
 COMPTONIZED_PHOTON, fPtr);
 return n_killed;
 #endif
}
```

- `SYNCH_PHOTON` packets are rouletted with the user-supplied threshold fraction $\varepsilon_{\mathrm{RR}} =$ `RR_WEIGHT_FRACTION`, as described throughout this section.
- `INJECTED_PHOTON` packets are also rouletted with $\varepsilon_{\mathrm{RR}}$: these can acquire very small weights after Compton scattering off the non-thermal power-law electron population (Section 4.6.8), independent of SSA.
- `COMPTONIZED_PHOTON` packets are scattered child packets created during biased Compton scattering. This type includes SSC photons but is not limited to synchrotron-seeded packets. These packets are rouletted with a threshold fraction reduced by an additional factor of 10^{-8} relative to $\varepsilon_{\mathrm{RR}}$. This deliberately more permissive threshold prevents the roulette procedure from prematurely culling the high-energy SSC spectral tail, which is astrophysically important but statistically underpopulated; the specific factor was tuned against the broken-power-law SSC test case described in Kawashima et al. (2023, Fig. 11).

Key result. `applyRussianRoulette` is a thin wrapper around three separate calls to `applyRussianRouletteByType` (for `SYNCH_PHOTON`, `INJECTED_PHOTON`, and `COMPTONIZED_PHOTON`), and the local variable `n_killed` is *overwritten* rather than accumulated on each call. The function's return value therefore reflects only the kill count from the *last* call, i.e. the `COMPTONIZED_PHOTON` pass ? which is also the value used by the caller to decide whether SSC-photon rebinning is required ? not a cumulative kill count across all three photon types.

With this clarification, the remainder of this subsection describes the internal three-step procedure of `applyRussianRouletteByType` for a single photon type and a single threshold $\varepsilon_{\mathrm{RR}}$.

Step 1 (parallel): collect weights. Each OpenMP thread fills its own local weight sub-array by scanning its assigned portion of `photon_list`. After the parallel region the sub-arrays are merged serially into a single flat array. Using per-thread sub-arrays avoids any shared-write contention on a single array.

Step 2 (serial): compute median and threshold. The merged weight array is sorted in ascending order using `gsl_sort`. The median is:

$$w_{\mathrm{median}} = \begin{cases} \mathrm{weights}[n/2] & n \text{ odd}, \\ \frac{1}{2}\big(\mathrm{weights}[n/2-1] + \mathrm{weights}[n/2]\big) & n \text{ even}, \end{cases} \tag{138}$$

where n is the number of live `SYNCH_PHOTON` packets. The threshold is then $w_{\text{thresh}} = \varepsilon_{\text{RR}} \times w_{\text{median}}$ (137).

Step 3 (parallel): roulette pass. Each thread processes its assigned portion of the photon list. For each `SYNCH_PHOTON` packet with $w < w_{\text{thresh}}$, a uniform variate is drawn from the thread-local RNG `global_thread_rng[thread_id]` and the kill/survive decision (135) is applied. Surviving packets have their weight boosted to w_{thresh}. Killed packets are removed via `setNullPhoton`, which frees the slot for future injections. The photon list counters (`num_photons`, `num_null_photons`) are updated inside a `#pragma omp critical` section to prevent race conditions from concurrent kills by different threads.

4.6.10 Summary of the Call Chain

Tables 2 and 4 summarise every function involved in synchrotron emission and absorption, Table 5 summarises the scattering bias and subgroup selection functions, and Table 6 summarises the Russian roulette procedure.

Implementation properties and current limitations

1. The physical photon production rate Λ_i (48) is computed exactly via K_{phys} (46), with no free proportionality constants.

2. The full relativistic frame correction $\Gamma(1 - \beta\cos\theta)$ (Abramowicz et al., 1991) is assembled consistently for both scattering and SSA absorption within `calculateOpticalDepth`, but via two distinct mechanisms: for *scattering*, the Γ factor is absorbed into the lab-frame electron density (92) before the cross-section lookup; for *SSA*, Γ is instead multiplied directly onto the absorption coefficient $\alpha_{\nu_f}^{(f)}$, which is itself evaluated at the comoving-frame density (91). In both cases $(1 - \beta\cos\theta)$ is applied via the shared `fluid_factor` (89), keeping the Doppler-factor arithmetic in one place even though the two processes distribute the Γ factor differently.

3. The per-subgroup bias scheme is *designed* to assign each electron population a bias b_j such that all populations contribute equally to τ_{total} (123), giving equal subgroup selection probability. In the mixed thermal and non-thermal case the thermal optical depth serves as the reference anchor (119); in the purely non-thermal case the equal-share optical depth $\tau_{\text{nth,total}}$/`N_GAMMA` serves as the reference (120). The $\max(1, \ldots)$ floor in (115) is *intended* to prevent any subgroup bias from falling below unity, which would produce unphysical negative unscattered weights; however, as detailed in the erratum in Section 4.6.8, this floor is currently disabled in `calculateNonthermalScatteringBias`, so this guarantee does not hold for the default runtime behaviour as shipped.

4. The scattered photon weight is corrected using only the per-subgroup biased optical depth $b_j\,\tau_j\,dl$ (127), not the total biased optical depth. This ensures that the weight correction is precisely matched to the artificial enhancement applied to each subgroup independently, preserving the unbiasedness of the estimator (1) across all electron populations simultaneously.

5. The Russian roulette procedure is exactly unbiased (136) and uses an adaptive threshold (137) that self-calibrates to the absorption history of the live photon

population. The wrapper `applyRussianRoulette` applies this procedure independently to three photon types (`SYNCH_PHOTON`, `INJECTED_PHOTON`, and `COMPTONIZED_PHOTON`, the last with a $10^{-8}\times$ reduced threshold to protect the SSC spectral tail; Section 4.6.9), and is invoked at two points per hydro frame: every 1000 scatterings, and at the end of eligible intermediate hydro frames. The end-of-frame pass is skipped for the initial scattering frame of a packet batch and for the final hydro frame. SSA absorption acts only during transport, never at photon birth, preserving the physical interpretation of the packet weight as the number of real photons at the moment of emission.

5 User-Defined Injection and Outflows

MCRaT provides two hooks that allow a user to override the default behaviour of the code without modifying the core radiation-transfer routines. The first, contained in `custom_spectrum.c`, lets the user inject photons with an arbitrary spectrum, 4-momentum, and polarization state instead of the built-in Wien or Blackbody spectra. The second, contained in `custom_outflow.c`, lets the user impose an arbitrary analytic outflow onto the fluid quantities of each hydro frame, in the same spirit as the spherical and cylindrical test outflows described in Section 3.2. Both capabilities are production-ready and are enabled through the switches described below.

5.1 Custom Photon Injection (`custom_spectrum.c`)

The `custom_spectrum.c` file provides the hook that MCRaT uses when the user wishes to inject photons drawn from a spectrum other than the built-in Wien or Blackbody spectra. It contains two functions: `custom_spectrum` and `custom_photon_sampler`.

5.1.1 How MCRaT Uses These Functions

During photon injection, MCRaT's `photonInjection` routine (in `mclib.c`) selects which spectrum to sample for each photon based on the spectrum type read from the `[MCRaT Photon Block]` of the `mc.par` file (see Section 2.4). When the requested spectrum is neither Wien nor Blackbody, MCRaT falls through to the custom-spectrum branch and calls

```
   initialized_photon = custom_photon_sampler(hydro_data, i, rand,
fPtr);
```

for the ith hydro element that photons are being injected into. The returned `photon` structure is then handed to the internal `saveUserDefinePhoton` routine, which performs all of the frame transformations, positioning, and bookkeeping that the user does *not* need to supply. The number of photons to inject into each hydro element, and their positions, are still determined by MCRaT exactly as described in Section 4.2; the custom sampler only governs the energy, direction, and polarization of each injected photon.

5.1.2 The `custom_photon_sampler` Function

The function has the signature

```
struct photon custom_photon_sampler(struct hydro_dataframe *hydro_data,
int hydro_index, gsl_rng *rand, FILE *fPtr);
```

and its arguments are:

- `hydro_data` — a pointer to the `hydro_dataframe` structure for the current frame, giving the user access to all fluid quantities (temperature, density, velocity, pressure, magnetic field, coordinates, etc.) of every hydro element.
- `hydro_index` — the index of the hydro element that the photon is being injected into. The user accesses a fluid quantity for this element as, for example, `(hydro_data->temp)[hydro_ind`
- `rand` — the GSL random number generator that the user should use for any Monte Carlo sampling, so that the sampling is consistent with the rest of MCRaT's (thread-safe) random number generation.
- `fPtr` — the log file pointer for the calling MPI process, which the user may write diagnostics to.

The function must return a `struct photon`. The user should always begin by calling the provided helper

```
struct photon custom_photon = createPhoton();
```

which returns a `photon` structure with every field initialized to `NaN` (with the exception of a few internal bookkeeping fields). Initializing every physical field to `NaN` is deliberate: it allows MCRaT to detect precisely which fields the user has overwritten and to fill in the remaining fields appropriately. The user then overwrites only the fields they wish to control, and returns the structure.

5.1.3 What the User May Set, and the Contract with MCRaT

The user may set *any one* of the following, and MCRaT will consistently compute everything else:

- **Energy only.** Setting only the comoving photon energy, `custom_photon.comv_p0`, (or the lab-frame energy, `custom_photon.p0`) fixes the photon energy while allowing MCRaT to assign the angular direction of the 4-momentum using its default isotropic-in-the-comoving-frame prescription (Section 4.2). This is the most common use case and is what the example implementation does.
- **Full 4-momentum.** Setting either the full comoving 4-momentum (`comv_p0`, `comv_p1`, `comv_p2`, `comv_p3`) or the full lab-frame 4-momentum (`p0`, `p1`, `p2`, `p3`) fixes both the energy and direction of the photon.
- **Polarization.** Setting the Stokes parameters (`s0`, `s1`, `s2`, `s3`) fixes the initial polarization state of the photon.

MCRaT honours the following contract when processing the returned photon:

- If the user specifies the 4-momentum in *either* the lab frame or the comoving (fluid) frame, MCRaT automatically computes the 4-momentum in the other frame by performing the appropriate Lorentz boost using the velocity of the fluid element indexed by `hydro_index`. The user therefore only needs to supply the 4-momentum in whichever frame is most convenient.

- The 4-momentum must use the normalizations and units expected by MCRaT, namely components of `p0`, `p1`, `p2`, and `p3` in units of erg/c, consistent with the saved quantities described in Section 3.2.

- For the Stokes parameters, MCRaT expects the normalized convention described in Section 4.4, with `s0` $= I/I = 1$ and `s1` $= q = Q/I$, `s2` $= u = U/I$, `s3` $= v = V/I$. MCRaT additionally validates that the supplied Stokes vector is physical by requiring

$$\texttt{s0}^2 \geq \texttt{s1}^2 + \texttt{s2}^2 + \texttt{s3}^2.$$

  If the user does not set the Stokes parameters, the photon is injected unpolarized, with $s = (1, 0, 0, 0)$, exactly as for the built-in spectra.

5.1.4 The `custom_spectrum` Helper Function

The `custom_spectrum.c` file also provides a convenience function

```
double custom_spectrum(double frequency);
```

that evaluates the *shape* of a user-defined spectrum at a given frequency (in Hz). The example implementation provided in the file returns a Gaussian line profile centred at 6.4 keV, i.e.

$$f(\nu) \propto \exp\left(-\frac{(\nu - \nu_0)^2}{2\sigma^2}\right),$$

with the profile set to zero outside $\pm 6\sigma$ of the line centre ν_0. The function returns the *unnormalized* spectral shape (i.e. it omits the $1/(\sigma\sqrt{2\pi})$ prefactor), since only relative probabilities are required for sampling. The user is expected to replace the body of this function with their own desired spectral shape and to call it from within `custom_photon_sampler` using an acceptance–rejection Monte Carlo sampling scheme (as is used throughout MCRaT; see Section 4.2) in order to draw a frequency, convert it to an energy via $E = h\nu$, and assign it to the returned photon.

5.1.5 A Minimal Example

The default implementation shipped with MCRaT samples a frequency from a fixed-temperature Blackbody photon-number spectrum by acceptance–rejection sampling and assigns only the comoving energy:

```
custom_photon.comv_p0 = fr_dum * PL_CONST / C_LIGHT;
```

where `fr_dum` is the sampled frequency in Hz. Because only `comv_p0` is set, MCRaT assigns the photon's direction using its default prescription and computes the lab-frame 4-momentum by boosting with the fluid velocity of the element. A user wishing to inject, for instance, the 6.4 keV Gaussian line described above would instead sample `custom_spectrum` within the acceptance–rejection loop and set `comv_p0` to the corresponding energy.

5.1.6 Enabling Custom Photon Injection

To use the custom photon sampler, the user sets the spectrum type in the first line of the `[MCRaT Photon Block]` of the `mc.par` file to `c` (or, in fact, to any character other than `w` for Wien or `b` for Blackbody: MCRaT's `photonInjection` routes every non-Wien, non-Blackbody spectrum code to `custom_photon_sampler` through its fall-through branch). This causes `photonInjection` to route photon generation through `custom_photon_sampler`, as described above. The minimum and maximum numbers of photons and all other `mc.par` entries behave exactly as described in Section 2.4.

5.2 Custom Analytic Outflows (`custom_outflow.c`)

The `custom_outflow.c` file allows the user to impose an arbitrary analytic outflow onto the fluid quantities of each hydro frame. This mechanism is identical in spirit to the spherical and cylindrical outflow test problems described in Section 3.2: MCRaT reads a hydro frame into memory and then overwrites the fluid quantities (in memory only, never in the hydro file on disk) with the values dictated by the user's analytic prescription. This lets the user explore idealized outflow structures, or impose a known analytic solution for validation, without needing a dedicated hydro simulation.

5.2.1 The `customOutflowPrep` Function

The file provides a single function with the signature

```
void customOutflowPrep(struct hydro_dataframe *hydro_data, FILE
*fPtr);
```

whose arguments are:

- `hydro_data` — a pointer to the `hydro_dataframe` structure for the current frame. On entry, this structure has already been populated by MCRaT with the coordinates (`r0`, `r1`, `r2`), the spherical radius `r`, and the cell sizes of every hydro element, as read from the hydro file. The number of hydro elements is available as `hydro_data->num_elements`. On exit, the function is expected to have overwritten the fluid quantities of every element with the user's desired values.
- `fPtr` — the log file pointer for the calling MPI process, which the user may write diagnostics to.

The function loops over every hydro element, from `i=0` to `hydro_data->num_elements`, and, for each element, assigns the fluid quantities required by the rest of MCRaT. The fields that the user *must* set for each element are:

- `(hydro_data->gamma)[i]` — the bulk Lorentz factor of the fluid element.
- `(hydro_data->dens)[i]` — the comoving (fluid-frame) mass density of the element, in g cm^{-3}.
- `(hydro_data->dens_lab)[i]` — the lab-frame mass density of the element. For a flow with bulk Lorentz factor Γ, this is $\rho_\mathrm{lab} = \Gamma\rho$, where ρ is the comoving density.

- `(hydro_data->pres)[i]` — the comoving pressure of the element. Consistent with the assumptions described in Section 3.2, the pressure is taken to be radiation dominated, so it is related to the comoving temperature T' by $p = \frac{1}{3}aT'^4$, where a is the radiation density constant.

- `(hydro_data->temp)[i]` — the comoving temperature of the element, in Kelvin.

- **The velocity components.** The user must set the fluid three-velocity, normalized by the speed of light, in the components `(hydro_data->v0)[i]`, `(hydro_data->v1)[i]`, and (in 2.5D and 3D) `(hydro_data->v2)[i]`. The components correspond to the coordinate directions of the chosen `GEOMETRY` (see the coordinate table in Section 2.4). In the 2.5D case, the third velocity component must be explicitly set to zero.

5.2.2 Consistency Requirements

Because MCRaT builds the injected photon spectrum and the scattering physics directly from these fluid quantities, the user must ensure that the prescription is internally consistent:

- The temperature and pressure must satisfy the radiation-dominated relation $p = \frac{1}{3}aT'^4$, since MCRaT assumes this throughout (Section 3.2). The example implementation enforces this automatically by computing the pressure from the chosen comoving temperature.

- The lab-frame density `dens_lab` must be consistent with the comoving density `dens` and the bulk Lorentz factor `gamma` via $\rho_\text{lab} = \Gamma\rho$.

- The velocity components, once normalized by c, must yield a speed $\beta < 1$ that is consistent with the assigned `gamma`, i.e. $\Gamma = 1/\sqrt{1-\beta^2}$.

- All velocities are expressed in units of c, matching the first assumption MCRaT makes about the hydro simulation (Section 3.2).

5.2.3 Interaction with the Magnetic Field, Non-Thermal Electrons, and Scattering Bias

A critical intricacy arises when a custom outflow is combined with the non-thermal synchrotron/scattering machinery (i.e. when `NONTHERMAL_E_DIST` is set to a power-law or broken power-law distribution, and/or `SCATTERING_BIAS_SWITCH` is enabled). For every *other* simulation type, MCRaT automatically derives the quantities needed by the non-thermal physics immediately after a hydro frame is read. For a custom outflow, however, part of this automatic derivation is deliberately *skipped*, and the user's `customOutflowPrep` function becomes responsible for supplying the skipped quantities. Specifically, after reading (or, for a custom outflow, synthesizing) each frame, MCRaT executes the following logic in its `getHydroData` routine:

```
   #if NONTHERMAL_E_DIST != OFF && SIMULATION_TYPE != CUSTOM_OUTFLOW
calculateElectronDistSubgroupDens(hydro_data->electron_dens_subgroup, fPtr);
calculateNonthermalElectronDens(hydro_data, fPtr);
#endif
#if SCATTERING_BIAS_SWITCH != OFF
```

```
calculateAverageDimlessTheta(hydro_data, fPtr);
#endif
```

The key observation is that the first conditional block carries the additional guard `SIMULATION_TYPE != CUSTOM_OUTFLOW`, whereas the scattering-bias block does not. The consequences, which the user must account for, are as follows.

Non-thermal electron density is *not* auto-populated. For all non-custom simulation types, `calculateNonthermalElectronDens` fills the per-cell non-thermal electron number density `hydro_data->nonthermal_dens` by equating a user-specified fraction (`NONTHERMAL_CONVERSION_` of the local magnetic energy density $B^2/8\pi$ to the rest-mass energy density of the non-thermal electron population; it also enforces a baryonic-loading sanity check that aborts the run if `NONTHERMAL_CONVERSION_FACTOR` is so large that the non-thermal electrons would carry more rest-mass energy than the magnetic field energy density. **For a custom outflow this routine is not called.** Therefore, if the user intends to use the non-thermal synchrotron emission or non-thermal scattering physics with a custom outflow, the user's `customOutflowPrep` function *must itself* populate `hydro_data->nonthermal_dens[i]` for every hydro element `i`, in units of cm^{-3} and in the comoving frame. If this field is left unset, it will retain whatever value it held previously (or its allocation default) and the non-thermal physics will produce meaningless results. The user is free to adopt the same prescription that MCRaT uses internally,

$$n_{e,\mathrm{nth}} = f_e \frac{B^2/8\pi}{\langle\gamma\rangle m_e c^2},$$

where f_e is the fraction of magnetic energy density placed into the non-thermal electrons and $\langle\gamma\rangle$ is the mean Lorentz factor of the chosen distribution, or any other prescription appropriate to their problem.

The magnetic field must be provided by the user. Because the custom outflow bypasses the automatic non-thermal density calculation, and because that calculation is where MCRaT ordinarily evaluates the magnetic field via `getMagneticFieldMagnitude`, the user must ensure that whatever magnetic field their physics requires is consistent with how `B_FIELD_CALC` is set:

- If `B_FIELD_CALC` is set to `INTERNAL_E` or `TOTAL_E`, MCRaT derives the magnetic field from the fluid internal/total energy density (see Section 4.5). In this case the user only needs to ensure that the fluid quantities they set in `customOutflowPrep` (in particular `pres`, `dens`, and `gamma`) are physically consistent, since the field is computed from them together with `EPSILON_B`.
- If `B_FIELD_CALC` is set to `SIMULATION`, MCRaT expects the magnetic field components `hydro_data->B0[i]`, `hydro_data->B1[i]`, and (in 2.5D/3D) `hydro_data->B2[i]` to be populated. Since a custom outflow has no underlying magnetohydrodynamic field to read, the user's `customOutflowPrep` function *must* assign these components explicitly for every element, in Gaussian units, exactly as the PLUTO and FLASH readers do when `B_FIELD_CALC == SIMULATION`. The example `customOutflowPrep` shipped with MCRaT does *not* set these components; this is a deliberate gap that the user must fill for their own outflow.

The non-thermal subgroup fractions are *not* auto-populated. The companion routine `calculateElectronDistSubgroupDens` computes the array `hydro_data->electron_dens_subg` which holds the *fraction* of the non-thermal electron distribution that lies within each of the `N_GAMMA` Lorentz-factor subgroups (these fractions are later multiplied by the per-cell `nonthermal_dens` to obtain the physical subgroup densities used in the scattering cross sections). This routine depends only on the compile-time distribution parameters (`POWERLAW_INDEX`, `GAMMA_MIN`, `GAMMA_MAX`, etc.) and not on the hydro frame, so it is computed once per frame alongside the density. Because it sits inside the same guarded block, it too is skipped for a custom outflow. A user who requires the non-thermal subgroup densities with a custom outflow must therefore also populate `hydro_data->electron_dens_subgroup` themselves, most conveniently by calling `calculateElectronDistSubgroupDens(hydro_data->electro fPtr)` from within `customOutflowPrep`, which reproduces exactly the behaviour that MCRaT applies for the non-custom simulation types.

The scattering-bias temperature average *is* still computed. By contrast, the volume-averaged dimensionless temperature $\bar{\Theta}$, stored in `hydro_data->average_dimless_theta` and used in computing the thermal scattering bias (Section 4.5), is computed by `calculateAverageDimle` under a guard that depends *only* on `SCATTERING_BIAS_SWITCH` and carries *no* custom-outflow exclusion. It is therefore evaluated normally for a custom outflow, using the temperatures that `customOutflowPrep` assigns. The user does not need to populate `average_dimless_theta` manually, but must ensure that the comoving temperatures they assign to `hydro_data->temp` are physical, since the scattering bias (and hence the sampling of rare, high-energy scatterings) depends on them through $\bar{\Theta}$.

Summary of the user's additional responsibilities for a custom outflow. When using a custom outflow *together with* the non-thermal and/or scattering-bias machinery, the user's `customOutflowPrep` function should, in addition to the standard fluid quantities listed above, set the following for every hydro element:

- `hydro_data->nonthermal_dens[i]` — the comoving non-thermal electron number density (never auto-populated for a custom outflow);
- `hydro_data->B0[i]`, `B1[i]`, and (in 2.5D/3D) `B2[i]` — the magnetic field components, *only* if `B_FIELD_CALC == SIMULATION`;
- `hydro_data->electron_dens_subgroup` — the per-subgroup distribution fractions, if the non-thermal scattering physics is used (most easily by calling `calculateElectronDistSubgro` from within `customOutflowPrep`).

The quantity `hydro_data->average_dimless_theta` is handled automatically and should *not* be set by the user. Note also that, because `calculateNonthermalElectronDens` is skipped, the baryonic-loading sanity check on `NONTHERMAL_CONVERSION_FACTOR` that it performs is likewise skipped for a custom outflow; the user is responsible for ensuring that their chosen non-thermal electron density is physically sensible relative to the local magnetic energy density.

5.2.4 The Example Implementation

The implementation shipped with MCRaT provides a template analytic outflow and should be adapted by the user to their problem of interest. It defines a radial outflow

characterized by a small number of parameters: an outflow slope parameter, a fixed comoving temperature corresponding to 3 keV, a mass-outflow normalization, an initial optical depth τ_0, and a base radius r_0. From these it constructs, for each element, a radius-dependent velocity profile, a radius-dependent density profile, the corresponding bulk Lorentz factor $\Gamma = 1/\sqrt{1-\beta^2}$, the radiation-dominated pressure from the fixed temperature, and the lab-frame density.

> *Note on the slope parameter.* In the example template, the parameter named `beta` in the source is the *density/velocity profile exponent*, and is *not* the relativistic $\beta = v/c$. The relativistic velocity is computed separately from the velocity profile. A user adapting this template should take care not to conflate the two.

The user is strongly encouraged to use this example as a starting point, replacing the analytic profiles with those appropriate to their science case, while retaining the overall structure of looping over every hydro element and assigning each of the required fluid quantities listed above.

5.2.5 Enabling the Custom Outflow

To use a custom outflow, the user sets the `SIMULATION_TYPE` parameter in the `mcrat_input.h` file to `CUSTOM_OUTFLOW` and recompiles the code. When this simulation type is selected, MCRaT calls `customOutflowPrep` immediately after reading each hydro frame into memory, so that the user's analytic fluid quantities replace those read from the hydro file before any photon injection, propagation, or scattering takes place. Because the geometry and coordinates of the underlying hydro frame are preserved, any hydro simulation file (for example, the same files used for the spherical or cylindrical test outflows) may be used as the coordinate scaffold onto which the custom outflow is imposed. All other aspects of running MCRaT — the `mc.par` file, the parallelism, the output files, and the post-processing — behave exactly as described in the preceding sections.

References

Abramowicz, M. A., Novikov, I. D., & Paczynski, B. 1991, The Astrophysical Journal, 369, 175

Canfield, E., Howard, W. M., & Liang, E. P. 1987, The Astrophysical Journal, 323, 565

Depaola, G. 2003, Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment, 512, 619

Dexter, J., McKinney, C., & Agol, E. 2012, Monthly Notices of the Royal Astronomical Society, 421, 1517

Dolence, J. C., Gammie, C. F., Mościbrodzka, M., & Leung, P. K. 2009, The Astrophysical Journal Supplement Series, 184, 387

Ito, H., Nagataki, S., Matsumoto, J., et al. 2014, The Astrophysical Journal, 789, 159

Kawashima, T., Ohsuga, K., & Takahashi, H. R. 2023, The Astrophysical Journal, 949, 101

Krawczynski, H. 2011, The Astrophysical Journal, 744, 30

Lazzati, D. 2016, The Astrophysical Journal, 829, 76

Lundman, C., Pe'er, A., & Ryde, F. 2014, Monthly Notices of the Royal Astronomical Society, 440, 3292

Mathews, K. 2013, Nuclear Science and Engineering, 173, 207

McMaster, W. H. 1961, Reviews of Modern Physics, 33, 8

Parsotan, T., & Lazzati, D. 2018, The Astrophysical Journal, 853, 8

Parsotan, T., López-Cámara, D., & Lazzati, D. 2018, The Astrophysical Journal, 869, 103

—. 2020, The Astrophysical Journal, 896, 139

Rybicki, G. B., & Lightman, A. P. 1979, Radiative processes in astrophysics

Rybicki, G. B., & Lightman, A. P. 1979, Radiative Processes in Astrophysics (New York: Wiley)

Table 2: Emission call chain.

Function	**File**	**Physics / role**
`initSynchTables`	`mc_synchrotron.c`	Build $F(x)$ (33), $G_a(x)$ (35), emission and pitch-angle inverse CDFs, and call `buildUniversalNuCDF` to construct the single, universal photon-number CDF (Section 4.6.5) and $\mathcal{I}$ (`nu_cdf_norm`) for $K_{\rm phys}$ (46).
`buildUniversalNuCDF`	`mc_synchrotron.c`	Evaluate the photon-number pdf (65) on a log-spaced $\tilde{\nu}$ grid by trapezoid integration over $\log_{10}\gamma$; integrate cumulatively to form the universal CDF($\tilde{\nu}$) (stored as `nu_cdf_u`/`nu_cdf_log_nu_tilde`) and the stratum probability masses p_k (66) (called once at startup, not per cell).
`photonEmitSynch`	`mc_synchrotron.c`	Compute $K_{\rm phys}$ (46); evaluate $\Lambda_i = K_{\rm phys}\, n_{e,\rm nth}^{(i)}\, B_i\, V_i$ (48); weight-tuning (62); Poisson draw (63).
`emitCellPackets`	`mc_synchrotron.c`	Divide n_i packets across strata; importance weight $w_k = w_0\, p_k\, K$ (67); call `sampleSynchFrequency` and `synchFillPhoton`.
`sampleSynchFrequency`	`mc_synchrotron.c`	Draw ν_f from stratum k via inverse interpolation of the universal CDF built by `buildUniversalNuCDF` (Section 4.6.5), then rescale by $\nu_c(B_{\rm cell})$.
`synchFillPhoton`	`mc_synchrotron.c`	Comoving 4-momentum, Lorentz boost, birth position, Stokes parameters; `recalc_properties` $= 1$ (Sec. 4.6.5).

Table 4: Absorption call chain.

Function	**File**	**Physics / role**
`calculateOpticalDepth`	`optical_depth.c`	Compute fluid_factor $= 1 - \beta\cos\theta$ (93); apply to scattering; call `calculateOpticalDepthSSA` with fluid_factor.
`calculateOpticalDepthSSA`	`synchrotron.c`	Evaluate $\alpha_{\nu_f}^{(f)}$ at the *comoving*-frame density $n_{e,\mathrm{nth}}^{(\mathrm{comv})}$ via `synchAlphaNu`; multiply Γ directly onto the resulting coefficient (not onto the density) to store `absorption_opacity` $= \Gamma\,(1-\beta\cos\theta) \times \alpha_{\nu_f}^{(f)}$ (91), giving full $\Gamma(1-\beta\cos\theta)$ (Abramowicz et al., 1991). (Contrast with the lab-density conversion $n_{e,\mathrm{nth}}^{(\mathrm{lab})} = \Gamma\, n_{e,\mathrm{nth}}^{(\mathrm{comv})}$ (92) used for the *scattering* opacity instead.)
`synchAlphaNu`	`mc_synchrotron.c`	Evaluate $\alpha_{\nu_f}^{(f)}$ (82) or (84) using $G_a(x)$ splines (35).
`applyAbsorption`	`mc_synchrotron.c`	$w_{\mathrm{new}} = w_{\mathrm{old}}\exp(-$`absorption_opacity` $\times\, dl)$ (98) at every transport sub-step; expands to full Abramowicz et al. (1991) (99).

Table 5: Scattering bias and subgroup selection call chain.

Function	File	Physics / role
`calculateThermalScatteringBias`	`optical_depth.c`	Compute b_{th} (113) for thermal electrons; also used as b_{ref} in the purely non-thermal case (121).
`calculateNonthermalScatteringBias`	`optical_depth.c`	*Design intent*: $b_k = \max(1, b_{\text{ref}}\,\tau_{\text{ref}}/\tau_k)$ (115) for each subgroup; $\tau_{\text{ref}} = \tau_0$ (mixed case) or $\tau_{\text{nth,total}}$/`N_GAMMA` (pure non-thermal case) (120). **Current behaviour**: the `fmax(1.0,...)` floor is commented out (see erratum, Section 4.6.8); the function returns the unclamped ratio $b_{\text{ref}}\tau_{\text{ref}}/\tau_k$.
`generateSingleElectron`	`electron.c`	Select subgroup j by forward CDF scan of $b_j\tau_j/\tau_{\text{total}}$ (131); sample electron from `singleThermalElectron` or `singleNonThermalElectron`.
`scatteredPhotonWeight`	`mcrat_scattering.c`	Compute w_{scatt} (127) using $\tau_j \cdot dl$ and $b_j\tau_j \cdot dl$ for the selected subgroup j only.

Table 6: Russian roulette call chain.

Function	File	Physics / role
`applyRussianRoulette`	`mclib.c`	Thin wrapper: calls `applyRussianRouletteByType` three times, for `SYNCH_PHOTON`, `INJECTED_PHOTON`, and `COMPTONIZED_PHOTON` (the latter with threshold $10^{-8}\varepsilon_{\rm RR}$). Triggered at two locations in the main loop: (i) every 1000 scattering events inside the transport loop; and (ii) near the end of an intermediate hydro frame when `scatt_frame != scatt_framestart` and `scatt_frame != last_frm`, immediately before the corresponding rebinning checks.
`applyRussianRouletteByType`	`mclib.c`	Implements the roulette procedure for a single photon type and threshold fraction. Step 1 (parallel): collect weights of live packets of the given type into per-thread sub-arrays. Step 2 (serial): sort, compute $w_{\rm median}$ (138), set $w_{\rm thresh} = \varepsilon_{\rm RR}\, w_{\rm median}$ (137). Step 3 (parallel): for each packet with $w < w_{\rm thresh}$, kill with probability $1 - w/w_{\rm thresh}$ or boost to $w_{\rm thresh}$ (135)–(136). Killed packets → `setNullPhoton`. Returns the number of packets killed of the given type.

Table 7: Electron-distribution normalisation and hot-cross-section support functions.

Function	**File**	**Physics / role**
`powerLawNorm`	`electron.c`	Compute $A_{\rm norm}$ (23) for a single power law. Takes no $n_{e,\rm nth}$ argument (see notation glossary).
`brokenPowerLawNorm`	`electron.c`	Compute $A^{(\rm br)}_{\rm norm}$ (26) for a broken power law, including the continuity factor $\gamma_{\rm br}^{p_2-p_1}$ (24).
`calculateNonthermalElectronDens`	`electron.c`	Compute $n_{e,\rm nth}$ (30) in every cell from B, f_e, and $\langle\gamma\rangle$; enforces the energy-budget guard of Section 4.6.3.
`initalizeHotCrossSection`	`hot_x_section.c`	Build (or load from disk) the thermal and non-thermal hot cross-section tables (100)–(104) (Section 4.6.7); MPI rank 0 only, then broadcast.
`interpolateThermalHotCrossSection`	`hot_x_section.c`	Bilinear interpolation of σ/σ_T at runtime, using the per-thread `ThreadLocalAccel` pattern (Section 4.6.4).
`interpolateSubgroupNonThermalHotCrossSection`	`hot_x_section.c`	Same as above, for each non-thermal Lorentz-factor subgroup k used in Eq. (104).